



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture – 01

BITS Pilani
Pilani Campus

Harish Aggarwal

Work Integrated Learning Programmes

1



BITS Pilani
Pilani Campus

Essential SQA: Processes and Success Factors

Work Integrated Learning Programmes

2

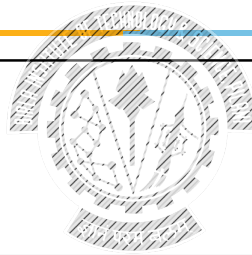
Agenda



- Definition
- Quality Assurance Vs Quality Control
- Business Models
- Cost of Quality
- Quality culture
- Success Factors

BITS Pilani, Pilani Campus

3



Ground Rules!



- Be Regular
- Be Punctual.
 - 5:50 means 5:50 (Not 5:55)
- Enjoy Learning
- Mentally present – Observe!! Listen!!
- Keep your questions for the Q&A section ...
- Have Discussions
 - Discuss for the sake of clarity not for the sake of proving ☺.

4

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

4

Evaluation Components



EC-1	Marks	Time Lines
Assignment - 01	7	After L-5
Quiz-01	7	After L-6
Assignment - 02	8	After L-11
Quiz - 02	8	After L-12
EC-2 (Mid-Sem)	30	Closed Book
EC-3 (Compre)	40	Open Book

** EC-1 subdivision is subject to change

BITS Pilani, Pilani Campus

5

Software - Definition



Development Life Cycle

Software life cycle is process that contains the activities of requirements analysis, design, coding, integration, testing, installation, and support for acceptance of software products.

ISO 12207 [ISO 17]

BITS Pilani, Pilani Campus

6

Software Quality Definition



Software Quality

- Conformance to established software requirements; the capability of a software product to satisfy stated and implied needs when used under specified conditions (ISO 25010 [ISO 11i]).
- The degree to which a software product meets established requirements; however, quality depends upon the degree to which those established requirements accurately represent stakeholder needs, wants, and expectations [(IEEE 730)] [IEE 14].

(Needs, Wants, Expectations can include - Functionalities, performance, efficiency, accurate results, reliability, usability, costs, deadlines, etc.)

(True needs, Expressed needs, Specified needs, Achieved needs)

BITS Pilani, Pilani Campus

7

Software Quality Assurance Definition



Software Quality Assurance

- The systematic application of scientific and technological knowledge, methods, and experience for the design, implementation, testing, and documentation of software. ISO 24765 [ISO 17a]
- A set of activities that define and assess the adequacy of software processes to provide evidence that establishes confidence that the software processes are appropriate for and produce software products of suitable quality for their intended purposes. A key attribute of SQA is the objectivity of the SQA function with respect to the project. The SQA function may also be organizationally independent of the project; that is, free from technical, managerial, and financial pressures from the project. [(IEEE 730)]

BITS Pilani, Pilani Campus

8

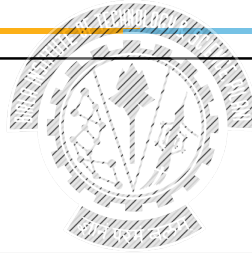
QA Vs QC



- **Quality Assurance (QA)** refers to process management activities which are aimed at reducing defects and errors for the end customer. QA involves looking at how the processes are performed and making sure that the quality requirements are being fulfilled.
 - It is a proactive, process-based approach which aims to manage the quality of a product before and during the production process.
- **Quality control (QC)** is concerned with the end product and seeks to ensure that it is not defective or damaged before reaching the customer.
 - QC is a reactive process which is employed after the product has been created to verify its quality.

BITS Pilani, Pilani Campus

9



QA Vs Testing



- **Software Quality Assurance**
 - It is the process of ensuring that the software meets the set standards of quality.
 -
- **Software Testing**
 - It is the process of identifying and verifying that software applications or programs will meet the user's requirements.

Both processes are essential for delivering a high-quality product

BITS Pilani, Pilani Campus

10

QA Vs Testing



	Quality Assurance	Testing
Definition	A way to prevent mistakes and defects in the manufactured products.	The process of executing a system with the intent of finding defects.
Objective	To improve development and test processes to prevent defects from arising when the product is being developed.	To identify bugs and defects in the software.
Focus	Focuses on improving and optimizing the processes involved in the software development lifecycle.	Centers on verifying that the system meets specified requirements and finding bugs and issues.
Scope	Covers the entire software development process.	Limited to the testing phase of the software development lifecycle.
Activities Involved	Involves process management, setting quality objectives, and process optimization.	Involves test case execution, bug reporting, and ensuring that the software behaves as expected.
Responsibility	Quality Assurance teams should ensure that processes are effective and efficient as per quality standards.	Software testers are responsible for executing test cases and reporting defects.
Outcome	Ensures that the best practices and standards are followed throughout the software development lifecycle.	Ensures that the software is bug-free and works as expected.
Tools Used	Utilizes tools for process monitoring, project management, and process documentation.	Utilizes tools designed for test case management, bug tracking, and test automation.
Timing	Ongoing throughout the software development lifecycle.	Performed after the software is developed and before it is released.

BITS Pilani, Pilani Campus

11

Why QA is important



- Reduces Cost and Saves time
- Maintains Quality of the Product
- Ensures that product is Secure
- Improves accessibility and usability
- Improves Performance
- Increases customer satisfaction
- Protects company's Reputation

BITS Pilani, Pilani Campus

12

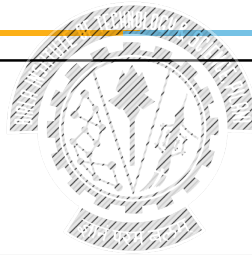
Risks of neglecting QA



- **Software Development Risks**
 - Undetected code errors and Bugs
 - System misbehaviors
 - Poor system security
 - Unstable performance
- **Business Risk**
 - Missed deadlines
 - Financial losses
 - Reputational damage
 - Unsatisfied customer

BITS Pilani, Pilani Campus

13



Software Components



Overall software quality can be achieved if quality of every component of software is achieved.

Thus it is important to know the components of Software –

- **Programs** – The instruction that have been translated into source code
- **Procedures** – User procedures and other processes
- **Rules** – Business Rules
- **Documentation** – Design Docs, Test Cases, User Manual, etc
- **Data** – Information that is inventoried, modeled, created.

BITS Pilani, Pilani Campus

14

Causes of Software Defect



Popular error-cause categories:

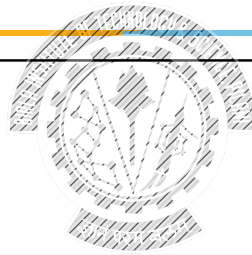
1. Problems with defining requirements
2. Maintaining effective communication between client and developer
3. Deviations from specifications
4. Architecture and design errors
5. Coding errors (including test code)
6. Non-compliance with current processes/procedures
7. Inadequate reviews and tests
8. Documentation errors

Caused by –

- Clients, Analysts, Designers, Software engineers, Testers, or Users.

BITS Pilani, Pilani Campus

15



Causes of Software Defect



1. Problems with defining requirements
 - Good requirements are - Correct, complete, clear for each stakeholder group (e.g., the client, the system architect, testers, etc), unambiguous, concise (simple, precise), consistent, feasible (realistic, possible), necessary, independent of the design, independent of the implementation, verifiable and testable, can be traced back to a business need, unique.
2. Lack of effective communication between client and developer
 - Poor documentation, ineffective change management process, etc.
3. Deviations from specifications
 - Reuse of existing code, trimming of partial requirements, etc.
4. Architecture and design errors
 - Incomplete overview of S/W to be developed, incorrect business or technical process sequence, poor design of business or process rule criteria.

BITS Pilani, Pilani Campus

16

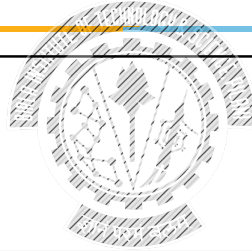
Causes of Software Defect



5. Coding errors (including test code)
 - Inappropriate choice of programming language and conventions
 - Poor understanding/interpretation of design documents
 - Incoherent abstractions, Boundary condition errors.
6. Non-compliance with current processes/procedures
 - Non-compliance of Organizations internal methodology including processes, procedures, steps, deliverables, templates, and standards (coding standard).
7. Inadequate reviews and tests
 - Design review, code review, Test plan and Test case review.
8. Documentation errors
 - Incomplete, obsolete, outdated documentation.

BITS Pilani, Pilani Campus

17



S/W delivery Issues



Issues faced by S/W developers (Organization) :

- Incorrect results
- Exceeding budget
- Penalties for late delivery
- Legal Proceedings - Not delivering what the client asked for
- Missing Market Opportunity
- Bad Reviews in Press
- High level of Support calls
- Tarnishing reputation of the organization.

BITS Pilani, Pilani Campus

18

S/W delivery Assumptions and Practices



Assumptions and facts for S/W Delivery -

- Delivery on schedule and within budget is crucial
- Reliable, correct software is crucial
- Requirements must be known and detailed from the project onset
- Projects are typically large scale with many communication channels
- It is necessary to show that what was promised has indeed been delivered
- Plans must be developed, and regular progress reports prepared (which are sent to project management and the client).

Practices to be Followed -

- Lots of documentation
- Practices followed in Estimating and Management
- Waterfall development Cycle
- Project Audits

BITS Pilani, Pilani Campus

19

Business Models in S/W



- Custom systems written on contract
 - Accenture, TATA, and Infosys.
- Custom software written in-house : To improve organizational efficiency
 - Internal IT organization.
- Commercial software: B2B
 - e.g., Oracle and SAP.
- Mass-market software: B2C
 - e.g., Microsoft and Adobe.
- Commercial and mass-market firmware: Embedded hardware and systems
 - e.g. Digital cameras, automobile, braking system, and airplane engines.

BITS Pilani, Pilani Campus

20

Business Models in S/W



Factors in each Business Model

- Criticality
- Uncertainty of users' wants and needs
- Range of environments
- Cost of fixing errors
- Regulations
- Project size
- Communication
 - Concurrent developer-developer communication:
 - Developer-maintainer communication
 - Communication between managers and developers
- Organization's culture –
 - Control culture,
 - Skill culture,
 - Collaborative culture,
 - Thriving culture

BITS Pilani, Pilani Campus

21

Business Models in S/W Contd.



Open-Source Software: Distributed with its source code and the authorization to modify and distribute it freely under the condition that it is also provided as open-source software once modified.

Concerns

- Undemonstrated quality
- Lack of support
- Delays in providing fixes

BITS Pilani, Pilani Campus

22

Software Cost



Cost of Project can be grouped into following buckets

- Implementation costs
- Prevention Costs
- Appraisal Costs (Testing)
- Costs associated with failures or anomalies.

If implementation is 100% correct, the cost of the project is implementation cost and there is no need to incur any cost for quality.

BITS Pilani, Pilani Campus

23

Software Quality Cost



Cost of Software Quality

- Prevention Costs
 - Training to Employees in implementation, review, etc.
 - Better processes. Setting quality goals, standards, thresholds.
 - Cost of better tools and technology used for implementation
- Appraisal Costs
 - Testing cost (Verification and validation)
 - Cost for Testing tools and technologies
- Failures Cost
 - Internal – During development - Rework, Redesign.
 - External – At Client's premises.
 - Replacement cost.
 - Managing Disputes.
- Warranty claims, Loss of reputation.

Costs ↓ / Quality →	Low	High
Prevention Costs	Low	High
Appraisal Costs	Low	High
Failures Cost	High	Low
Warranty claims, Loss of reputation	High	Low

Need to settle at Optimal cost and optimal quality

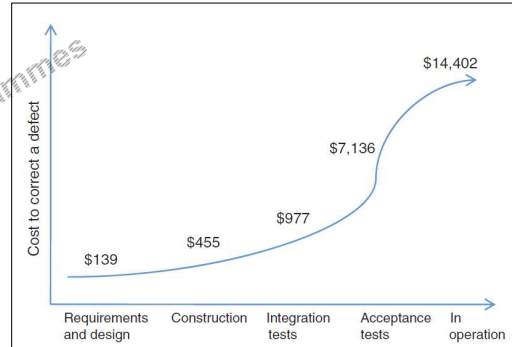
BITS Pilani, Pilani Campus

24

Software Quality Cost



Cost of Fixing Defect

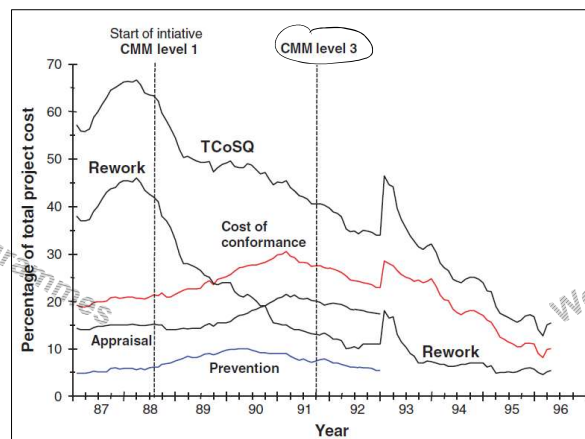


Cost of Propagating an Error
[Src - Caper Jones]

BITS Pilani, Pilani Campus

25

Software Quality Cost



Cost of Software Quality
[HAL-96]

BITS Pilani, Pilani Campus

26

Software Quality Cost

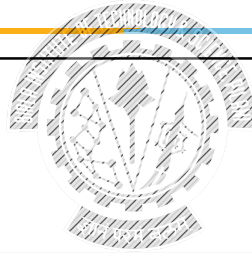


Process maturity	Rework (percent of total development effort)
Immature	$\geq 50\%$
Project controlled	25% – 50%
Defined organizational process	15% – 25%
Management by fact	5% – 15%
Continuous learning and improvement	$\leq 5\%$

Relationship Between the Process Maturity Characteristic and Rework [KRA 98]

BITS Pilani, Pilani Campus

27



Quality Culture



Cultural Principles in Software Engineering

1. Do not compromise quality due to cost or deadlines. Quality is the number one priority.
2. Quality Work is appreciated.
3. Continuing education and learning.
4. Participation of the client in S/W development process.
5. Share the vision of the final product with the client.
6. Continuous improvement in your software development process.
7. Ensure that it is a peer, not a client, who finds the defect.
8. Repeatedly go through all development steps except coding. coding should only be done once.
9. Controlling error reports and change requests is essential to quality and maintenance.
10. If you measure what you do, you can learn to do it better.
11. Not everything can be changed, Identify and work on changes that will reap the most benefits

BITS Pilani, Pilani Campus

28

Factors that Foster Software Quality



1. Good team spirit.
2. The skills of the members in the organization – Competency of Team
3. Managers who set a good example.
4. Effective communication between colleagues, managers, and the client.
5. Recognizing and valuing initiatives to improve quality.
6. Highlighting the notion of organizational culture of guaranteeing quality.
7. Including the notion of culture in an organization's strategy.
8. Common goals and perception about quality between managers, software engineer and other members of the organization.
9. Client involvement throughout the project. Early clarity of requirements and removal of ambiguities. Clearly defined roles and responsibilities.

BITS Pilani, Pilani Campus

29

Factors that Foster Software Quality



11. Allocating the necessary budgets.
12. SQA techniques adapted to the environment.
13. Clear terminology with regards to software problems.
14. An understanding and specific attention to each major category of software error sources.
15. An awareness of the SQA body of knowledge of the SWEBOK as a guide for SQA
16. Good understanding of non-functional quality.
17. Good process for defining, following up, and communicating quality requirements.
18. Evaluating quality throughout the life cycle of the software.
19. Establishing software criticality before starting a project.
20. Using the benefits of software traceability.

BITS Pilani, Pilani Campus

30

Factors that adversely affect Software Quality



1. Give employees the responsibility but not the authority to take the actions necessary to ensure the project's success.
2. When we "shoot the messenger."
3. When managers hide their heads in the sand rather than solve problems.
4. Lack of knowledge in quality assurance.
5. Unrealistic time frames.
6. A lack of common working methodology between team members.
7. A manager who says yes to everyone
8. Not taking quality requirements into account.
9. Not taking software criticality into account.
10. Making excuses to not be concerned with quality.

BITS Pilani, Pilani Campus

31

Factors that adversely affect Software Quality



11. A lack of cohesion between SQA techniques and environmental factors in organization.
12. Confusing terminology used to describe software problems.
13. A lack of understanding or interest for collecting information on software error sources.
14. Poor understanding of software quality fundamentals.
15. Ignorance or non-adherence with published SQA techniques.

BITS Pilani, Pilani Campus

32



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture - 02

BITS Pilani
Pilani Campus

Harish Aggarwal

1



BITS Pilani
Pilani Campus

Quality Models

2

Re-Cap



- Lecture - 01
 - Definitions – S/W Quality, S/W Quality Assurance
 - Importance of QA
 - Causes of Defects
 - Business Models in S/W
 - S/W Cost, Quality Cost
 - Quality culture

BITS Pilani, Pilani Campus

3

Agenda



- Software Quality Models
 - McCall
 - IEEE 1061
 - ISO 25000 Series
- Quality Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes
 - CMMI Development
 - ITIL Framework

BITS Pilani, Pilani Campus

4

Quality Perspectives



Five quality perspectives as described by Garvin (1984) [GAR 84]

- **Transcendental approach to quality** : Although I can't define quality, I know it when I see it.
- **User-based approach** : As expected from the user's perspective. Can change with the expectations of each user
- **Manufacturing-based approach** : A "process based approach". Compliance with the process while defining the requirements and throughout the life cycle.
- **Product-based approach** : Quality of internal properties of the software components, for example, the quality of its architecture, quality of code, etc
- **Value-based approach** : Focuses on the elimination of all activities that do not add value.

BITS Pilani, Pilani Campus

5

Software Quality Model



Quality Model

- A defined set of characteristics, and of relationships between them, which provides a framework for specifying quality requirements and evaluating quality. ISO 25000 [ISO 14a]

A software quality model can be is used to

- Define software quality characteristics that can be evaluated;
- Define quality characteristics that will serve as the non-functional requirements
- Set a measure and its objectives for each of the quality requirements.

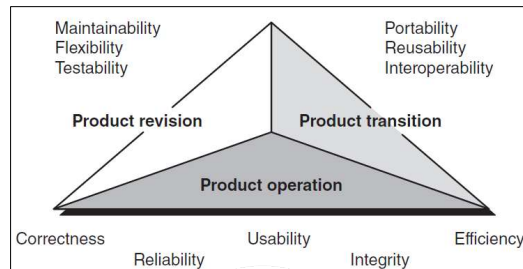
BITS Pilani, Pilani Campus

6

Quality Models – McCall [MCC 77]



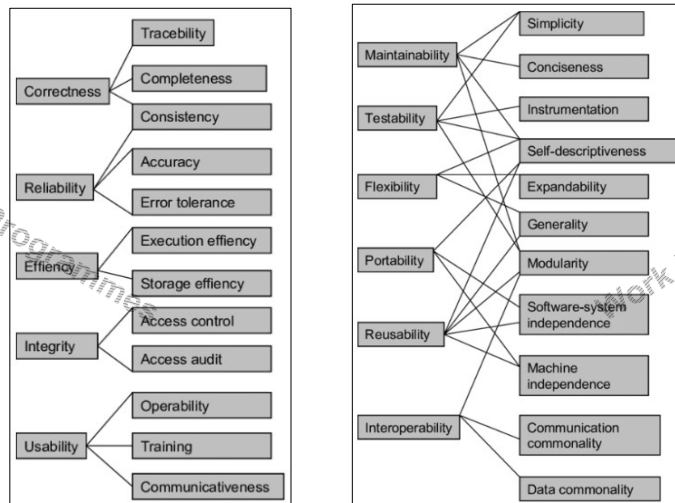
- Proposed by McCall [MCC 77], developed in the 1970s for the United States Air Force and was designed for use by software designers and engineers
- Uses Product based approach.
- More on Internal Quality
- Proposes three perspective for the user
- Each perspective into Quality Factor
- Quality Factor → Quality criteria (Measurable).



BITS Pilani, Pilani Campus

7

Quality Models – McCall [MCC 77]



BITS Pilani, Pilani Campus

8

Internal Quality Vs External Quality



A change request

- External view –
 - How soon it can be implemented, and whether the requirement is fulfilled or not.
- Internal View –
 - Effort to identify the change – Location and size.
 - Implementing the change
 - Testing the change
 - Documenting the change
 - Releasing the updated version

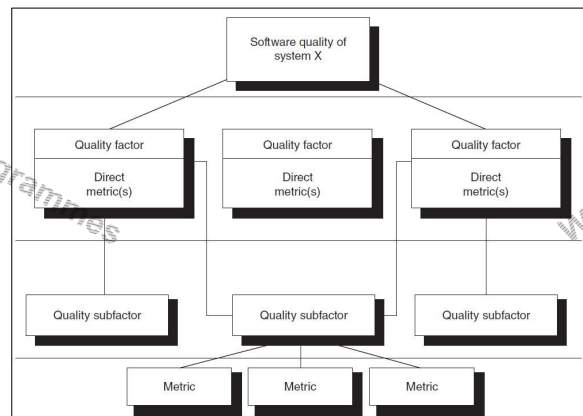
BITS Pilani, Pilani Campus

9

Quality Metrics – IEEE 1061



- 1061-1998 - IEEE Standard for a Software Quality Metrics Methodology
- Presents examples of measures without formally prescribing specific measures.



BITS Pilani, Pilani Campus

10

IEEE 1061 Stake Holders



This standard can be used by the following –

- **An acquisition/project manager** - To identify, define, and prioritize the quality requirements for a System.
 - For Software program acquisition. Measurements when adapting and releasing the software.
- **A designers and System developer** - To identify specific traits that should be built into the software in order to meet the quality requirements.
- **Quality assurance/quality control/audit Organization** – For outside team to evaluate whether the quality requirements are being met.
- **A system maintainer** - To assist in implementing modifications during product evolution. While making changes or upgrades to software
- **Client/user** – Allows User to State Quality characteristics and evaluate during user acceptance Test

BITS Pilani, Pilani Campus

11

IEEE 1061 Steps to be Followed



Steps to be followed in quality metrics methodology

1. Establish software quality requirements
2. Identify software quality metrics
3. Implement the software quality metrics
4. Analyze the software metrics results
5. Validate the software quality metrics

BITS Pilani, Pilani Campus

12

IEEE 1061 Steps to be followed



Steps to be followed

- List down non-functional quality requirements. Set a priority of these requirements
- Meet every stakeholder
- List down and resolve conflicts
- Quantify each quality factor. Measure of threshold and quality.
- Have thresholds approved.
- Perform cost-benefit study – Take into account additional cost to enter information, automate calculations, interpret and present the results, to modify and support software, for software assessment specialists, for specialized software to measure the software application, Training required to apply the measurement plan.
- Have clear measurement method – Process of measurement, Data collection method, areas of S/W which needs to be measured, etc
- Analyze Results – Analyze difference between measurements taken at different times and different environment.
- Validate the measures - Identify measures that can predict the value of quality factors, which are numeric representations of quality requirements
 - Model recommends techniques to Validate Measures.

BITS Pilani, Pilani Campus

13

IEEE 1061 Adoption



Adoption hesitancy

- Seen as being too expensive
- Some did not see its usefulness
- The industry was not ready to use it
- Suppliers did not want to be measured in this way by their clients.

Thus the adoption was not too high and it was not too popular outside of Military

BITS Pilani, Pilani Campus

14

ISO 25000 Series

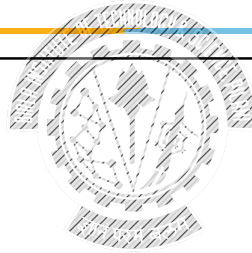


Introduction

- The international standardization of a software quality model, the ISO 9126 standard [ISO 01], was published for the first time in 1991
- Lots of terminology used from McCall [MCC 77] and IEEE 1061 [IEE 98b]
- International Organization for Standardization
- Replaced by the ISO/IEC 25000 standard [ISO 14a].
- The ISO 25000 [ISO 14a] standard allows for the evaluation of the quality of the final software product as it
 1. Assess the quality of the development process,
 2. Assess the quality of the final product.
- The ISO 25000's series of standards recommends the following four steps [ISO 14a]:
 - Set quality requirements
 - Establish a quality model
 - Define quality measures
 - Conduct evaluations

BITS Pilani, Pilani Campus

15



ISO 25000 Series



Introduction

- ISO/IEC 25000 series of standards, is also known as SQuaRE (System and Software Quality Requirements and Evaluation)
- A framework for the evaluation of software product quality



BITS Pilani, Pilani Campus

16

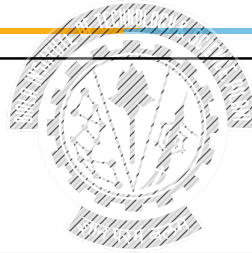
ISO 25000 Series



- ISO/IEC 2500n – Quality Management Division
 - ISO/IEC 25000 - Guide to SQaRE
 - ISO/IEC 25001 - Planning and Management
- ISO/IEC 2501n – Quality Model Division
 - **ISO/IEC 25010 - System and software quality models**
 - ISO/IEC 25012 - Data Quality model
- ISO/IEC 2502n – Quality Measurement Division
 - ISO/IEC 25020 - Measurement reference model and guide
 - ISO/IEC 25021 - Quality measure elements
 - ISO/IEC 25022 - Measurement of quality in use
 - ISO/IEC 25023 - Measurement of system and software product quality
 - ISO/IEC 25024 - Measurement of data quality

BITS Pilani, Pilani Campus

17



ISO 25000 Series

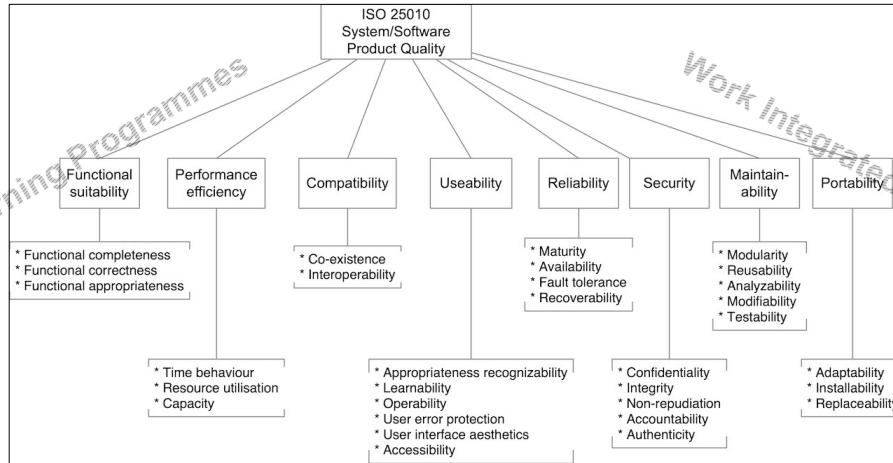


- ISO/IEC 2503n – Quality Requirements Division
 - ISO/IEC 25030 - Quality requirements
- ISO/IEC 2504n – Quality Evaluation Division
 - ISO/IEC 25040 - Evaluation reference model and guide
 - ISO/IEC 25041 - Evaluation guide for developers, acquirers and independent evaluators
 - ISO/IEC 25042 - Evaluation modules
 - ISO/IEC 25045 - Evaluation module for recoverability

BITS Pilani, Pilani Campus

18

ISO 25010



BITS Pilani, Pilani Campus

19

Requirement - Defn



Functional Requirement

- A requirement that specifies a function that a system or system component must be able to perform. ISO 24765 [ISO 17a]

Non-Functional Requirement

- A software requirement that describes not what the software will do but how the software will do it. Synonym: design constraint. ISO 24765 [ISO 17a]

Performance Requirement

- The measurable criterion that identifies a quality attribute of a function or how well a functional requirement must be accomplished. A performance requirement is always an attribute of a functional requirement. IEEE 730 [IEE 14]

BITS Pilani, Pilani Campus

20

Software Quality Requirements



In general, the requirements document form the basis of design and development of any product.

Steps for defining Requirements

- **Gather** : collect all wishes, expectations, and needs of the stakeholders;
- **Prioritize** : Prioritize based on relative importance of requirements (essential, desirable);
- **Analyze** : Check for consistency and completeness of requirements
- **Describe** : Can be easily understood by users and developers;
- **Specify** : Transform the business requirements into software specifications

Category of Requirements –

- **Functional Requirements**: Includes business and functional requirements for the user.
- **Non-Functional (Quality) Requirements**: quality characteristics and sub-characteristics such as security, confidentiality, integrity, availability, performance, and accessibility.
- **Constraints** : Limitations such as infrastructure on which the system must run or the programming language that must be used to implement the system.

BITS Pilani, Pilani Campus

21

Quality of Requirements



Good requirements should have following characteristics

- **Necessary** : Prioritized list as per the objective of what needs to be achieved.
- **Unambiguous**: Clear enough to be interpreted in only one way.
- **Concise**: Precise, brief, and easy to read.
- **Coherent**: Must not contradict the other requirements. Consistent terminology
- **Complete**: Stated fully in one location and convey completeness, not requiring any other references.
- **Accessible**: They must be realistic regarding their implementation in terms of available finances, available resources, and within the time available.
- **Verifiable**: Should be verifiable whether they are met or not based on four possible methods - inspection, analysis, demonstration, or tests.
- ISO 29148 or IEEE 830 is a complete section on Requirement engineering
 - <https://www.iso.org/obp/ui/#iso:std:iso-iec-ieee:29148:ed-2:v1:en>
 - <https://standards.ieee.org/ieee/830/1222/>

BITS Pilani, Pilani Campus

22

Software Quality Requirements



Quality requirements – Non functional requirements.

Should be well documented with details such as -

- Quality characteristic, Quality sub-characteristic
- Measure (i.e., formula)
- Objectives (i.e., target)
- Importance - "Indispensable", "Desirable" or "Non-applicable."

Steps suggested for defining non-functional requirements –

Identify stakeholders → Develop Questionnaire → Conduct Interview → Consolidate and prioritize results → Obtain consensus on quality factors

REQUIREMENT TRACEABILITY - Should be Traceable using documents such as –

- Specifications document
- Architecture and design document
- Code, and
- User Manuals

BITS Pilani, Pilani Campus

23

ISO/IEC/IEEE 12207 Software Life Cycle Processes

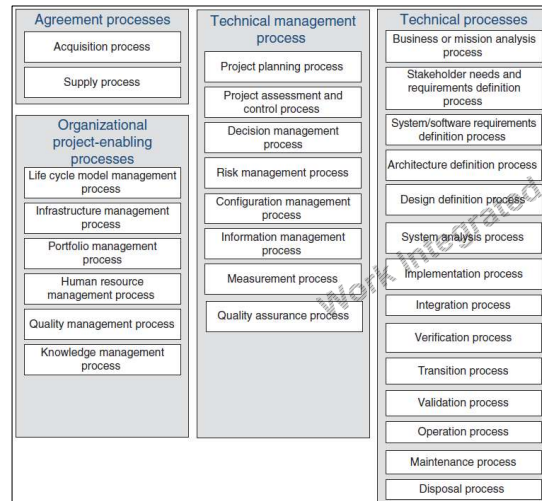


- Establishes a common framework for software life cycle processes.

Each process has –

- Title
- Purpose
- Outcome
- Activities
- Task

(QA – 7.2.3)



BITS Pilani, Pilani Campus

24

ISO/IEC/IEEE 12207



QA Process in Detail

- Title – The Quality Assurance Process
- Purpose
 - To help ensure the effective application of the organization's quality management process to the project.
 - QA focuses on providing confidence that quality requirements will be fulfilled. Proactive analysis of the project life cycle processes and outputs is performed to assure that the product being produced will be of the desired quality and that organization and project policies and procedures are followed.
- Outcome
 - Project QA procedures are defined and implemented.
 - Criteria and methods for QA evaluations are defined.
 - Evaluations of the project's products, services, and processes are performed, consistent with quality management policies, procedures, and requirements.
 - Results of evaluations are provided to relevant stakeholders.
 - Incidents are resolved.
 - Prioritized problems are treated.

BITS Pilani, Pilani Campus

25

ISO/IEC/IEEE 12207



Activities & Task

- Prepare for QA.
 - Define a QA strategy.
 - Establish independence of QA from other life cycle processes.
- Perform product or service evaluations.
 - Evaluate products and services for conformance to established criteria, contracts, standards, and regulations.
 - Monitor that verification and validation of the outputs of the life cycle processes are performed to determine conformance to specified requirements.
- Perform process evaluations.
 - Evaluate project life cycle processes for conformance.
 - Evaluate tools and environments that support or automate the process for conformance.
 - Evaluate supplier processes for conformance to process requirements.
- Manage QA records and reports.
 - Create records and reports related to QA activities.
 - Maintain, store, and distribute records and reports.
 - Identify incidents and problems associated with product, service, and process evaluations.

BITS Pilani, Pilani Campus

26

ISO/IEC/IEEE 12207



Activities & Task

- Treat incidents and problems. This activity consists of the following tasks:
 - Record, analyze, and classify incidents.
 - Identify selected incidents to associate with known errors or problems.
 - Record, analyze and classify problems.
 - Identify root causes and treatment of problems where feasible.
 - Prioritize treatment of problems (problem resolution) and track corrective actions.
 - Analyze trends in incidents and problems.
 - Identify improvements in processes and products that may prevent future incidents and problems.
 - Inform designated stakeholders of the status of incidents and problems.
 - Track incidents and problems to closure.

BITS Pilani, Pilani Campus

27

IEEE 730 for SQA Processes



Establishes the requirements for the planning and implementation of SQA activities for a software project (<https://www.yegor256.com/pdf/ieee-730-2014.pdf>)

The IEEE 730 is structured as follows :

- Clause 1 - Describes the scope, purpose and an introduction.
- Clause 2 - Identifies normative references used by the IEEE 730.
- Clause 3 - Defines the terms, abbreviations and acronyms.
- Clause 4 - Describes the context for the SQA processes and SQA activities, and covers expectations for how this standard will be applied.
- Clause 5 - Specifies the processes, activities and SQA tasks. Sixteen activities grouped into three categories are described: implementation of SQA process, product assurance, and process assurance.
- Twelve informative annexes A–L, where annex C provides guidelines for creating a SQAP.

BITS Pilani, Pilani Campus

28

IEEE 730 for SQA Processes

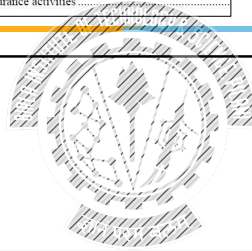


Contents

1. Overview	
1.1 Scope	
1.2 Purpose	
1.3 Field of application	
1.4 Limitations	
1.5 Conformance	
1.6 Intended usage of this standard	
1.7 Organization of this standard	
2. Normative references	
3. Definitions, acronyms, and abbreviations	
3.1 Conventions	
3.2 Definitions	
3.3 Acronyms and abbreviations	
4. Key concepts of Software Quality Assurance	
4.1 Organizational management responsibility	
4.2 Organization and project relationship	
4.3 Software quality and relationship to requirements	
4.4 Overview of SQA activities	
4.5 Acquirer and supplier perspectives	
4.6 Key concepts of this standard	
4.7 Software process improvement	
4.8 Maintaining quality management system consistency	
5. Software Quality Assurance process	
5.1 Purpose	
5.2 Organization of SQA process outcomes	
5.3 SQA process implementation activities	
5.4 Product assurance activities	
5.5 Process assurance activities	

BITS Pilani, Pilani Campus

29



Thank You

BITS Pilani, Pilani Campus

30



Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

BITS Pilani, Pilani Campus

3

Agenda



- Software Quality Factors
- Quality Attributes
 1. Functional Suitability
 2. Performance Efficiency
 3. Compatibility
 4. Usability
 5. Reliability
 6. Security
 7. Maintainability
 8. Portability

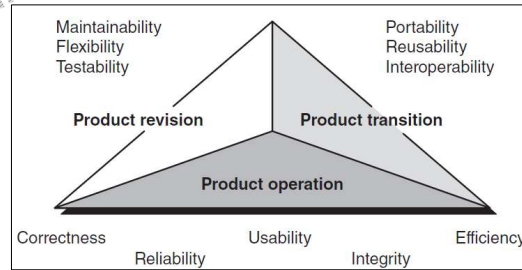
BITS Pilani, Pilani Campus

4

Quality Models – McCall [MCC 77]



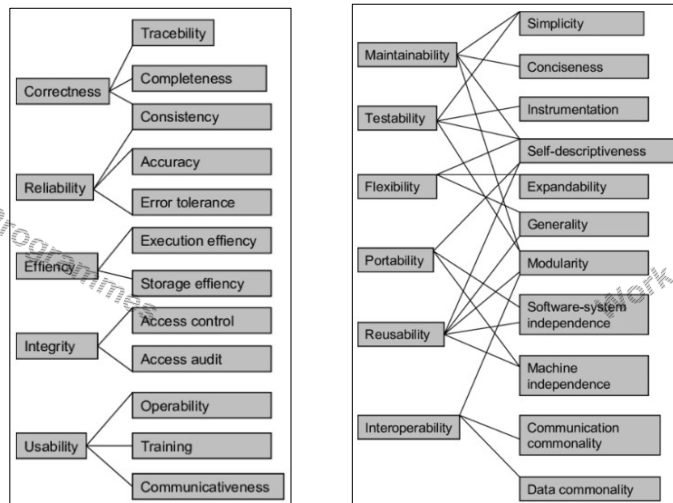
- McCall [MCC 77] Quality Model
- Perspective → Into Quality Factor → Quality criteria (Measurable).



BITS Pilani, Pilani Campus

5

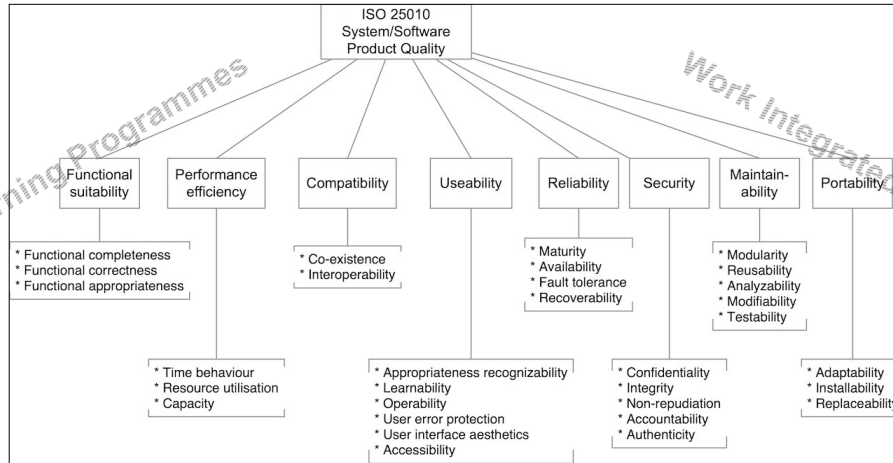
Quality Models – McCall [MCC 77]



BITS Pilani, Pilani Campus

6

ISO 25010



BITS Pilani, Pilani Campus

7

1. Functional Suitability



Functional Suitability - Capability of a product to provide functions that meet stated and implied needs of intended users when it is used under specified conditions

- **Functional completeness** - Capability of a product to provide a set of functions that covers all the specified tasks and intended users' objectives.
 - All functionality implemented as per requirements. No requirements should be left out.
 - To verify that each functionality is present in the system, every requirement should be traceable through Testing.
 - Data should not be missing. Example in reports, etc

BITS Pilani, Pilani Campus

8

Functional Suitability



- **Functional Correctness** - Capability of a product to provide accurate results when used by intended users.
 - Inaccuracy in calculations. Rounding off errors
 - Data should be up-to-date. No stale data. Duration of data should be accurate.
 - Standards for coding and documenting the software system.
- **Functional Appropriateness** - Capability of a product to provide functions that facilitate the accomplishment of specified tasks and objectives.
 - A user is only presented with the necessary steps to complete a task, excluding any unnecessary steps.

BITS Pilani, Pilani Campus

9

2. Performance Efficiency



- Performance Efficiency** - Capability of a product to perform its functions within specified time and throughput parameters and be efficient in the use of resources under specified conditions.
- Resources can be CPU, memory, storage, and network devices.
 - Resources can include other software products, the software and hardware configuration of the system, energy, and materials (e.g. print paper, storage media).
- **Time Behaviour** - Capability of a product to perform its specified function under specified conditions so that the response time and throughput rates meet the requirements.
 - Time it takes to perform an operation.
 - Refresh rate of screen.
 - Time taken for background operations.

BITS Pilani, Pilani Campus

10

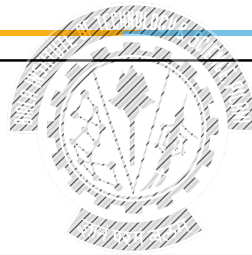
Performance Efficiency



- **Resource Utilization** - Capability of a product to use no more than the specified amount of resources to perform its function under specified conditions.
 - Memory usage
 - Disk usage
 - Network Bandwidth usage
 - In a multi user environment, one single application should not consume all the resources
- **Capacity** - Capability of a product to meet requirements for the maximum limits of a product parameter
 - Parameters can include the number of items that can be stored, the number of concurrent users, the communication bandwidth, the throughput of transactions, and the size of a database.

BITS Pilani, Pilani Campus

11



3. Compatibility



Compatibility - Capability of a product to exchange information with other products, and/or to perform its required functions while sharing the same common environment and resources.

- **Co-existence** - Capability of a product to perform its required functions efficiently while sharing a common environment and resources with other products, without detrimental impact on any other product.
 - Multiple apps working simultaneously on the same operating system
 - Sharing same Memory, Disk, etc
- **Interoperability** - Capability of a product to exchange information with other products and mutually use the information that has been exchanged
 - Information is meaningful data; and information exchange includes transformation of data for exchange.
 - There can be protocols for data conversion.

BITS Pilani, Pilani Campus

12

4. Usability



Usability - Degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use.

- **Appropriateness recognizability** - Degree to which users can recognize whether a product or system is appropriate for their needs.
 - Time needed to understand the software functionality.
 - Finding an Excel formula and understanding its usage for a particular task.
- **Learnability** - Degree to which a product or system can be used by specified users to achieve specified goals of learning to use the product or system with effectiveness, efficiency, freedom from risk and satisfaction in a specified context of use.
 - Time needed to learn to use
 - Similar operation under same menu.

BITS Pilani, Pilani Campus

13

4. Usability



- **Operability** - Attributes that make it easy to operate and control.
 - Number of steps needed to achieve a certain functionality.
- **User error protection** - Protects users against making errors.
 - Error messages and warnings for wrong usage (Warning message before deleting any file)
 - Completely stop users to perform certain functions which can send system into unusable state. (Cannot delete system files - Only view them)

BITS Pilani, Pilani Campus

14

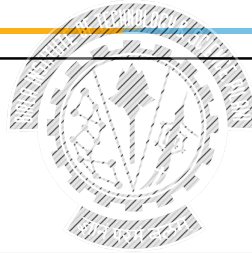
4. Usability



- **User interface aesthetics** - User interface enables pleasing and satisfying interaction for the user.
 - Configurable Color themes
 - Rounded button Vs Square buttons Vs Square buttons with rounded corners
- **Accessibility** - Degree to which a product or system can be used by people with the widest range of characteristics and capabilities to achieve a specified goal in a specified context of use.
 - System can be used by, "Specially Abled" people
 - Big font size for old age people
 - Voice conversion for written text.

BITS Pilani, Pilani Campus

15



5. Reliability



- **Reliability** - Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time.
 - Deals with failure to provide service.
 - Failure rate could refer to a component or system as a whole.
- **Maturity** - Degree to which a system meets needs for reliability under normal operation.
 - MS Teams meeting - Voice clarity under normal circumstance.
- **Availability** - Degree to which a system, product or component is operational and accessible when required for use.
 - Controlling downtime caused due to maintenance, breakdown, etc

BITS Pilani, Pilani Campus

16

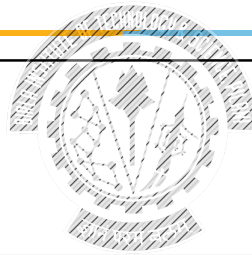
5. Reliability



- **Fault tolerance** - Degree to which a system, product, or component operates as intended despite the presence of hardware or software Faults.
 - RAID configurations - Read and write across multiple disks. Mirroring data sets across multiple disks, which makes RAID systems more fault tolerant by creating built-in data redundancy.
- **Recoverability** - Degree to which, in the event of an interruption or a failure, a product or system can recover the data directly affected and re-establish the desired state of the system.
 - Power goes off in the meeting and is back in a minute. Meeting is restarted automatically. All the people do not have to join the meeting again.
 - Slight disruption in Wi-Fi. Status of meeting does not changes

BITS Pilani, Pilani Campus

17



6. Security



Security - Degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization.

- **Confidentiality** - Degree to which a product or system ensures that data are accessible only to those authorized to have access.
 - Only authorized users can access the data
 - Login/Pwd for users.
 - E.g. People not in this course cannot access any material related to this course
- **Integrity** - Degree to which a system, product or component prevents unauthorized access to, or modification of, computer programs or data.
 - Only authorized users do certain operations on the data.
 - E.g. Only instructor can add/delete in Files section

BITS Pilani, Pilani Campus

18

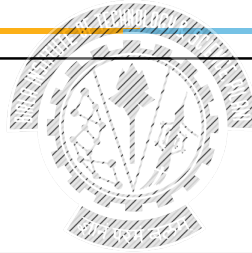
6. Security



- **Non-repudiation** - Degree to which actions or events can be proven to have taken place so that the events or actions cannot be repudiated later.
 - E.g. If files gets deleted in the system, it can be proven that the file did get deleted.
 - Versioning in files maintain all the changes happening in the file.
- **Accountability** - Degree to which the actions of an entity can be traced uniquely to the entity.
 - Logs and traces in the system to track as to who performed any action
- **Authenticity** - Degree to which the identity of a subject or resource can be proved to be the one claimed.
 - Can be done using certifications, encryption.
 - No change can be done to document and also it can be proved as to who did it.

BITS Pilani, Pilani Campus

19



7. Maintainability



Maintainability - This characteristic represents the degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements.

- **Modularity** - Degree to which a system or computer program is composed of discrete components such that a change to one component has minimal impact on other components.
 - Easy to do change management.
 - Object oriented programing.
- **Reusability** - Degree to which an asset can be used in more than one system, or in building other assets.
 - Libraries, DLLs

BITS Pilani, Pilani Campus

20

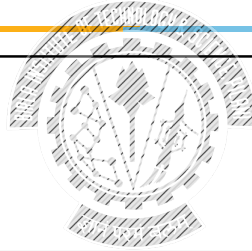
7. Maintainability



- **Analyzability** - Degree of effectiveness and efficiency with which it is possible to assess the impact on a product or system of an intended change to one or more of its parts, or to diagnose a product for deficiencies or causes of failures, or to identify parts to be modified.
 - Helps in bug fixing, new feature development
- **Modifiability** - Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality.
 - Able to get confidence in quality with minimal regression testing
- **Testability** - Degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.
 - Design test cases corresponding to requirements and able to test the system using those Test Cases

BITS Pilani, Pilani Campus

21



8. Portability



Portability - Degree of effectiveness and efficiency with which a system, product or component can be transferred from one hardware, software or other operational or usage environment to another. This characteristic is composed of the following sub-characteristics:

- **Adaptability** - Degree to which a product or system can effectively and efficiently be adapted for different or evolving hardware, software or other operational or usage environments.
 - New Driver for newer Printer, Audio device, etc
 - Newer version of Apps corresponding to newer operating system

BITS Pilani, Pilani Campus

22

8. Portability



- **Installability** - Degree of effectiveness and efficiency with which a product or system can be successfully installed and/or uninstalled in a specified environment.
 - New software
 - Patches, New Features
 - Service Packs
 - Newer versions.
- **Replaceability** - Degree to which a product can replace another specified software product for the same purpose in the same environment.
 - IE, Chrome, Firefox
 - Mail clients

BITS Pilani, Pilani Campus

23

Reliability Analysis



- Reliability theory is concerned with determining the probability, that a system, possibly consisting of many components will function.
- Overall, System components can be treated in configurations
 - Series System: System will function if all components are functioning
 - Parallel System: System will function if one of the component is functioning. (Provided the ultimate aim is to have the same component working).
 - Composite System: (Series + Parallel) in varying configurations

BITS Pilani, Pilani Campus

24

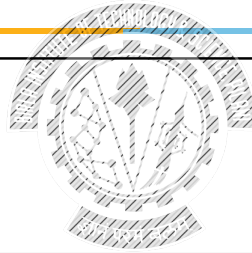
Reliability Analysis



- Each component will function with some known probability (independent of each other)
- Upper and lower bounds of probability
- A component may show a dynamic behavior over time i.e. it functions for a random length of time and after which it fails
- Amount of time a system functions is related to the distribution of a component lifetimes
- If the amount of time that a component functions has an increasing failure rate on average distribution, then so does the distribution of system lifetimes

BITS Pilani, Pilani Campus

25



Reliability Analysis



- Mean Time to Failure (MTTF) = Total Testing time/# of defects

$$MTTF = T/d$$
- Failure Rate

$$F_a = \frac{1}{MTTF}$$
- Mean Time to Repair (MTTR) = Sum(|Time of detection of fault – time of closure|)/Total # of defects

$$MTTR = \sum (T_d - T_c)/d$$
- Availability

$$A = MTTF/(MTTR + MTTF)$$
- Mean Time between Failure

$$MTBF = MTTF + MTTR$$

BITS Pilani, Pilani Campus

26

Reliability Analysis



- Problem: A SW component was tested for a period of 3 months. The total number of defects found were 300. On an average the time required to fix the defect was 2.5 days. Find the availability of the system

- Solution:

$$MTTF = (3 * 30 * 8) / 300 = 2.4$$

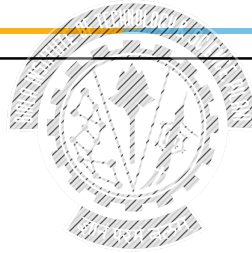
$$F_a = 1 / 2.4 = 0.4166$$

$$MTTR = (2.5 * 8) = 20$$

$$\text{Availability } A = MTTF / (MTTF + MTTR) = 0.107 \sim 10.7 \%$$

BITS Pilani, Pilani Campus

27



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -04

Harish Aggarwal

BITS Pilani
Pilani Campus

1



BITS Pilani
Pilani Campus

CMMI, ITIL

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Software Quality Factors
- Quality Attributes
 - Functional Suitability
 - Performance Efficiency
 - Compatibility
 - Usability
 - Reliability
 - Security
 - Maintainability
 - Portability
- Reliability Analysis

BITS Pilani, Pilani Campus

3

Agenda



- Frameworks
 - CMMI-Development
 - ITIL Framework
- Other Frameworks
 - ISO/IEC 20000 Standard
 - CobiT Process - Best practices for IT governance.
 - ISO/IEC 27000 Family of Standards for Information Security
 - ISO/IEC 29110 Standards and Guides for Very Small Entities

BITS Pilani, Pilani Campus

4

CMM



Capability Maturity Model (CMM®).

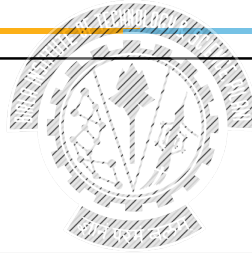
- Developed at the request of the American DoD, by the Software Engineering Institute (SEI) at Carnegie Mellon University.
- Primarily focused on software engineering practices
- Provides a road map of engineering practices to improve the performance of the development, maintenance and service provisioning processes.

Capability Maturity Model Integration (CMMI):

- CMMI is an evolved version of CMM and is a framework that encompasses not only software engineering but also various other domains, including systems engineering, project management, and service delivery.

BITS Pilani, Pilani Campus

5



CMMI – Key Aspects



Key aspects of CMMI include:

- **Continuous and Staged Representations:** CMMI supports two representations –
 - **Continuous** - Allows to select and implement individual process areas based on their specific needs.
 - **Staged** - Requires organizations to follow a predefined sequence of maturity levels.
- **Process Areas:** Defines specific process areas at each maturity level
 - Examples include Requirements Management, Project Planning, and Configuration Management.
- **Appraisals:** CMMI certification such as SCAMPI (Standard CMMI Appraisal Method for Process Improvement).
 - Process maturity and
 - Adherence to CMMI practices.
- **Evolution:** CMMI has evolved beyond its initial focus on software engineering to cover various disciplines.
 - CMMI for Development,
 - CMMI for Services, and
 - CMMI for Acquisition

BITS Pilani, Pilani Campus

6

CMMI – Appraisal



CMMI Institute authorizes and oversees organizations to provide official CMMI training and appraisal services.

CMMI Institute is a subsidiary of ISACA - International professional association focused on IT (information technology) governance or (Information Systems Audit and Control Association)

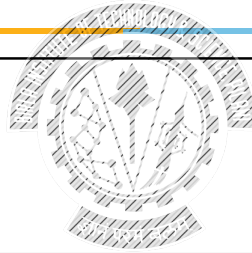
Appraisal Sponsor Organizations (ASOs): Organizations authorized by the CMMI Institute. They have certified lead appraisers and are responsible for ensuring the quality and integrity of the appraisal process.

CMMI Institute Partner Organizations: These organizations collaborate with the CMMI Institute to deliver official CMMI training and appraisal services.

Certified Lead Appraisers (CLAs): Individuals who have undergone CMMI Institute-approved training and certification processes to become certified lead appraisers.

BITS Pilani, Pilani Campus

7



CMMI for Development



Level	Focus	Key process area	
5 Optimizing	Continuous process improvement	Organizational performance management causal analysis and resolution	Quality productivity Risk rework
4 Quantitatively managed	Quantitative management	Organizational process performance quantitative project management	
3 Defined	Process standardization	Requirements development Technical solution Product integration Verification Validation Organizational process focus Organizational process definition Organizational training Integrated project management Risk management Decision analysis and resolution	
2 Managed	Basic project management	Requirement management Project planning Project monitoring and control Supplier agreement management Measurement and analysis Process and product quality assurance Configuration management	
1 Initial			

BITS Pilani, Pilani Campus

8

CMMI



The **five maturity levels** of CMM were:

Level 1 - Initial:

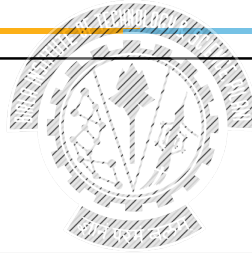
- Processes are ad-hoc and chaotic.
- Processes are unpredictable, poorly controlled, and reactive.
- There is no standardization, and success depends on individual heroics and efforts.
- Organizations at this level often experience high variation in process performance
- Frequent crisis management.

Level 2 - Repeatable: Basic project management processes are established.

- Processes are defined at the project level, and there is consistency in project execution.
- Processes are documented and communicated.
- Project performance is monitored, and corrective actions are taken.

BITS Pilani, Pilani Campus

9



CMMI



Level 3 - Defined: Processes are well-defined, documented, and standardized across the organization

- Standardized and documented processes at the organizational level.
- Emphasis on proactive management.
- Process metrics are collected and used for process improvement.
- Implement a culture of continuous improvement.

Level 4 - Managed: Detailed measurements of processes and their effectiveness are collected.

- Quantitative objectives are set for process performance.
- Process performance is measured and controlled quantitatively.
- Variability in process performance is reduced.
- Identify and address the root causes of process variation.

Level 5 - Optimizing: Continuous process improvement is institutionalized.

- Focus on innovation, optimization, and process improvement at both the organizational and project levels.
- Continuous improvement is part of the organizational culture.
- Process improvements are based on quantitative feedback.
- Best practices are identified and shared across the organization.

BITS Pilani, Pilani Campus

10

CMMI-Dev Model L-1



Maturity Level 1: Initial

- Processes are usually ad hoc and chaotic.
- Success in these organizations depends on the competence and heroics of the people
- Abandon their processes in a time of crisis, and be unable to repeat their successes.

BITS Pilani, Pilani Campus

11

CMMI-Dev Model L-2



Maturity Level 2: Managed

- Processes are planned and executed in accordance with policy
- Able to produce controlled outputs. Practices are retained during times of stress.
- Process areas:
 - **Requirements management** - manage requirements of the project's products and product components.
 - **Project planning** - Establish and maintain plans that define project activities
 - **Project monitoring and control** - Provide an understanding of the project's progress
 - **Supplier agreement management** - Manage the acquisition of products and services from suppliers
 - **Measurement and analysis** - Develop and sustain a measurement capability used to support management information needs.
 - **Process and product quality assurance** - Provide staff and management with objective insight into processes and associated work products. Identify non-compliance.
 - **Configuration management** – Maintaining Baselines and controlling changes.

BITS Pilani, Pilani Campus

12

CMMI-Dev Model L-3



Maturity Level 3: Defined

- Processes are well characterized and understood, and are described in standards, procedures, tools, and methods.
- Processes are improved over time.
- Processes are used to establish consistency across the organization.
- Process areas include :
 - Requirements development - Elicit, analyze, and establish customer, product, and product component requirements
 - Technical solution - Select, design, and implement solutions to requirements
 - Product integration - Assemble the product from the product components
 - Verification - Ensure that selected work products meet their specified requirements
 - Validation - Demonstrate that a product or product component fulfills its intended use.
 - Organizational process focus - Plan, implement, and deploy organizational process improvements.
 - Organizational process definition - Establish and maintain organizational process, standards, rules and guidelines for teams.
 - Organizational training - Develop skills and knowledge of people.
 - Integrated project management – Manage Project, Include stakeholders.
 - Risk management - Identify potential problems before they occur.
 - Decision analysis and resolution - Analyze possible decisions using a formal evaluation process.

BITS Pilani, Pilani Campus

13

CMMI-Dev Model L- 4 & 5



Maturity Level 4: Quantitatively managed

- The organization and projects establish quantitative objectives for quality and process performance and use them as criteria in managing projects.
- Quality and process performance is understood in statistical terms and is managed throughout the life of projects.
- Process areas include:
 - Organizational process performance - Establish and maintain a quantitative understanding of the performance of selected processes.
 - Quantitative project management - Quantitatively manage the project to achieve the project's established quality and process performance objectives.

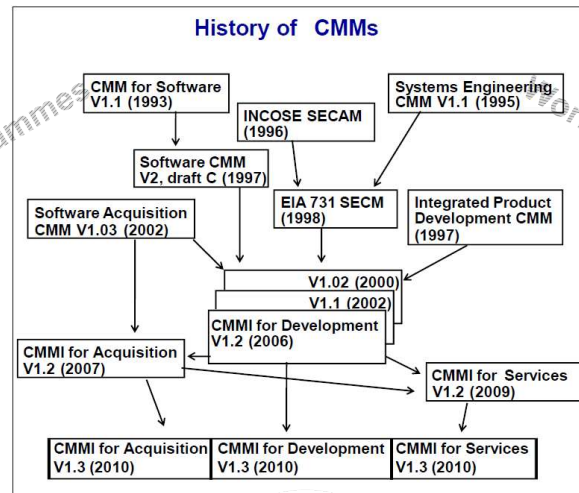
Maturity Level 5: Optimizing

- An organization continually improves its processes based on a quantitative understanding of its business objectives and performance needs. Process areas -
 - Organizational performance management - Proactively manage the organization's performance to meet its business objectives.
 - Causal analysis and resolution - Identify causes of selected outcomes and take action to improve process performance

BITS Pilani, Pilani Campus

14

CMM - History



BITS Pilani, Pilani Campus

15

ITIL Framework



- ITIL stands for Information Technology Infrastructure Library
- A set of best practices for IT service management (ITSM)
- Created in Great Britain based on good management practices for computer services
- For IT services - Backup copies, Recovery, Computer Administration, Telecommunications, Production data, etc
- Service operation processes focus more on longer term management than support processes
- The main objective is to ensure that the IT infrastructure meets the business requirements of the organization.

BITS Pilani, Pilani Campus

16

ITIL – Key components



Key Components (Phases)

- **Service Strategy**
 - Focuses on understanding and aligning IT services with the business strategy.
 - Defines the overall vision, mission, and goals for IT services.
- **Service Design**
 - Design of new or changed services, including processes, architectures, and documentation.
 - Ensures that IT services are designed to meet business requirements and expectations.
- **Service Transition**
 - Focuses on transitioning new or changed services into the live environment.
 - Involves change management, release and deployment management, and knowledge management.
- **Service Operation**
 - Concerned with the day-to-day operation of IT services.
 - Includes incident management, problem management, event management, request fulfillment, access management.
- **Continual Service Improvement (CSI)**
 - A core principle throughout the ITIL lifecycle.
 - Involves measuring, monitoring, and improving IT services and processes over time.

BITS Pilani, Pilani Campus

17

ITIL Processes



ITIL describes the **Service Support** center function and the following five processes:

- **Incident management** - Ensures all IT issues ("incidents") are logged, and tracked to resolution without being lost, ignored, or forgotten about.
- **Problem management** - To reduce the likelihood and impact of incidents by identifying actual and potential causes of incidents and managing workarounds and known errors.
- **Configuration management** - Manage and control assets that make up an IT service.
- **Change management** - To ensure that changes are recorded, evaluated, authorized, prioritized, planned, tested, implemented, documented, and reviewed in a controlled manner
- **Commissioning/Release management** - Plan, schedule, and control releases from end to end

BITS Pilani, Pilani Campus

18

ITIL Processes



ITIL describes the following five processes for **Service Operation**:

- **Service level management** - Maintaining Service Catalogue. Agreement and maintaining of Service Level Agreements
- **Financial management of IT services** - Accounting, budgeting, and charging services so that the organization covers costs and generates profits for those services
- **Capacity management** – Planning capacity (Software, Hardware, Human Resources) to provide optimum and cost effective provision of IT Services based on current and future demand.
- **IT service continuity management** - Defines and plans all measures and processes for unpredicted events of disaster based on regular analysis of vulnerabilities, threats and risks.
- **Availability management** - Sustained availability of the IT infrastructure as per agreed SLA.

BITS Pilani, Pilani Campus

19

ITIL History and evolution



- History and evolution
 - ITIL – 1980's
 - ITIL V2 – 2000/2001
 - ITIL v3 – 2007
 - ITIL V4 – 2019

The ISO/IEC 20000 standard is the first ISO standard dedicated to managing IT services and is based on ITIL Framework

BITS Pilani, Pilani Campus

20

ITIL Certifications



- ITIL Foundation:
 - Entry-level certification providing an understanding of ITIL concepts and terminology.
- ITIL Practitioner:
 - Focuses on applying ITIL principles in a practical context.
- ITIL Intermediate:
 - Offers two paths (Service Lifecycle and Service Capability) with multiple modules covering specific areas of ITIL.
- ITIL Expert:
 - Achieved by accumulating credits from both Foundation and Intermediate levels.
- ITIL Master:
 - The highest level of ITIL certification, demonstrating the ability to apply ITIL principles in real-world situations.

BITS Pilani, Pilani Campus

21

Other Standards



- ISO/IEC 20000 Standard
 - The ISO/IEC 20000 standard is the first ISO standard dedicated to managing IT services and is based on ITIL Framework
- CobiT Process (Control Objectives for Information and Related Technology)
 - Repository of best practices for IT governance established by ISACA (IT auditors)
 - Harmonized with the ITIL reference, the PMBOK® Guide from the Project Management Institute [PMI 13] as well as the ISO 27001 and ISO 27002 standards
- ISO/IEC 27000 Family of Standards for Information Security
 - Preservation of confidentiality, integrity and availability of information
- ISO/IEC 29110 Standards and Guides for Very Small Entities
 - For VSEs, namely enterprises, organizations, departments or projects with up to 25 people.

BITS Pilani, Pilani Campus

22

SQA – Standards and Resources



- **ISO/IEC/IEEE 12207:2017**

- ISO/IEC –
 - International Organization for Standards/International Electrotechnical Commission
- Software life cycle processes standards
- <https://www.iso.org/standard/63712.html>
- Earlier ISO/IEC 12207:2008

- **IEEE 730-2014 (Institute of Electrical and Electronics Engineers (IEEE))**

- IEEE Standard for Software Quality Assurance Processes
- <https://standards.ieee.org/ieee/730/5284/>

BITS Pilani, Pilani Campus

23

SQA – Standards and Resources



- **CMMI Development (Capability Maturity Model Integration)**

- <https://cmmiinstitute.com/cmmi/dev>

- **ISO Guide to the Software Engineering Body of Knowledge (SWEBOK)**

- <https://www.computer.org/education/bodies-of-knowledge/software-engineering>

BITS Pilani, Pilani Campus

24




SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -05

Harish Aggarwal

BITS Pilani
Pilani Campus

1





BITS Pilani
Pilani Campus

Software Testing Fundamentals
Verification and Validation

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 04

- Frameworks
 - CMMI-Development
 - ITIL Framework
- Other Frameworks

BITS Pilani, Pilani Campus

3

Agenda



- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing
 - Black Box Testing
 - White Box Testing
- Test Levels and Types
 - Unit Testing
 - Integration Testing
 - System Testing

BITS Pilani, Pilani Campus

4

Software Testing Fundamentals



Definition –

- Testing is the process of executing a program with intention of finding errors. [Myers]
- Testing is the process of exercising or evaluating a system or system component by manual or automated means to verify that it satisfies specified requirements. [IEEE 83a]

[IEEE Std 610.12 (IEEE, 1990)] -

- The process of operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system or component.
- The process of analyzing a software item to detect the differences between existing and required conditions (that is, bugs) and to evaluate the features of the software item

BITS Pilani, Pilani Campus

5

Software Testing Fundamentals



- Software testing is a **formal** process carried out by a **specialized testing team** in which a software unit, several integrated software units or an entire software package are examined by **running the programs** on a computer. All the associated tests are performed according to **approved test procedures** on **approved test cases**.

BITS Pilani, Pilani Campus

6

Software Testing - Why



The Technical Case:

- Competent developers are not infallible.
- The implications of requirements are not always foreseeable.
- The behavior of a system is not necessarily predictable from its components.
- Languages, databases, user interfaces, and operating systems have bugs that can cause application failures.
- Reusable classes and objects may have underlying issues.

The Business Case:

- If you don't find bugs your customers or users will.
- Post-release debugging is the most expensive form of development.
- Buggy software hurts operations, sales, and reputation.
- Buggy software can be hazardous to life and property.
- Cost of fixing is higher if bugs are found later in the cycle.

BITS Pilani, Pilani Campus

7

Software Testing - Who



Who should be doing the Testing

- Development Team.
- Product Management Team
- Project Management Team
- Dedicated Test Team.
- Leadership (Alpha Testing, Beta Testing)
- Customers
- Effectively – It is Team Effort and ongoing process throughout lifecycle of Product Development

BITS Pilani, Pilani Campus

8

Software Testing - How much



How much Testing should be done.

- Studies show that software testing constitutes
 - About 40% of overall effort and
 - 25% of the overall software budget
- Testing is never 100% exhaustive.
- Bug Free product is Myth
- Reach Acceptable Quality
- There is no standard way to measure testing process.
 - Metrics can be computed at the organizational, process, project, and product levels.
 - Each set of measurements has its value in monitoring, planning, and control.
- Testing is a trade-off between budget, time, and quality.

BITS Pilani, Pilani Campus

9

Software Testing - Error, Defect, Failure



- **Error** – It is a human mistake which can be caused in any phase of SDLC (Requirements, Design, Coding, Testing)
- **Defect** – Once the error is caused, its representation or manifestation is called as Defect. Defect is also called as Fault or Bug. A Defect is a variance between expected and actual Result
- **Failure** – Execution of Bug is called failure.



BITS Pilani, Pilani Campus

10

Software Testing – Error, Defect, Failure



Error – Its main causes are :

- Negligence: Carelessness often leads to errors
- Miscommunication: An unclear feature specification or its improper interpretation.
- Inexperience: Inexperienced developers often miss out on essential details.
- Complexity: Intricate algorithms can cause mistakes in coding logic.

Defect – Its main causes are :

- Missing test cases
- Test cases not executed by the tester
- Improper execution of test cases
- Code changed after testing

Example – Transfer of funds from one account to another

- Error - Developer Codes transfer of funds without checking Minimum Balance
- Bug/Defect – Fund can be transferred even if there is no fund in the Account.
- Failure – Transfer of funds even without Minimum Balance

BITS Pilani, Pilani Campus

11

Software Testing – Test Case



• **Test Case** – A test case is a set of actions performed on a system to determine if it satisfies software requirements and functions correctly

- | | |
|--------------------|---------------------|
| • Test Case ID | • Execution History |
| • Purpose | • Date |
| • Preconditions | • Result |
| • Inputs | • Version |
| • Expected Outputs | • Run By |
| • Postconditions | |

BITS Pilani, Pilani Campus

12

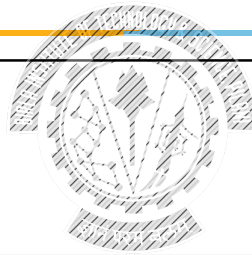
Software Testing - Terms and Definitions



- **Test Suit** – A collection of test scripts or test cases
- **Test Script** - The step-by-step instructions that describe how a test case is to be executed.
- **Test ware**: Includes all of testing documentation. For example, test specification, test scripts, test cases, test data, the environment specification.
- **Test log**: A chronological record of all relevant details about the execution of a test.
- **Test report**: A document describing the conduct and results of testing carried out for a system.

BITS Pilani, Pilani Campus

13



Principles of Testing -



1. Testing time and resources are limited: Avoid redundant tests.
2. Exhaustive testing is impossible. Find a middle path. Make a decision.
3. Use effective resources to test : Use most suitable tools, procedures, and individuals to conduct the tests.
4. Separate testing Team. Avoid programmers/Development team.
5. Begin "in small" and progress toward testing "in large"
6. All customer requirements should have Test Cases and Test Scenarios.
7. Prepare test reports including test cases and test results to summarize the results of testing.
8. Advance test planning is a must and should be updated in a timely manner. Start as early as requirement gathering phase.

BITS Pilani, Pilani Campus

14

Software Testing Verification Vs Validation



As per IEEE definition(s):

Software verification:

- "It is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase." OR
- "It is the process of evaluating, reviewing, inspecting and doing desk checks of work products such as requirement specifications, design specifications and code." OR
- "It is a human testing activity as it involves looking at the documents on paper."

Software validation:

- "It is defined as the process of evaluating a system or component during or at the end of development process to determine whether it satisfies the specified requirements. It involves executing the actual software. It is a computer based testing process."

BITS Pilani, Pilani Campus

15

Software Testing Verification Vs Validation



S. No.	Verification	Validation
1	It is a static process of verifying documents, design, and code.	It is a dynamic process of validating/testing the actual product.
2	It does not involve executing the code.	It involves executing the code.
3	It is human based checking of documents/files.	It is the computer-based execution of program.
4	Target is requirements specification, application architecture, high level and detailed design, and database design.	Target is actual product—a unit, a module, a set of integrated modules, and the final product.
5	It uses methods like inspections, walk throughs, desk-checking, etc.	It uses methods like black-box, gray-box, and white-box testing.
6	It, generally, comes first—before validation.	It generally follows verification.
7	It answers the question—Are we building the product right?	It answers the question—Did we built product Right?
8	It can catch errors that validation cannot catch.	It can catch errors that verification cannot catch.

BITS Pilani, Pilani Campus

16

Software Testing QA Vs QC



S. No.	Quality Assurance (QA)	Quality Control (QC)
1	It is process-related.	It is product related.
2	It focuses on the process used to develop a product	It focuses on testing of a product developed or a product under development
3	It involves the quality of the processes	It involves the quality of the products.
4	It is a preventive control	It is a detective control
5	Allegiance is to development	Allegiance is not to development

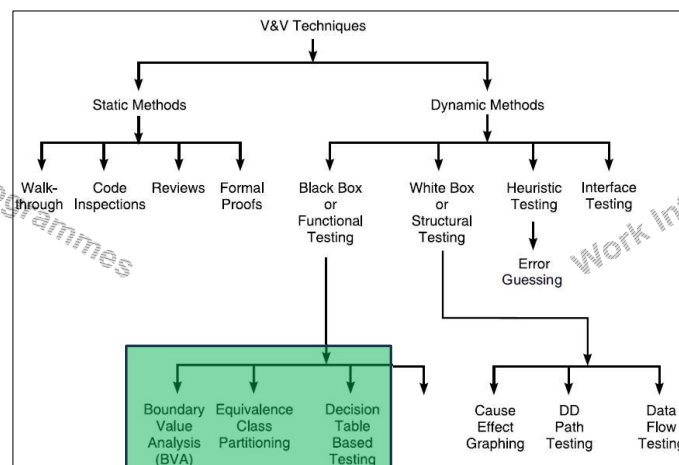
BITS Pilani, Pilani Campus

17

Software Testing V&V Techniques



Together Verification and Validation forms the complete Testing

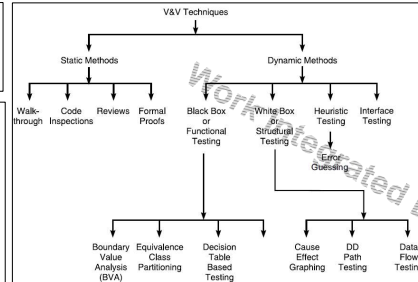
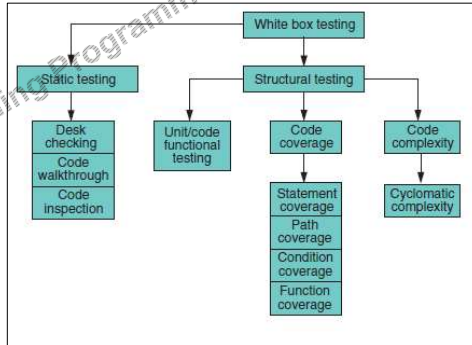


BITS Pilani, Pilani Campus

18

White Box Testing –

- Static Testing
- Structured Testing



BITS Pilani, Pilani Campus

19

Software Testing V&V

- V&V is to be done on –
- Requirements
 - Design
 - Implementation (Code)
 - Testing Infra (Test Cases, etc)
 - Maintenance (Change Request)

- V&V Activities –
- Traceability Analysis
 - Interface Analysis
 - Risk Analysis
 - Walkthrough
 - Inspection
 - Test Plan Generation
 - Test Execution

- Goals – To Ensure
- Correctness
 - Completeness
 - Accuracy
 - Consistency
 - Testability
 - Readability

BITS Pilani, Pilani Campus

20

Software Testing V&V Planning



V&V Plan or simply Test Plan. IEEE83b documents the guidelines for the contents of system, software, build, and module test plans

1. Identification of Goals
2. Selection of V&V Techniques –
 - Requirement Phase - Technical Reviews, Prototyping, and Simulations
 - Specifications Phase - Technical reviews, requirements tracing, prototyping, and simulations
 - Design Phase - Technical reviews, Requirements tracing, Prototyping, Simulation, HLD Review, LLD Reviews.
 - Implementation Phase - Technical reviews, requirements tracing, testing, and proof of correctness.
 - Maintenance Phase – As in Design Phase, Regression Testing
 - Technical Reviews Include - Walk Throughs and Inspections.

BITS Pilani, Pilani Campus

21

Software Testing V&V Planning



3. Organizational Roles and Responsibilities – Dev, Test, SQA, Contractors.
4. Integrating V&V Approaches
5. Problem/Bug Tracking/Management
6. Test Execution Tracking – Test Runs, Resources Used, Time Taken, Bugs Found, etc
7. Asses – Quality of Product, Processes, Techniques.

BITS Pilani, Pilani Campus

22

Software Technical Reviews



Review Methodologies include –

1. Walkthroughs
2. Inspections
3. Audit – Review process by external authority

S. No.	Informal reviews	Formal reviews
1	It is a type of review that typically occurs spontaneously among peers	It is a planned meeting
2	Reviewers have no responsibility	Reviewers are held accountable for their participation in the review
3	No review reports are generated	Review reports containing action items are generated and acted upon

BITS Pilani, Pilani Campus

23

Software Technical Reviews



S. No.	Inspections	Walkthroughs
1	It is a five-step process that is well formalized	It has fewer steps than inspections and is a less formal process
2	It uses checklists for locating errors	It does not use a checklist
3	It is used to analyze the quality of the Process.	It is used to improve the quality of the product
4	This process takes a longer time	It is a shorter process
5	It focuses on training of junior staff	It focuses on finding defects

BITS Pilani, Pilani Campus

24

Inspection



Inspection –

- **Planning** — The moderator plans the inspection. The inspection team is given relevant materials, and after that, the team schedules the inspection meeting and works together.
- **Overview meeting** - Brief summary of the project and its code.
- **Preparation** — Each inspection team conducts inspection inspection checklists.
- **Inspection meeting** — Inspectors point out flaws section by Section.
- **Rework** — Fixing of issues found.
- **Follow-up** — Meeting with the team to discuss the reviewed and fixed code.

BITS Pilani, Pilani Campus

25

STANDARD FOR SOFTWARE TEST DOCUMENTATION (IEEE829)



1. Test Plan

- Test-plan Identifier
- Introduction - Features to be tested, Reference to other documents
- Test Items
- Features to be Tested
- Approach
- Item Pass/Fail Criteria
- Suspension Criteria and Resumption
- Test Deliverables
- Testing Tasks
- Environmental Needs
- Responsibilities
- Staffing and Training Needs
- Schedule
- Risks and Contingencies¹⁶.
- Approvals: Specifies the persons who must approve this plan

BITS Pilani, Pilani Campus

26

STANDARD FOR SOFTWARE TEST DOCUMENTATION (IEEE829)



2. Test Case Specification
 - Test-case Specification Identifier
 - Test Items
 - Input Specifications
 - Output Specifications
 - Environmental Needs
3. Test-Incident Report (Bug Report)
 - Bug-Report Identifier
 - Summary
 - Bug Description
 - Impact: Priority, Severity (urgent, high, medium, low).
4. Test-Summary Report
5. Inspection Checklist for Test Plans
6. Inspection Checklist for Test Cases

BITS Pilani, Pilani Campus

27

Software Testing White-Box Testing and Black-Box



Software Test Classification

- **White box (structural) testing.** Examines internal calculation paths in order to identify bugs. Also called as structural testing or clear-box or Glass-Box or Open-box testing.

IEEE definition

- Testing that takes into account the internal mechanism of a system or component

- **Black box (functionality) testing.** Identifies bugs only according to software malfunctioning as they are revealed in its erroneous outputs. Does not care about internal path of calculations and processing performed.

IEEE definitions

- (1) Testing that ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions.
- (2) Testing conducted to evaluate the compliance of a system or component with specified functional requirements.

BITS Pilani, Pilani Campus

28

Software Testing Strategies



- **Software Testing Strategies**

- Positive Testing – Test with legal Test Data to observe Normal behavior
- Negative Testing – Test with illegal or abnormal data and observe the behavior.
- Big Bang Testing (Entire System)
- Unit testing
- Integration Testing
- System Testing
- Top-Down ; Bottom-Up
- Stubs Testing (Unavailable Lower Level Module) (Top Down Methodology)
- Driver Testing (Unavailable Upper Level Module) (Bottom Up Methodology)
- Static Testing
 - Code Reviews, Code Inspections, Walkthroughs, and software technical reviews (STRs)

BITS Pilani, Pilani Campus

29

Unit, Integration, System Testing



- **Unit Testing**
 - Individual module is tested in isolation from the Rest of the Software.
- **Integration Testing**
 - One or more modules are tested together
 - Interactions between the modules is tested
 - Top-Down, Bottom-Up, Sandwich (Bi-Directional Integration), Big-Bang.
- **System Testing**
 - Testing on complete integrated product including H/W and S/W
 - Includes functional and Non-functional tests.

BITS Pilani, Pilani Campus

30



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture - 06

BITS Pilani
Pilani Campus

Harish Aggarwal

1



BITS Pilani
Pilani Campus

Black Box Testing Methodologies

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Frameworks
 - CMMI-Development
 - ITIL Framework
- Other Frameworks

Lecture - 04

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 05

- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing - Black Box, White Box Testing
- Test Levels and Types –
 - Unit, Integration, System Testing

BITS Pilani, Pilani Campus

3

Agenda



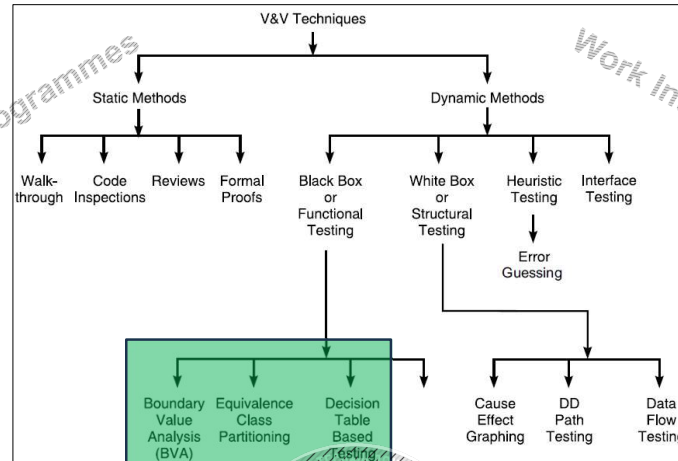
- Test Design Techniques
 - Combinatorial Testing
 - Boundary Value Analysis
 - Equivalence Class Partitioning

BITS Pilani, Pilani Campus

4

Software Testing V&V Techniques

Together Verification and Validation forms the complete Testing



BITS Pilani, Pilani Campus

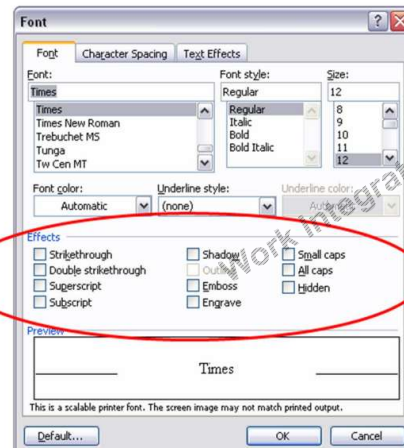
5

Need of Combinatorial

Possible Test case

$$2^{10} = 1,024$$

P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	1
0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	1	1
1	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	1
1	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	1	1
1	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	1	0
1	0	0	0	0	0	1	1	0	0
1	0	0	0	0	0	1	1	1	0
1	0	0	0	1	0	0	0	0	0
1	0	0	0	1	0	0	0	0	1
1	0	0	0	1	0	0	1	0	0
1	0	0	0	1	0	0	1	0	1
1	0	0	0	1	1	0	0	0	0
1	0	0	0	1	1	0	0	0	1
1	0	0	0	1	1	1	0	0	0
1	0	0	0	1	1	1	0	0	1
1	0	0	1	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	1
1	0	0	1	0	0	0	1	0	0
1	0	0	1	0	0	0	1	0	1
1	0	0	1	1	0	0	0	0	0
1	0	0	1	1	0	0	0	0	1
1	0	0	1	1	0	1	0	0	0
1	0	0	1	1	0	1	0	0	1
1	0	0	1	1	1	0	0	0	0
1	0	0	1	1	1	0	0	0	1
1	0	0	1	1	1	1	0	0	0
1	0	0	1	1	1	1	0	0	1
1	1	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	1
1	1	0	0	0	0	0	0	1	0
1	1	0	0	0	0	0	0	1	1
1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	1	0	0	0	1
1	1	0	0	0	1	1	0	0	0
1	1	0	0	0	1	1	0	0	1
1	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	1
1	1	0	0	1	0	0	1	0	0
1	1	0	0	1	0	0	1	0	1
1	1	0	0	1	1	0	0	0	0
1	1	0	0	1	1	0	0	0	1
1	1	0	0	1	1	1	0	0	0
1	1	0	0	1	1	1	0	0	1
1	1	0	1	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	1
1	1	0	1	0	0	0	1	0	0
1	1	0	1	0	0	0	1	0	1
1	1	0	1	1	0	0	0	0	0
1	1	0	1	1	0	0	0	0	1
1	1	0	1	1	0	1	0	0	0
1	1	0	1	1	0	1	0	0	1
1	1	0	1	1	1	0	0	0	0
1	1	0	1	1	1	0	0	0	1
1	1	0	1	1	1	1	0	0	0
1	1	0	1	1	1	1	0	0	1
1	1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	1
1	1	1	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	1	1
1	1	1	0	0	1	0	0	0	0
1	1	1	0	0	1	0	0	0	1
1	1	1	0	0	1	1	0	0	0
1	1	1	0	0	1	1	0	0	1
1	1	1	0	1	0	0	0	0	0
1	1	1	0	1	0	0	0	0	1
1	1	1	0	1	0	0	1	0	0
1	1	1	0	1	0	0	1	0	1
1	1	1	0	1	1	0	0	0	0
1	1	1	0	1	1	0	0	0	1
1	1	1	0	1	1	1	0	0	0
1	1	1	0	1	1	1	0	0	1
1	1	1	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	1
1	1	1	1	0	0	0	0	1	0
1	1	1	1	0	0	0	0	1	1
1	1	1	1	0	1	0	0	0	0
1	1	1	1	0	1	0	0	0	1
1	1	1	1	0	1	1	0	0	0
1	1	1	1	0	1	1	0	0	1
1	1	1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0	0	1
1	1	1	1	1	0	0	0	1	0
1	1	1	1	1	0	0	0	1	1
1	1	1	1	1	1	0	0	0	0
1	1	1	1	1	1	0	0	0	1
1	1	1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	0	0	1
1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1



BITS Pilani, Pilani Campus

6

Need of Combinatorial



OS - XP, OS X, RHL (3)
 Browser - IE, Firefox (2)
 Protocol - IPV4, IPV6 (2)
 CPU - Intel, AMD (2)
 DBMS - Sybase, Oracle, MySQL (3)

Total Possible Test case $3 \times 2 \times 2 \times 3 = 72$

Test	OS	Browser	Protocol	CPU	DBMS
1	XP	IE	IPV4	Intel	MySQL
2	XP	Firefox	IPV6	AMD	Sybase
3	XP	IE	IPV6	Intel	Oracle
4	OS X	Firefox	IPV4	AMD	MySQL
5	OS X	IE	IPV4	Intel	Sybase
6	OS X	Firefox	IPV4	Intel	Oracle
7	RHL	IE	IPV6	AMD	MySQL
8	RHL	Firefox	IPV4	Intel	Sybase
9	RHL	Firefox	IPV4	AMD	Oracle
10	OS X	Firefox	IPV6	AMD	Oracle

S. No.	OS	Browser	Protocol	CPU	DBMS
1	XP	IE	IPV4	Intel	Sybase
2	XP	IE	IPV4	Intel	Oracle
3	XP	IE	IPV4	Intel	MySQL
4	XP	IE	IPV4	AMD	Sybase
5	XP	IE	IPV4	AMD	Oracle
6	XP	IE	IPV4	AMD	MySQL
7	XP	IE	IPV6	Intel	Sybase
8	XP	IE	IPV6	Intel	Oracle
9	XP	IE	IPV6	Intel	MySQL
10	XP	IE	IPV6	AMD	Sybase
11	XP	IE	IPV6	AMD	Oracle
12	XP	IE	IPV6	AMD	MySQL
13	XP	Firefox	IPV4	Intel	Sybase
14	XP	Firefox	IPV4	Intel	Oracle
15	XP	Firefox	IPV4	Intel	MySQL
16	XP	Firefox	IPV4	AMD	Sybase
17	XP	Firefox	IPV4	AMD	Oracle
18	XP	Firefox	IPV4	AMD	MySQL
19	XP	Firefox	IPV6	Intel	Sybase
20	XP	Firefox	IPV6	Intel	Oracle
21	XP	Firefox	IPV6	Intel	MySQL
22	XP	Firefox	IPV6	AMD	Sybase
23	XP	Firefox	IPV6	AMD	Oracle
24	XP	Firefox	IPV6	AMD	MySQL

BITS Pilani, Pilani Campus

7

Model the input



- Fault Model (Interaction Faults)
 - A t-way interaction fault is a fault that is triggered by a certain combination of t input values.
 - A simple fault is a t-way fault where $t = 1$
 - A pairwise fault is a t-way fault where $t = 2$.
 - In practice, a majority of software faults consist of simple and pairwise faults.
- Unique combinations
 - Latin Squares
 - Pairwise Testing
 - Orthogonal Array

BITS Pilani, Pilani Campus

8

Model the input



- P1 = (A,B)
- P2 = (1,2,3)
- P3 = (X, Y)
- Total Test Cases – 12
- Simple Fault - Each parameter value must be covered in at least one test case

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

BITS Pilani, Pilani Campus

9

Model the input



- P1 = (A,B)
- P2 = (1,2,3)
- P3 = (X, Y)
- Total Test Cases – 12
- 2-Way Fault - Every combination of values of two parameters are covered in at least one test case.
- Also called as Pairwise coverage

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

BITS Pilani, Pilani Campus

10

Model the input

- P1 = (A,B)
- P2 = (1,2,3)
- P3 = (X, Y)
- Total Test Cases – 12

Base choice coverage

- For each parameter, one of the possible values is designated as a base choice of the parameter
- A base test is formed by using the base choice for each parameter
- Subsequent tests are chosen by holding all base choices constant, except for one, which is replaced using a non-base choice of the corresponding parameter:

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

P1	P2	P3
A	1	X
A	1	Y
A	2	X
A	2	Y
A	3	X
A	3	Y
B	1	X
B	1	Y
B	2	X
B	2	Y
B	3	X
B	3	Y

BITS Pilani, Pilani Campus

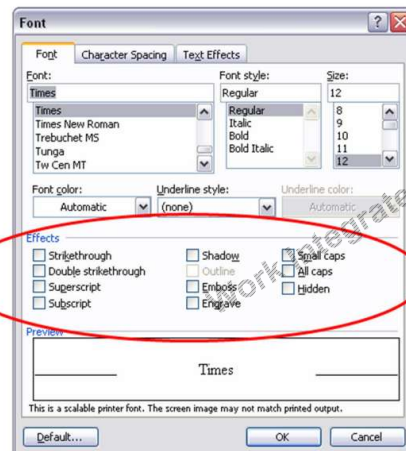
11

3-Way coverage

3-way coverage -

- $10C3 = 120$
- Thus, there would be $120 \times 2^3 = 960$ Test case
- Since 3 Triples can be packed in each Test case, we need 320 TC

0 1 1 0 0 0 0 1 1 0



BITS Pilani, Pilani Campus

12

3-Way coverage



3-way coverage -

- Actually, only 13 tests cases are needed to cover 960 TC
- Finding covering arrays is NP hard.

0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	0	1	0	0	0	0	1
1	0	1	1	1	0	1	0	1	0	0	0
1	0	0	0	0	1	1	1	0	0	0	0
0	1	1	0	0	0	1	0	0	1	1	0
0	0	1	0	0	1	0	1	0	1	1	0
1	1	0	1	0	0	1	0	1	0	1	0
0	0	0	1	1	1	1	0	0	1	1	1
0	0	1	1	0	0	1	0	1	0	0	1
0	1	0	1	1	0	0	1	0	0	1	0
1	0	0	0	1	1	0	0	0	1	1	1
0	1	0	0	0	0	1	1	0	1	0	1

BITS Pilani, Pilani Campus

13

Need of Combinatorial



OS - XP, OS X, RHL (3)
 Browser - IE, Firefox (2)
 Protocol - IPV4, IPV6 (2)
 CPU - Intel, AMD (2)
 DBMS - Sybase, Oracle, MySQL (3)

Total Possible Test case $3 \times 2 \times 2 \times 2 \times 3 = 72$

Test	OS	Browser	Protocol	CPU	DBMS
1	XP	IE	IPV4	Intel	MySQL
2	XP	Firefox	IPV6	AMD	Sybase
3	XP	IE	IPV6	Intel	Oracle
4	OS X	Firefox	IPV4	AMD	MySQL
5	OS X	IE	IPV4	Intel	Sybase
6	OS X	Firefox	IPV4	Intel	Oracle
7	RHL	IE	IPV6	AMD	MySQL
8	RHL	Firefox	IPV4	Intel	Sybase
9	RHL	Firefox	IPV4	AMD	Oracle
10	OS X	Firefox	IPV6	AMD	Oracle

t	#Configs	%age
2	10	14
3	18	25
4	36	50
5	72	100

BITS Pilani, Pilani Campus

14

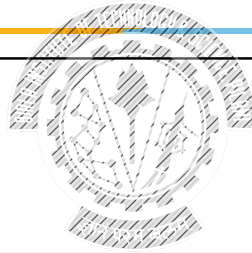
Pairwise Testing



- **Pairwise Testing** is a type of software testing in which permutation and combination method is used to test the software. Pairwise testing is used to test all the possible discrete combinations of the parameters involved.
- Pairwise testing is a P&C based method, in which to test a system or an application, for each pair of input parameters of a system, all possible discrete combinations of the parameters are tested. By using the conventional or exhaustive testing approach it may be hard to test the system but by using the permutation and combination method it can be easily done.

BITS Pilani, Pilani Campus

15



Pairwise Testing (Example)



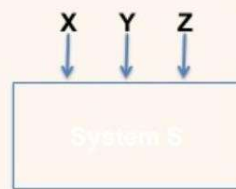
Problem: Test System S

Inputs

X = A,B

Y = C,D

Z = E,F



There are $2^3 = 8$ possible combinations

BITS Pilani, Pilani Campus

16

Pairwise Testing (Example)

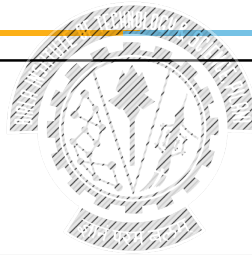


8 Test Cases			
Test ID	X	Y	Z
T1	A	C	E
T2	A	D	E
T3	A	C	F
T4	A	D	F
T5	B	C	E
T6	B	D	E
T7	B	C	F
T8	B	D	F

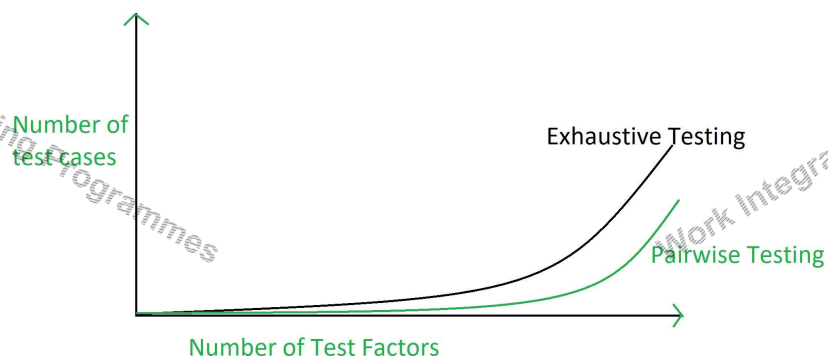
Pairwise testing 4 Test Cases			
Test ID	X	Y	Z
T1	A	C	E
T4	A	D	F
T6	B	D	E
T7	B	C	F

BITS Pilani, Pilani Campus

17



Pairwise Testing



BITS Pilani, Pilani Campus

18

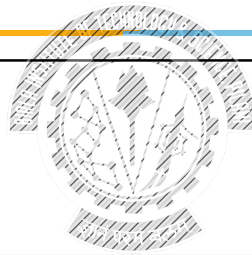
Advantages of Pairwise Testing



- Reduces the number of execution of test cases.
- Increases the test coverage almost up to hundred percentage.
- Increases the defect detection ratio.
- Takes less time to complete the execution of the test suite
- Reduces the overall testing budget for a project.

BITS Pilani, Pilani Campus

19



Disadvantages of Pairwise Testing



- Not beneficial if the values of the variables are inappropriate.
- It is possible to miss the highly probable combination while selecting the test data.
- Defect yield ratio may be reduced if a combination is missed.
- Not useful if combinations of variables are not understood correctly.

BITS Pilani, Pilani Campus

20

Coverage



- Studies such as “Estimating t-Way Fault Profile Evolution During Testing” and “Practical Combinatorial Testing” (presented by the National Institute of Standards and Technology in 2017 and 2010 respectively) indicate that the vast majority of defects (67%-93%) related to input values are due to a problem in either one parameter value (single-value fault) or a combination of two parameter values (2-way interaction fault).
- NIST research showed that most software bugs and failures are caused by one or two parameters, with progressively fewer by three or more, which means that combinatorial testing can provide more efficient fault detection than conventional methods. Multiple studies have shown fault detection equal to exhaustive testing with a 20X to 700X reduction in test set size. New algorithms compressing combinations into a small number of tests have made this method practical for industrial use, providing better testing at lower cost. (NIST)

BITS Pilani, Pilani Campus

21

Black Box Testing BVA



- Boundary Value Analysis
- Software is treated as a Black Box.
- Combination of Input is decided.
 - Actual o/p is matched with expected o/p
- Test case with {min, min+, nom, max-, max} values

BITS Pilani, Pilani Campus

22

Boundary Value Analysis



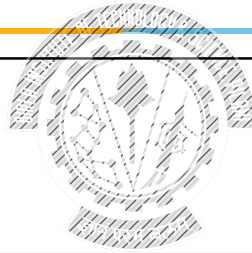
- Boundary Value Analysis focuses on the boundary of the input space to identify test cases
 - Rationale is, errors tend to occur near the extreme value of the input variable

Examples

- Loop counters off by 1
- Inputs at the boundary of ranges. $10 < x < 100$

BITS Pilani, Pilani Campus

23



Boundary Value Analysis



- Idea of BVA is to use input variable values at
 - Their minimum
 - Just above the minimum
 - A nominal value
 - Just below their maximum
 - At their maximum

BITS Pilani, Pilani Campus

24

BVA - Explore the types



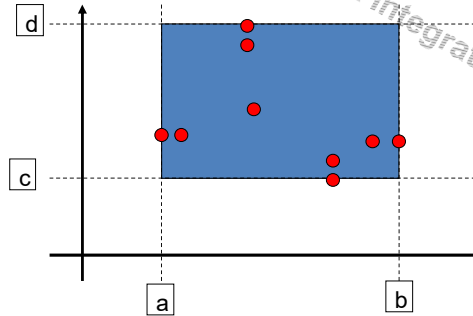
Example

A program takes 2 inputs x_1 and x_2

- $a \leq x_1 \leq b$
- $c \leq x_2 \leq d$

We have the intervals

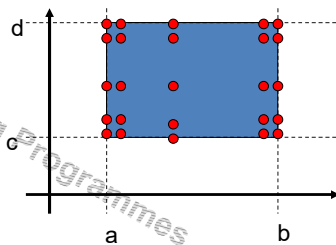
- $[a, b] \leftarrow x_1$
- $[c, d] \leftarrow x_2$
- BVA test cases for a function of two variables – single fault assumption
- $4n+1$ test cases



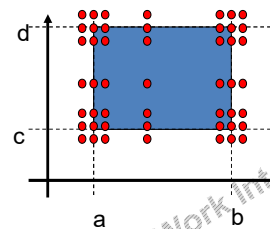
BITS Pilani, Pilani Campus

25

BVA - Worst Case Analysis



Worst Case



Robust Worst Case

BITS Pilani, Pilani Campus

26

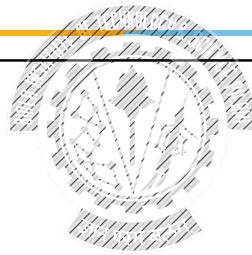
BVA Limitations



- BVA works well when the program to be tested is a function of several independent variables that represent bounded physical quantities
- No consideration to the functionality or semantic meaning of variables

BITS Pilani, Pilani Campus

27



Black Box Testing BVA (Boundary Value Analysis)



Example

The triangle program accepts three integers, a, b, and c, as input. These are taken to be the sides of a triangle. The integers a, b, and c must satisfy the following conditions:

C1: $1 \leq a \leq 200$ C4: $a < b + c$

C2: $1 \leq b \leq 200$ C5: $b < a + c$

C3: $1 \leq c \leq 200$ C6: $c < a + b$

The output of the program may be:
Equilateral, Isosceles, Scalene, or
"NOT-A-TRIANGLE."

Case ID	a	b	c	Expected output
1.	100	100	1	Isosceles
2.	100	100	2	Isosceles
3.	100	100	100	Equilateral
4.	100	100	199	Isosceles
5.	100	100	200	Not a triangle
6.	100	1	100	Isosceles
7.	100	2	100	Isosceles
8.	100	100	100	Equilateral
9.	100	199	100	Isosceles
10.	100	200	100	Not a triangle
11.	1	100	100	Isosceles
12.	2	100	100	Isosceles
13.	100	100	100	Equilateral
14.	199	100	100	Isosceles
15.	200	100	100	Not a triangle

Test Case 3, 8, 13 are same and hence 8, 13 are redundant and can be removed.

Total Test cases are $4n+1 = 4*3+1 = 13$

Ref : R2 - Software Testing By RAJIV CHOPRA.

BITS Pilani, Pilani Campus

28

Black Box Testing BVA (Boundary Value Analysis)



Next Date is a function of three variables: month, date, and year. It returns the date of next day as output. It reads current date as input date. The conditions are:

C1: $1 \leq \text{month} \leq 12$

C2: $1 \leq \text{day} \leq 31$

C3: $1900 \leq \text{year} \leq 2025$

If any of conditions C1, C2, or C3 fails, then this function produces an output "value of month not in the range 1...12."

Because many combinations of dates exist this function just displays one message: "Invalid Input Date."

Case ID	Month (mm)	Day (dd)	Year (yyyy)	Expected Output
1.	6	15	1900	16 June, 1900
2.	6	15	1901	16 June, 1901
3.	6	15	1962	16 June, 1962
4.	6	15	2024	16 June, 2024
5.	6	15	2025	16 June, 2025
6.	6	1	1962	2 June, 1962
7.	6	2	1962	1 June, 1962
8.	6	30	1962	1 July, 1962
9.	6	31	1962	Invalid date as June has 30 days.
10.	1	15	1962	16 January, 1962
11.	2	15	1962	16 February, 1962
12.	11	15	1962	16 November, 1962
13.	12	15	1962	16 December, 1962

Test cases have not covered Leap Year, 30 day months, etc

BITS Pilani, Pilani Campus

29

Equivalence class Testing



- The input and the output domain is divided into a finite number of equivalence classes.
- Eg select one representative of each class and test our program against it.
- Four types of equivalence class testing are discussed :
 1. **Weak normal** equivalence class testing – All Valid inputs
 2. **Strong normal** equivalence class testing - All Valid inputs, one from each sub-interval equivalence class
 3. **Weak robust** equivalence class testing – One invalid input
 4. **Strong robust** equivalence class testing – More than one invalid input

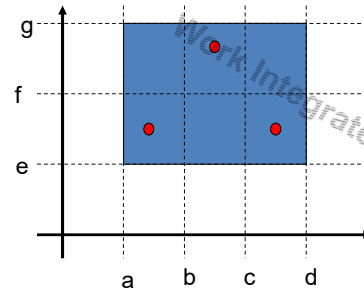
BITS Pilani, Pilani Campus

30

EC – Weak Normal



- One variable from each EC
- A systematic way of deriving the EC
- Same number of weak EC test cases as classes in the partition with the largest number of subsets
- Based on a single fault assumption
- Testing valid subdomains
- Assumption
 - Input variables are independent
 - One dimensional valid subdomains
- Selects tests from one dimensional (one variable) subdomains



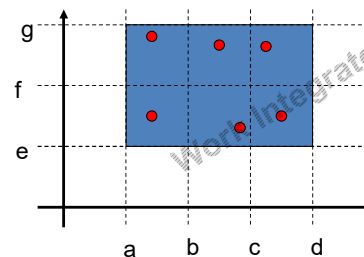
BITS Pilani, Pilani Campus

31

EC – Strong Normal



- Based on a multiple fault assumption
- We derive test cases out of the Cartesian product of equivalence classes
- Notion of “completeness”
- Testing valid subdomains
- Assumption
 - Input variables are related
 - Multidimensional subdomains. (Example)
- Test selection: Select at least one test from each of the multidimensional sub domain



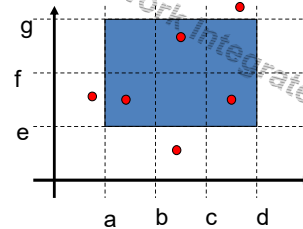
BITS Pilani, Pilani Campus

32

EC – Weak Robust



- Weak Robust is counterintuitive.
- Robust comes from the consideration of invalid values
- Weak refers to the single fault assumption
- A test case should have one invalid value and the remaining values should be valid
- One dimensional invalid subdomains



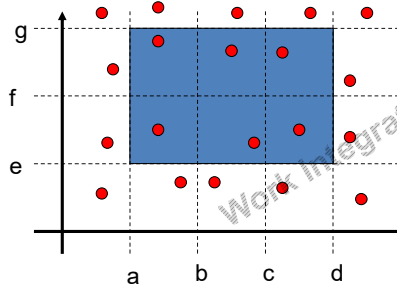
BITS Pilani, Pilani Campus

33

EC – Strong Robust



- Robust comes from consideration of invalid values for inputs
- Strong refers to the multiple fault assumption



BITS Pilani, Pilani Campus

34

Black Box Testing Equivalence Class Testing



Next Date is a function of three variables: month, date, and year. It returns the date of next day as output. It reads current date as input date. The conditions are:

M1: $1 \leq \text{month} \leq 12$

D1: $1 \leq \text{day} \leq 31$

Y1: $1812 \leq \text{year} \leq 2012$

Case ID	Month	Day	Year	Expected output
WN ₁ , SN ₁	6	15	1912	6/16/1912
Case ID	Month	Day	Year	Expected output
SR ₁	-1	15	1912	Invalid value of month as month cannot be -ve.
SR ₂	6	-1	1912	Invalid value of day as day is -ve.
SR ₃	6	15	1811	Invalid value of year.
SR ₄	-1	-1	1912	Invalid month and day.
SR ₅	6	-1	1811	Invalid day and year.
SR ₆	-1	15	1811	Invalid month and year.
SR ₇	-1	-1	1811	Invalid month, day, and year.
Case ID	Month	Day	Year	Expected output
WR ₁	6	15	1912	6/16/1912
WR ₂	-1	15	1912	Invalid value of month as month cannot be -ve.
WR ₃	13	15	1912	Invalid value of month as month <12, always.
WR ₄	6	-1	1912	Invalid value of day as day cannot be -ve.
WR ₅	6	32	1912	Invalid value of day as we cannot have 32 days in any month.
WR ₆	6	15	1811	Invalid value of year as its range is 1812 to 2012 only.
WR ₇	6	15	2013	Invalid value of year.

BITS Pilani, Pilani Campus

35

Black Box Testing Equivalence Class Testing



Next Date is a function of three variables: month, date, and year. It returns the date of next day as output. It reads current date as input date. The conditions are:

M1 = {month: month has 30 days} (1,3,5,7,8,10,12)

M2 = {month: month has 31 days} (4,6,9,11)

M3 = {month: month is February} (2)

D1 = {day: $1 \leq \text{day} \leq 28$ }

D2 = {day: day = 29}

D3 = {day: day = 30}

D4 = {day: day = 31}

Y1 = {year: year = 2000}

Y2 = {year: year is a leap year}

Y3 = {year: year is a common year}

Case ID	Month	Day	Year	Expected output
WN ₁	6	14	2000	6/15/2000
WN ₂	7	29	1996	7/30/1996
WN ₃	2	30	2002	2/31/2002 (Impossible)
WN ₄	6	31	2000	7/1/2000 (Impossible)

BITS Pilani, Pilani Campus

36

Black Box Testing Equivalence Class Testing



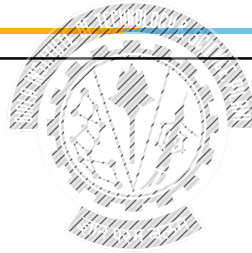
Total $3 \times 4 \times 3 = 36$ Test Cases

- For SR (Strong Robust), 2 invalid cases will be added for each input, hence total Test cases would be 150.

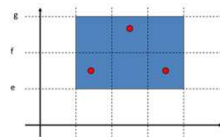
Case ID	Month	Day	Year	Expected output
SN ₁	6	14	2000	6/15/2000
SN ₂	6	14	1996	6/15/1996
SN ₃	6	14	2002	6/15/2002
SN ₄	6	29	2000	6/30/2000
SN ₅	6	29	1996	6/30/1996
SN ₆	6	29	2002	6/30/2002
SN ₇	6	30	2000	6/31/2000 (Impossible date)
SN ₈	6	30	1996	6/31/1996 (Impossible date)
SN ₉	6	30	2002	6/31/2002 (Impossible date)
SN ₁₀	6	31	2000	7/1/2000 (Invalid input)
SN ₁₁	6	31	1996	7/1/1996 (Invalid input)
SN ₁₂	6	31	2002	7/1/2002 (Invalid input)
SN ₁₃	7	14	2000	7/15/2000
SN ₁₄	7	14	1996	7/15/1996
SN ₁₅	7	14	2002	7/15/2002
SN ₁₆	7	29	2000	7/30/2000
SN ₁₇	7	29	1990	7/30/1990
SN ₁₈	7	29	2002	7/30/2002
SN ₁₉	7	30	2000	7/31/2000
SN ₂₀	7	30	1996	7/31/1996
SN ₂₁	7	30	2002	7/31/2002
SN ₂₂	7	31	2000	8/1/2000
SN ₂₃	7	31	1996	8/1/1996
SN ₂₄	7	31	2002	8/1/2002
SN ₂₅	2	14	2000	2/15/2000
SN ₂₆	2	14	1996	2/15/1996
SN ₂₇	2	14	2002	2/15/2002
SN ₂₈	2	29	2000	3/1/2000 (Invalid input)
SN ₂₉	2	29	1996	3/1/1996
SN ₃₀	2	29	2002	3/1/2002 (Impossible date)
SN ₃₁	2	30	2000	3/1/2000 (Impossible date)
SN ₃₂	2	30	1996	3/1/1996 (Impossible date)
SN ₃₃	2	30	2002	3/1/2002 (Impossible date)
SN ₃₄	6	31	2000	7/1/2000 (Impossible date)
SN ₃₅	6	31	1996	7/1/1996 (Impossible date)
SN ₃₆	6	31	2002	7/1/2002 (Impossible date)

BITS Pilani, Pilani Campus

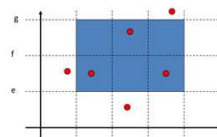
37



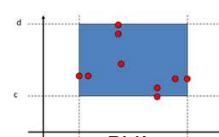
EC & BVA



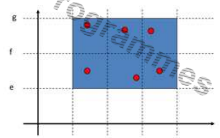
Weak Normal



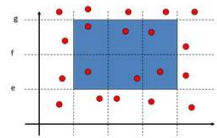
Weak Robust



BVA



Strong Normal



Strong Robust



Worst Case Analysis

BITS Pilani, Pilani Campus

38



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture - 07

BITS Pilani
Pilani Campus

Harish Aggarwal

Work Integrated Learning Programmes

1



BITS Pilani
Pilani Campus

Testing Methodology

Work Integrated Learning Programmes

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Frameworks
 - CMMI-Development
 - ITIL Framework
- Other Frameworks

Lecture - 04

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 05

- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing - Black Box, White Box Testing
- Test Levels and Types –
 - Unit, Integration, System Testing

BITS Pilani, Pilani Campus

3

Re-Cap



Lecture - 06

- Test Design Techniques
 - Combinatorial Testing
 - Boundary Value Analysis
 - Equivalence Class Partitioning

BITS Pilani, Pilani Campus

4

Agenda



- Test Methodology – Decision Table Testing
- Test Execution Process
- Test Case Design

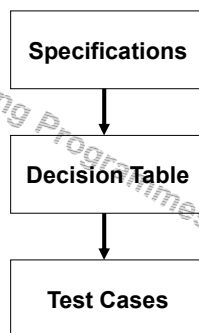
BITS Pilani, Pilani Campus

5

Decision Tables



Steps



- One of the most systematic approaches of designing Test Cases
- Decision Tables are rigorous – enforce logical rigor
- Related Method – Cause Effect Graphing
- Ideal for situations with number of combinations of actions are taken under varying sets of condition
- Structure of describing rules

BITS Pilani, Pilani Campus

6

Black Box Testing

Decision Table Based Testing



- A column in the entry portion is a **rule**.
- Rules indicate which actions are taken for the conditional circumstances indicated in the condition portion of the rule.
- Decision tables in which all conditions are binary are called limited entry decision tables.
- If conditions are allowed to have several values, the resulting tables are called extended entry decision tables.

To identify test cases with decision tables, we follow certain steps:

Step 1. For a module identify input conditions (causes) and action (effect).

Step 2. Develop a cause-effect graph.

Step 3. Transform this cause-effect graph, so obtained in step 2 to a decision table.

Step 4. Convert decision table rules to test cases. Each column of the decision table represents a test case. That is,

Number of Test Cases = Number of Rules
n conditions exist, there must be 2^n rules.

BITS Pilani, Pilani Campus

7

DT - Example



Component Specification: Input of 2 characters such that

1. The 1st character must be A or B.
2. The 2nd character must be a digit.
3. If the 1st character is A or B and the 2nd character is a digit the file is updated.
4. If the 1st character is incorrect, message X12 is displayed.
5. If the 2nd character is not a digit, message X13 is displayed.

Develop test cases using Decision table technique

BITS Pilani, Pilani Campus

8

DT Example Solution



Stub	R1	R2	R3	R4	R5	R6	R7	R8
C1: 1st Char is A	T	T	T	T	F	F	F	F
C2: 1st Char is B	T	T	F	F	T	T	F	F
C3: 2nd char is Digit	T	F	T	F	T	F	T	F
A1: File is updated			X		X			
A2: Message X:12							X	X
A3: Message X:13				X		X		X
A4: Impossible	X	X						

Each Rule one Test Case (minimum)

Test Case #	Input	Rule
Test Case 1	A5	R3
Test Case 2	AC	R4
Test Case 3	B5	R5
Test Case 4	BC	R6
Test Case 5	C5	R7
Test Case 6	CD	R8

Note that it may not be possible to execute or even design test cases for the impossible action rule. But to begin with never ignore these conditions. Design them and then evaluate if they can be included in the Test Suite.

BITS Pilani, Pilani Campus

9

Black Box Testing Decision Table Based Testing



Next Date is a function of three variables: month, date, and year. It returns the date of next day as output. It reads current date as input date.

- M1 = {month: month has 30 days}
- M2 = {month: month has 31 days except December}
- M3 = {month: month is December}
- M4 = {month: month is February}
- D1 = {day: $1 \leq \text{day} \leq 27$ }
- D2 = {day: day = 28}
- D3 = {day: day = 29}
- D4 = {day: day = 30}
- D5 = {day: day = 31}
- Y1 = {year: year is a leap year}
- Y2 = {year: year is a common year}

BITS Pilani, Pilani Campus

10

Black Box Testing Decision Table Based Testing



Rules → Conditions ↓	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
C1: month in	M1	M1	M1	M1	M1	M2	M2	M2	M2	M2	M3	M3	M3	M3	M3	M4	M4	M4	M4	M4	M4	M4
C2: day in	D1	D2	D3	D4	D5	D1	D2	D3	D4	D5	D1	D2	D3	D4	D5	D1	D2	D3	D3	D3	D4	D5
C3: year in	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	Y1	Y2	Y1	Y2	-	-
Actions																						
a1: impossible					x																	x
a2: increment day	x	x	x			x	x	x	x		x	x	x	x		x	x					
a3: reset day				x						x					x			x	x			
a4: increment month				x						x								x	x			
a5: reset month															x							
a6: increment year															x							

BITS Pilani, Pilani Campus

11

Black Box Testing Decision Table Based Testing



Case ID	Month	Day	Year	Expected output
1-3	April	15	2001	April 16, 2001
4	April	30	2001	May 1, 2001
5	April	31	2001	Impossible
6-9	January	15	2001	January 16, 2001
10	January	31	2001	February 1, 2001
11-14	December	15	2001	December 16, 2001
15	December	31	2001	January 1, 2002
16	February	15	2001	February 16, 2001
17	February	28	2004	February 29, 2004
18	February	28	2001	March 1, 2001
19	February	29	2004	March 1, 2004
20	February	29	2001	Impossible
21-22	February	30	2001	Impossible

BITS Pilani, Pilani Campus

12

Decision Table – Triangle problem



Example

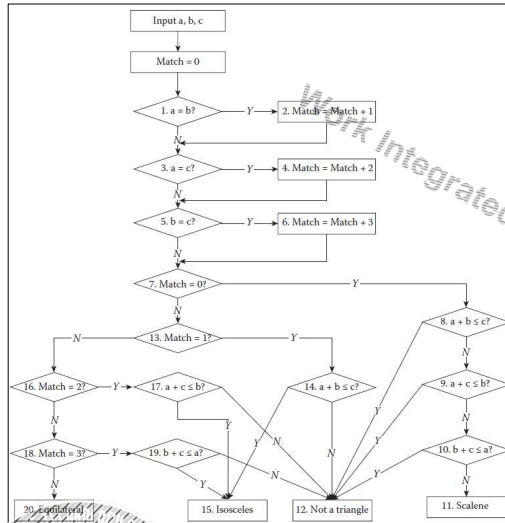
The triangle program accepts three integers, a, b, and c, as input. These are taken to be the sides of a triangle. The integers a, b, and c must satisfy the following conditions:

C1: $1 \leq a \leq 200$ C4: $a < b + c$

C2: $1 \leq b \leq 200$ C5: $b < a + c$

C3: $1 \leq c \leq 200$ C6: $c < a + b$

The output of the program may be:
Equilateral, Isosceles, Scalene, or
"NOT-A-TRIANGLE".



BITS Pilani, Pilani Campus

13

Decision Table – Triangle problem



	R ₁	R ₂	R ₃	R ₄	R ₅	R ₆	R ₇	R ₈	R ₉	R ₁₀	R ₁₁	
C ₁ : $a < b + c$?	F	T	T	T	T	T	T	T	T	T	T	
C ₂ : $b < a + c$?	—	F	T	T	T	T	T	T	T	T	T	
C ₃ : $c < a + b$?	—	—	F	T	T	T	T	T	T	T	T	
C ₄ : $a = b$?	—	—	—	T	T	T	T	F	F	F	F	
C ₅ : $a = c$?	—	—	—	T	T	F	F	T	T	F	F	
C ₆ : $b = c$?	—	—	—	T	F	T	F	T	F	T	F	
Rule Count	32	16	8	1	1	1	1	1	1	1	1	= 64
a ₁ : Not a triangle	x	x	x									
a ₂ : Scalene											x	
a ₃ : Isosceles							x		x	x		
a ₄ : Equilateral				x								
a ₅ : Impossible					x	x		x				

Ref. – Software Testing by Rajiv Chopra

BITS Pilani, Pilani Campus

14

Decision Table – Triangle problem



Case ID	a	b	c	Expected output
D ₁	4	1	2	Not a triangle
D ₂	1	4	2	Not a triangle
D ₃	1	2	4	Not a triangle
D ₄	5	5	5	Equilateral
D ₅	?	?	?	Impossible
D ₆	?	?	?	Impossible
D ₇	2	2	3	Isosceles
D ₈	?	?	?	Impossible
D ₉	2	3	2	Isosceles
D ₁₀	3	2	2	Isosceles
D ₁₁	3	4	5	Scalene

Ref: – Software Testing by Rajiv Chopra

BITS Pilani, Pilani Campus

15



Work Integrated Learning Programmes



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture - 08

BITS Pilani
Pilani Campus

Harish Aggarwal

1



BITS Pilani
Pilani Campus

Revision

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Frameworks
 - CMMI-Development
 - ITIL Framework
- Other Frameworks

Lecture - 04

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 05

- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing - Black Box, White Box Testing
- Test Levels and Types –
 - Unit, Integration, System Testing

BITS Pilani, Pilani Campus

3

Re-Cap



Lecture - 06

- Test Design Techniques
 - Combinatorial Testing
 - Boundary Value Analysis
 - Equivalence Class Partitioning

Lecture - 07

- Test Methodology
 - Decision Table Testing

BITS Pilani, Pilani Campus

4



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture - 09

BITS Pilani
Pilani Campus

Harish Aggarwal

Work Integrated Learning Programmes

1



BITS Pilani
Pilani Campus

innovate achieve lead

Test Execution

Work Integrated Learning Programmes

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Frameworks
 - CMMI-Development
 - ITIL Framework
 - Other Frameworks

Lecture - 04

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 05

- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing - Black Box, White Box Testing
- Test Levels and Types –
 - Unit, Integration, System Testing

BITS Pilani, Pilani Campus

3

Re-Cap



Lecture - 06

- Test Design Techniques
 - Combinatorial Testing
 - Boundary Value Analysis
 - Equivalence Class Partitioning

Lecture - 07

- Test Methodology
 - Decision Table Testing

Lecture - 08

- Revision

BITS Pilani, Pilani Campus

4

Agenda



- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

BITS Pilani, Pilani Campus

5

Software Test Execution



Goal -

- Design Test Cases which cover vast area of Software thoroughly and accurately
- Less number of Test Cases
- Improved Quality

Testing Phases

- Determine Test Methodology.
- Planning Tests
- Designing Tests
- Performing the Tests

BITS Pilani, Pilani Campus

6

Software Test Strategy



Decision to be made in Test Strategy -

- Approach – Top-Down, Bottom-Up, Sandwich, Big-Bang
- Which parts should be tested with White-Box Model
- Which parts should be tested with Black-Box Model
- How much and which area to be covered by Automated Tests

Planning Tests

- Unit Tests
- Integrations Tests
- System Tests

BITS Pilani, Pilani Campus

7

Software Test Strategy



- What to Test.
 - Avoid duplication in case of reused software.
 - Criticality of the area.
 - Newer versions of stable software. Added modules/feature.
- Who performs the Tests
 - Unit Test – Development Team
 - System Tests – Independent Test Team (Internal or External).
- Where to Perform Tests
 - Unit Test or Integrations Tests – Development Site
 - System tests – Development Site or Consultant Site or Customer Site.
- How much to Tests
 - When all the Tests in Test plan are executed and no issues are found.
 - Achieve certain minimum level of Error discovery rate.
 - Resources have ended – Time Limit, Budget.

BITS Pilani, Pilani Campus

8

Software Test Design



Test Design

- Scope of the Test
 - Software Package, module to be Tested.
- Testing Environment
 - Testing Site
 - Hardware and firmware configuration
- Test Details
 - Test Case ID
 - Objective
 - Design Doc reference or requirement doc, if any
 - Test level – Unit, integration, System
 - Pre-requisites
 - Data to be recorded.
 - Steps for execution

BITS Pilani, Pilani Campus

9

Test Prioritization



- Test cases can be prioritized.
- Priority setting can be based on
 - Type of product
 - Organization size/culture
 - Resources available
- Sample Prioritization
 - P0 – Tests that need to pass before full testing is started
 - P1 - This test must be executed before final delivery.
 - P2 - If time permits, execute this test.
 - P3 - This test can wait after the delivery date.

BITS Pilani, Pilani Campus

10

Software Test Execution



- Test execution Process
 - Execute Test cases → Find Defects → Fix Defects → Re-Test
 - Re-Test is called as regression Testing
 - To find if the Defect fixed is properly fixed
 - To find if no new Defects is created while fixing the old Defect
 - Regression Testing can be execution of full Test suit or its subset.
 - Results of every Test Run are recorded in Software Test Report (STR)

BITS Pilani, Pilani Campus

11

Software Test Report



- Test Identification
 - Tested software Identification – Version, Build, etc
 - Initiation Time, Concluding time, etc
 - Test Site
- Test Environment – H/W, firmware configuration
- Test Results
 - Test case ID
 - Test result – Pass/Failed
- Summary Results
 - Total Test case executed.
 - Total Pass/Fail
 - Comparison with previous results

BITS Pilani, Pilani Campus

12

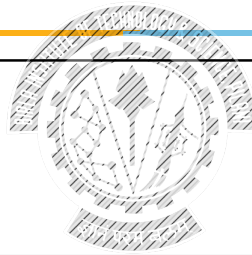
Automated Testing



- Why Automated Testing
 - Cost savings
 - Shortened test duration
 - Thoroughness
 - Test accuracy
 - Automated result reporting
 - Statistical processing and subsequent reporting of Results
- Process of Automated Testing
 - Just like Manual Testing, it requires –
 - Test planning, Test design, Test case preparation
 - Test Execution, Regression testing
 - Final Test log and report preparation including comparison reports

BITS Pilani, Pilani Campus

13



Types of Automated Testing



Types of Automated Tests

- Code Auditing
 - Check compliance with respect to coding standards.
 - Code complexity metrics (McCabe's cyclomatic complexity)
 - Levels of loop nesting
 - Levels of subroutine nesting
 - Prohibited constructs, such as GOTO
 - Check for Naming conventions for variables, files, etc.
 - Unreachable code lines of program or entire subroutines.
 - Format and size of comments
- Coverage monitoring
 - Line coverage achieved when implementing a given test case file.
 - Path Coverage achieved while the Tests are executed

BITS Pilani, Pilani Campus

14

Types of Automated Testing



- Functional Tests
 - Test Scripts are prepared which emulate the manual Tests
 - Test automation tools can be used to execute tests
 - Input data is picked up from Test DB Files.
 - Executing same test case with different input is easy.
 - Executing regression testing is easy.
- Load Tests
 - Creating Load Test scenario manually is impactable/impossible.
 - Can be done through simulations
 - Load is varied and response plotted in the form of a graph to see system behavior
 - System response Vs Environment configuration (Hardware and communication)

BITS Pilani, Pilani Campus

15

Automated Test Management



- Features of Automated Test Management
 - Test plans, Test results and correction follow-up.
 - Preparation of lists, tables and visual presentations of test plans
 - Listing of test case
 - Execution of automated software tests
 - Listing of test results
 - Listing of detected errors
 - Listing of correction schedule
 - Listing of uncompleted corrections for follow-up
 - Correction and Regression tests
 - Summary reports of testing and error correction follow-up
 - Summary reports for maintenance correction schedule

BITS Pilani, Pilani Campus

16

Automated Testing Advantages Vs Disadvantages



S. No.	Advantages	Disadvantages
1	Accuracy and completeness of Test Execution	High investments required in package purchasing and training
2	Accuracy of results log and summary reports	High package development investment costs
3	Comprehensive information	High manpower resources for test preparation
4	Few manpower resources for test execution	Considerable testing areas left uncovered
5	Shorter testing periods	
6	Performance of complete regression tests	
7	Performance of test classes beyond the scope of manual testing	

BITS Pilani, Pilani Campus

17

Automated Testing



- Automation testing is not a 100% replacement for Manual Testing
 - Does not work when the product itself is changing.
 - Works well in regression Testing
 - Test automation takes lots of efforts.
 - Maintenance cost of automated test cases is very high.
 - Resource Cost is high for Test Automation Engineers.
 - Trained resources are needed for Automated Test design, execution and maintenance.

BITS Pilani, Pilani Campus

18

Alpha and Beta Testing



- Not same as customer's acceptance Testing
- **Alpha Sites** - Software is put into real usage among some of the teams in the organization
 - E.g. Office getting deployed on Development team, HR team machines
 - Preparation of lists, tables and visual presentations of test plans
- **Beta Sites** –
 - Bugs found in formal Testing and alpha sites are already fixed.
 - Product is close to final product with very few known bugs
 - Product is ready to be used by customers
 - Before releasing to all the customers, product is given to select few customers called as Beta Sites

BITS Pilani, Pilani Campus

19

Alpha and Beta Testing



S. No.	Advantages	Disadvantages
1	Identification of unexpected errors	A lack of systematic testing
2	A wider population in search of errors	Low quality error reports. Repro not always available
3	Low costs	Difficult to reproduce the test environment
4	Real life data and scenarios can be tested	Much effort is required to examine reports
5	Users perspective known	Loss of real data on users machines

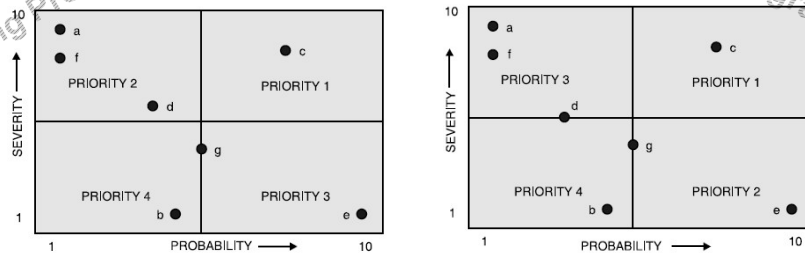
BITS Pilani, Pilani Campus

20

Defect Prioritization

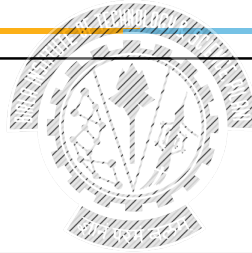


- Defects can be prioritized based on severity and probability of occurrence
- Fixing of defects by development team is based on defect priority.



BITS Pilani, Pilani Campus

21



Defect Prioritization



Defects that unblock the Testing are given high priority and fixed early

Example Priority and Severity

- High Severity, Low priority
 - Incorrect Logo.
- High priority, Low Severity
 - Slow response to transaction.
 - No functionality break but an irritant to Test team
 - Hampers the productivity of Test team
- High priority, High Severity
 - Data corruption issues
- Low priority, Low Severity
 - Incorrect Message on Pop-Up

BITS Pilani, Pilani Campus

22

Regression Testing

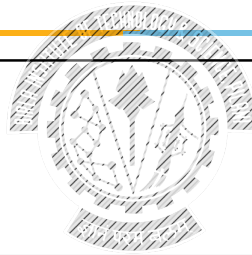


- Regression Testing is done to ensure that everything is working perfectly which was previously working.
- Regression testing is done after enhancements or Defect fixes.

S. No.	Normal testing	Regression testing
1	It is usually done during fourth phase of SDLC.	It is done during the maintenance phase.
2	It is basically software's verification and validation.	It is also called program revalidation.
3	New test suites are used to test our code.	Both old and new test cases can be used for testing.
4	It is done on the original software.	It is done on modified software.
5	It is cheaper.	It is a costlier test plan.

BITS Pilani, Pilani Campus

23



Types of Regression Testing



Types of Regression Testing

- Corrective regression testing – No change in specification. Same Test Suit
- Progressive regression testing – Change in specification. Modified Test Suit.
- Retest-all regression testing – Full Re-Test
- Selective regression testing – Test only the impacted and dependent areas.
- Regression Testing can be done at
 - Unit level, Integration level, System Level
- Criteria to decide when to stop regression testing.
 - Time constraints
 - Test criticality
 - Customer requirements

BITS Pilani, Pilani Campus

24

Source Code Analyzers



- Source code analysis tools, also known as Static Code Analyzers.
- No execution of program. Just static code analysis.
- To ensure code is compliant with a predefined set of rules or best practices.
- Static analysis is generally good at finding coding issues such as
 - Programming errors
 - Coding standard violations
 - Undefined values
 - Syntax violations
 - Security vulnerabilities

BITS Pilani, Pilani Campus

25

Source Code Analyzers



- Advantages
 - Speed
 - Depth
 - Accuracy
- Disadvantages
 - False Positives

BITS Pilani, Pilani Campus

26

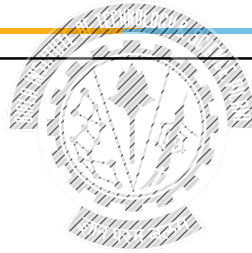
Software Composition Tools



- Software composition analysis (SCA) is an automated process that identifies the open source software in a codebase. This analysis is performed to evaluate security, license compliance, and code quality.
- It identifies all open source packages in an application and all the known vulnerabilities of those packages
- **Create an accurate Bill of Materials (BOM)** - components included, the version of the components used, and the license types for each.
- **Discover and track all open source** - Uncover all open source used in source code, binaries, containers, build dependencies, subcomponents.
- **Set and enforce policies** – Ensure OSS license compliance.
- **Enable proactive and continuous monitoring** -

BITS Pilani, Pilani Campus

27



Work Integrated Learning Programmes



 **SOFTWARE QUALITY ASSURANCE AND TESTING**

Lecture -10

BITS Pilani
Pilani Campus

Harish Aggarwal

1



 **BITS Pilani**
Pilani Campus

Reviews

2

Re-Cap



Lecture - 01

- Definitions – S/W Quality, S/W Quality Assurance
- Importance of QA
- Causes of Defects
- Business Models in S/W
- S/W Cost, Quality Cost
- Quality culture

Lecture - 02

- Software Quality Models
 - McCall, IEEE 1061, ISO 25000 Series
- Quality Requirements
- Quality of Requirements
- Frameworks
 - ISO/IEC/IEEE 12207 - Software Life Cycle Processes

Lecture - 03

- Frameworks
 - CMMI-Development
 - ITIL Framework
 - Other Frameworks

Lecture - 04

- ISO 25010 – Quality Attributes
- Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability

Lecture - 05

- Software Testing Fundamentals
- Software Verification and Validation
- Types of Testing - Black Box, White Box Testing
- Test Levels and Types –
 - Unit, Integration, System Testing

BITS Pilani, Pilani Campus

3

Re-Cap



Lecture - 06

- Test Design Techniques
 - Combinatorial Testing
 - Boundary Value Analysis
 - Equivalence Class Partitioning

Lecture - 07

- Test Methodology
 - Decision Table Testing

Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

BITS Pilani, Pilani Campus

4

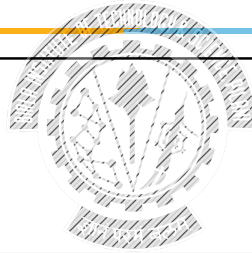
Agenda



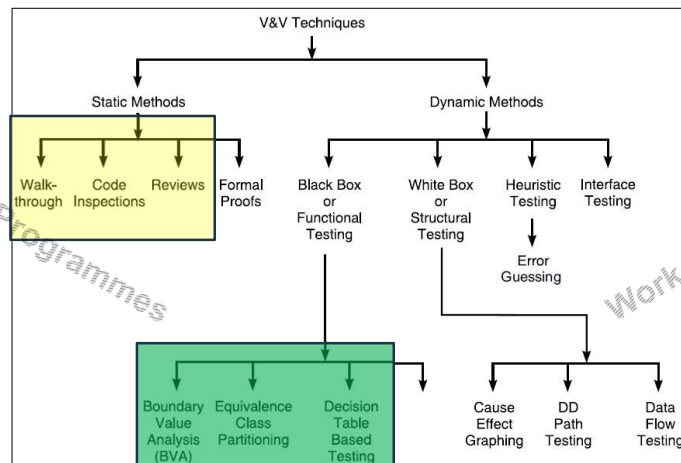
- Reviews
 - Informal Reviews
 - Personal Review
 - Peer Reviews
 - Formal Reviews
 - Walkthrough
 - Inspection Review
 - Audit
- Types of Audits (Internal, Third Party)
- Project Assessment and Control Process
- Corrective Actions

BITS Pilani, Pilani Campus

5



Reviews



BITS Pilani, Pilani Campus

6

Definitions



IEEE Standard for Software Reviews and Audits [IEEE 1028]

Review : A process or meeting during which a software product, set of software products, or a software process is presented to project personnel, managers, users, customers, user representatives, auditors or other interested parties for examination, comment or approval [IEEE 1028]

Review : A process or meeting during which a work product, or set of work products, is presented to project personnel, managers, users, customers, or other interested parties for comment or approval. [ISO 24765]

Management Review: A systematic evaluation of a software product or process performed by or on behalf of management that monitors progress, determines the status of plans and schedules, confirms requirements and their system allocation, or evaluates the effectiveness of management approaches used to achieve fitness for purpose [IEEE 1028]

BITS Pilani, Pilani Campus

7

Standards and Models for Review



- ISO/IEC 20246 Software and Systems Engineering: Work Product Reviews
 - To provide an International Standard that defines work product reviews, such as inspections, reviews and walk-throughs
- CMMI® for Development (CMMI-DEV)
 - In this model, a part of the "Verification" process area (Maturity Level -3) is devoted to peer reviews.

IEEE 1028 - Standard for Software Reviews and Audits.

- This standard provides minimum acceptable requirements for systematic software reviews

Standards that require use of systematic Reviews

- ISO/IEC/IEEE 12207 - Software Life Cycle Processes
- IEEE 1012 - IEEE Standard for System and Software Verification and Validation.
- IEEE 730 - IEEE Standard for Software Quality Assurance Processes

BITS Pilani, Pilani Campus

8

Informal Vs Formal Reviews



• Informal Reviews

- No documented process to describe reviews.
- Carried out by different people in the organization.
- Participants' roles are not defined
- Reviews have no objective, such as fault detection rate.
- They are not planned, they are improvised
- No Metrics - Measures, such as the number of defects, are not collected
- The effectiveness of reviews is not monitored by management
- There is no standard that describes them
- No checklist is used to identify defects.

• Formal Reviews

- Walk-Through
- Inspection
- Audit

BITS Pilani, Pilani Campus

9

Other Review Types



Other types of Reviews

- **Peer Reviews** – Just by colleagues. No Management involved.
- **Personal Review** – Done by person who produced the document, Code, etc. It is like Self-Review.
 - Practice to be followed in Personal Review –
 - Pause between the development of a software product and its review
 - Examine products in hard copy rather than electronically
 - Check each item on the checklist once completed
 - Update the checklists periodically to adjust to your personal data
 - Build and use a different checklist for each software product
 - Verify complex or critical elements with an in depth analysis.

BITS Pilani, Pilani Campus

10

Other Review Types



Other types of Reviews

- **Desk-Check Reviews**
 - A type of Peer Review, sometimes called as Pass around
 - Author to identify reviewers.
 - Author to come up with checklist designed specifically for document to be reviewed.
 - Document and Review form provided to Reviewers
 - Reviewers to provide feedback on Review forms.
 - Review meeting may or may not happen.
 - Not a very formal process.
 - Review forms can be used as evidence of review in SQA Audit.

BITS Pilani, Pilani Campus

11

IEEE 1028 - Types of Reviews



IEEE 1028 - Types of Reviews

- **Management Reviews** - A systematic evaluation of a software product or process performed by or on behalf of the management that monitors progress, determines the status of plans and schedules, confirms requirements and their system allocation, or evaluates the effectiveness of the management approaches used to achieve fitness for purpose
- **Technical reviews** - A systematic evaluation of a software product by a team of qualified personnel that examines the suitability of the software product for its intended use and identifies discrepancies from specifications and standards.
- **Inspections** - A visual examination of a software product to detect and identify software anomalies including errors and deviations from standards and specifications.
- **Walk-throughs** - A static analysis technique in which a designer or programmer leads members of the development team and other interested parties through a software product, and the participants ask questions and make comments about any anomalies, violation of development standards, and other problems.
- **Audits** - An independent assessment, by a third party, of a software product, a process or a set of software processes to determine compliance with the specifications, standards, contractual agreements, or other criteria.

BITS Pilani, Pilani Campus

12

IEEE 1028 - Sections of each Review



Each Review has sections

- **Introduction** to review: Describes the objectives of the systematic review and
- **Responsibilities** : Defines the roles and responsibilities needed for the review.
- **Input**: Describes the requirements for input needed by the review;
- **Entry Criteria**: Describes the criteria to be met before the systematic review can begin,
 - Authorization
 - Initiating event
- **Procedures**: Details the procedures for the systematic review
 - Planning the review
 - Overview of procedures
 - Preparation
 - Examination/evaluation/recording of results
 - Rework/follow-up
- **Exit Criteria** : Describe the criteria to be met before the systematic review can be considered complete
- **Output** : Describes the minimum set of deliverables to be produced by the review.

BITS Pilani, Pilani Campus

13

IEEE 1028 - Application



Applies to

- **Processes** related to –
 - Acquisition of software
 - Supply
 - Development
 - Operation and
 - Maintenance processes
- **Products**
 - Requirements and Specifications
 - Architecture
 - Code
 - Defect reports
 - Contracts and
 - Plans.

BITS Pilani, Pilani Campus

14

IEEE 1028 - Walkthrough

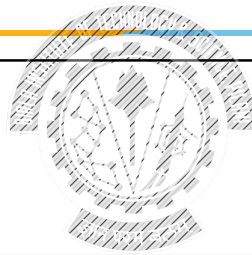


Roles and Responsibilities

- Walk-through leader
 - Conduct the walk-through
 - Distributing documents and arranging the meeting
 - Prepare the statement of objectives to guide the team through the walkthrough;
 - Issue the walk-through output (CheckList).
- Recorder
 - Note all comments made during the walk-through that pertain to anomalies found, questions of style, omissions, contradictions, suggestions for improvement, or alternative approaches.
- Author
 - Present the software product in the walk-through
- Team member

BITS Pilani, Pilani Campus

15



IEEE 1028 - Inspections



More thorough and Formal than Walk through

Roles and Responsibilities

- Inspection leader
 - Planning and organizational tasks pertaining to the inspection
 - Prepares Documents to be inspected. Issues Check List
- Recorder
 - Documents anomalies, action items, decisions, waivers, and recommendations made by the inspection team
- Reader
 - Lead the inspection team through the software product in a comprehensive and logical fashion, interpreting sections of the work and highlighting important aspects.
- Author
 - Ensures product to meet its inspection entry criteria and does any rework to meet exit criteria.
- Inspector
 - Identify and describe anomalies in the software product. Should have expertise in the area
 - Represent different viewpoints - For example, sponsor, end user, requirements, design, code, safety, test, independent test, project management, quality management, etc
 - Inspectors should be assigned specific topics to ensure effective coverage.

BITS Pilani, Pilani Campus

16

IEEE 1028 – Inspections



- Measuring Rate of Inspection
 - Time Taken
 - Defects/Issues found
 - Fix Rate
- Author Effectiveness
- Reviewer Effectiveness

Type of document inspected	Inspection rate
Architecture	2 PPH to 3 PPH (see NOTE 1)
Requirements	2 PPH to 3 PPH
Preliminary design	3 PPH to 4 PPH
Detailed design	3 PPH to 4 PPH
Source code	100 PPH to 200 LPH (see NOTE 2)
Test plan	5 PPH to 7 PPH
Fixes and changes	50 PPH to 75 LPH
User documentation	8 PPH to 20 PPH
NOTE 1—Page per hour.	
NOTE 2—Lines (of code) per hour.	

BITS Pilani, Pilani Campus

17

IEEE 1028 – Products to be Reviewed



In Either Inspection or Walkthrough

- Software requirements specification
- Software design description
- Source code
- Software test documentation
- Software user documentation
- Maintenance manual
- System build procedures
- Installation procedures
- Release notes
- Software models
- Specifications
- Software development process descriptions
- Policies, strategies, and plans
- Marketing and publicity documents
- Software architectural descriptions

BITS Pilani, Pilani Campus

18

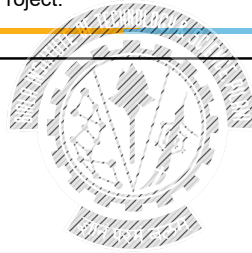
When to Review



- Product Launch Review
 - More of a Management review
 - Identify, review and ensure to have the milestone dates, requirements, schedule, budget constraints, deliverables, members of the development team, suppliers, etc.
 - Identify and define roles and responsibilities.
- Review at the end of any Phase
 - Reviews should be done at the end of every phase.
 - Ensure that Entry criteria for next phase is met before entering the next phase
- Retrospective Review (Also called as Post Mortem)
 - After the project is completed
 - What went well.
 - What could have been improved
 - Focus on Product and Process rather than Individual.
 - Report presented to Management
 - Learnings for the future Project.

BITS Pilani, Pilani Campus

19



Audits



- Most formal type of Reviews
- Provide an independent evaluation of conformance of software products and processes to applicable regulations, standards, guidelines, plans, specifications, and procedures.
 - Independent
 - Evaluation and conformance
 - Software Products and Processes
 - Applicable to
 - Regulations
 - Standards
 - Guidelines
 - Plans
 - Specification
 - Procedures

BITS Pilani, Pilani Campus

20

Types of Audits



- Internal or First Party Audit
 - Conducted Internally by the organization itself
 - Can be used as self-declaration of conformity
 - Can focus on security and fraud detection
 - Identification of inefficiencies.
- External
 - Second Party – Parties who have interest, like customers, suppliers, etc
 - Third Party – Independent auditing organizations
 - Regulators
 - Those providing certifications.

BITS Pilani, Pilani Campus

21

IEEE 1028 - Audits Applicable to



- | | |
|--|--|
| <ul style="list-style-type: none"> • Backup and recovery plans • Contingency plans • Contracts • Customer or user representative complaints • Disaster plans • Hardware performance plans • Installation plans • Installation procedures • Maintenance plans • Management review reports • Operations and user manuals • Procurement and contracting methods • Reports and data - Review, Audit, Project Status, Anomaly Reports, Test Data • Request for proposal • Risk management plans • Software configuration management plans | <ul style="list-style-type: none"> • Software design descriptions • Source code • Unit development folders • Software project management plans • Software quality assurance plans • Software requirements specifications • Software safety plans • Software test documentation • Software user documentation • Software V&V plans • Software architectural descriptions • Standards, regulations, guidelines, plans, specifications, and procedures • System build procedures • Technical review reports • Vendor documents • Walk-through reports |
|--|--|

BITS Pilani, Pilani Campus

22

IEEE 1028 – Audits



Roles and Responsibilities

- Lead auditor
 - Preparing the audit plan
 - Assembling the audit team
 - Managing the audit team
 - Making decisions regarding the conduct of the audit
 - Making decisions regarding any audit observations
 - Preparing the audit report
 - Recommending corrective actions
- Recorder
 - Document anomalies, action items, decisions, and recommendations made by the audit team.
- Auditor(s)
 - Examine products, as defined in the audit plan.

BITS Pilani, Pilani Campus

23

IEEE 1028 – Audits



Roles and Responsibilities

- Initiator
 - Decide upon the need for an audit
 - Decide upon the purpose and scope of the audit
 - Decide the software products or processes to be audited
 - Decide the evaluation criteria, including the regulations, standards, guidelines, plans, specifications, and procedures to be used for evaluation
 - Decide who will carry out the audit
 - Review the audit report
 - Decide what follow-up action will be required
 - Distribute the audit report
- Audited organization
 - Provide a liaison to the auditors
 - Provide all information requested by the auditors.
 - Implement corrective actions and recommendations

BITS Pilani, Pilani Campus

24

IEEE 1028 - Audit Steps

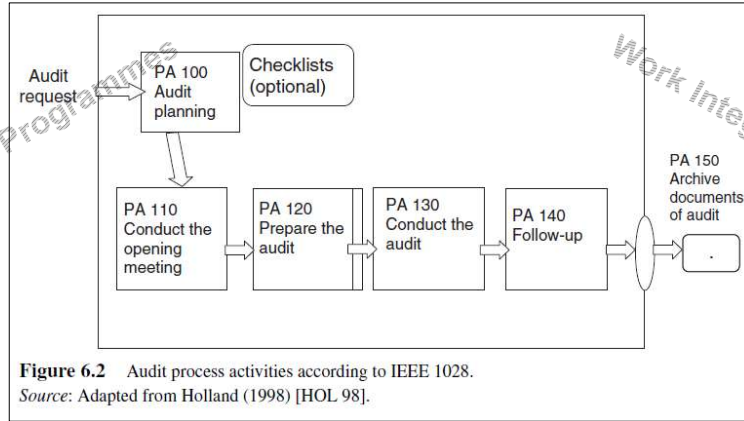


Figure 6.2 Audit process activities according to IEEE 1028.
Source: Adapted from Holland (1998) [HOL 98].

BITS Pilani, Pilani Campus

25

Audit as per CMMI-Dev

- In the CMMI, audits are activities described as part of the “process and product quality assurance
- To ensure objectivity, following issues must be addressed
 - Description of the reporting structure of the SQA to show its independence
 - Establish and maintain clearly formulated assessment criteria
 - What is going to be assessed?
 - When or how will the process be assessed?
 - How will the assessment be conducted?
 - Who must be involved in the assessment?
 - What product of an activity will be assessed?
 - When and how will the product of an activity be assessed?
 - Use the formulated criteria to assess the conformity of the -
 - Process descriptions
 - Standards
 - Procedures Executed

BITS Pilani, Pilani Campus

26

Project Assessment and Control Process



- Purpose of Project Assessment and control
 - To assess if the plans are aligned and feasible
 - To determine the status of the project, technical and process performance
 - To Ensure that the performance is as per
 - Plans and Schedules,
 - Within projected budgets, and
 - Satisfies technical objectives.
 - To Conduct of management and technical reviews, audits, and inspections.

BITS Pilani, Pilani Campus

27

Corrective Actions (CA)



- After an internal or external audit, an organization must perform CAs to correct the observed deficiencies
- CMMI has a process area named "Causal analysis and resolution" to identify and eliminate root causes

Corrective Action -

Action to eliminate the cause of a nonconformity and to prevent recurrence.

- Note 1: There can be more than one cause for a nonconformity.
- Note 2: Corrective action is taken to prevent recurrence whereas preventive action is taken to prevent occurrence

- An intentional activity that realigns the performance of the project work with the project management plan.

ISO 9000 [ISO 15b]

PMBOK® Guide [PMI 13]

Preventive Action -

- An intentional activity that ensures the future performance of the project work is aligned with the project management plan.

PMBOK® Guide [PMI 13]

BITS Pilani, Pilani Campus

28

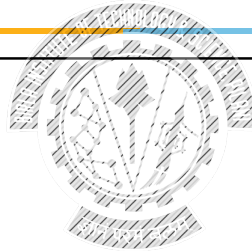
Corrective Actions (CA) Process



- Inputs:
 - An audit report
 - A non-compliance or a problem report
- Activities:
 - Register non-conformities in the issue tracking tool
 - Analyze and validate the problem
 - Classify and prioritize the issue/problem
 - Propose a solution to the problem
 - Solve the problem and ensure that the resolution does not cause other problems,
 - Analyze the problems to conduct trend analysis that can be identified and addressed,
 - If it cannot be resolved within the project, refer it to the appropriate management level
 - Verify the problem resolution
 - Inform stakeholders of the problem resolution
 - Archive the problem documentation
 - Update the information in the issue tracking tool
- Outputs
 - The resolution file
 - A corrected version of the software.

BITS Pilani, Pilani Campus

29



Work Integrated Learning Programmes




SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -11

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap

Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes	Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance, Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – - Unit, Integration, System Testing
--	---

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 06

- Test Design Techniques
- Combinatorial Testing
- Boundary Value Analysis
- Equivalence Class Partitioning

Lecture - 07

- Test Methodology
- Decision Table Testing

Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

BITS Pilani, Pilani Campus

3

Agenda



- Software configuration Management
 - Definitions
 - Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMi-Dev
- Baseline
- Branching
- Change Control

BITS Pilani, Pilani Campus

4

Software configuration Management



Definition

Configuration Management : Discipline applying technical and administrative direction and surveillance to:

- Identify and document the functional and physical characteristics of a configuration item
- Control changes to those characteristics,
- Record and report change processing and implementation status, and
- Verify compliance with specified requirements.

IEEE 828 [IEE 12b]

BITS Pilani, Pilani Campus

5

Software configuration Management



Essentially, SCM describes answers to the following questions :

- who can make changes?
- what changes were made?
- what were the impacts of a change?
- when did this change occur?
- why were these changes made?
- what version is currently in which environment?
- what branching approach are we using for this project ?

BITS Pilani, Pilani Campus

6

SCM - Benefits



Benefits :

- Reduce confusion, organize, and better manage software items
- Organize the required activities that ensure the integrity of the many software products
- Ensure traceable and current configuration of products
- Optimize the cost of development, maintenance, and after-sales support
- Facilitate the validation of the software with respect to its requirements
- Provide stable development, maintenance, testing, and production environments
- Improve quality and compliance to software engineering standards
- Reduce rework costs

BITS Pilani, Pilani Campus

7

SCM IEEE 828



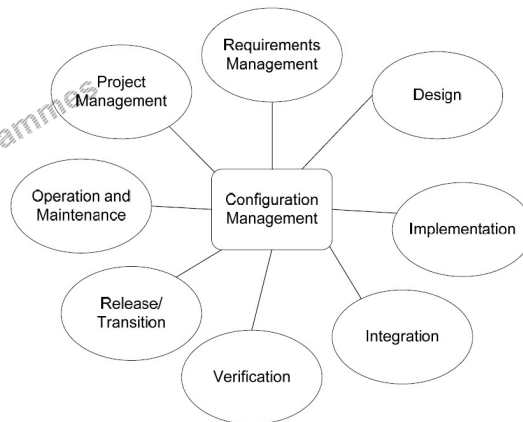
IEEE Standard for Configuration Management in Systems and Software Engineering

- IEEE 828 also supports the ISO 12207 standard
- As per IEEE 828, the purpose of CM is to
 - Identify and document the functional and physical characteristics of a product, component, output or of a service
 - Control any changes to these characteristics
 - Record and report each change and its implementation status
 - Support the audit of the products, results, services, or components to verify conformance to requirements.

BITS Pilani, Pilani Campus

8

SCM IEEE 828



Example life cycle processes that CM supports

BITS Pilani, Pilani Campus

9

SCM ISO 12207



According to the ISO 12207 standard, the purpose of CM is to manage and control system elements and configurations over the life cycle.

In ISO 12207 Standard – SCM is part of SQA practices

Outcome CM process -

- A software configuration management strategy is developed
- Items generated by the process or project are identified, defined and baselined
- Modifications and releases of the items are controlled
- Modifications and releases are made available to affected parties
- The status of the items and modifications are recorded and reported
- The completeness and consistency of the items is ensured
- The storage, handling and delivery of the items are controlled.

BITS Pilani, Pilani Campus

10

SCM ISO 12207



The six activities of the ISO 12207 CM process are [ISO 17]:

- Process Implementation - Plan CM - Shall describe –
 - Configuration management activities
 - Procedures and schedule for performing these activities
 - Who is responsible for performing these activities
 - The plan shall be documented and implemented
- Perform configuration identification – The documentation that establishes the baseline; the version references; and other identification details.
- Perform configuration change management
- Perform release control
- Perform configuration status accounting
- Perform configuration evaluation.

These Six CM activities of ISO 12207 are composed of 19 tasks.

BITS Pilani, Pilani Campus

11

SCM CMMI



The CMMI-DEV has a process area named “configuration management”

- Activities to be conducted as per CMMI-Dev
 - Identifying the configuration of selected work products that compose baselines at given points in time
 - Controlling changes to configuration items (CI)
 - Building or providing specifications to build work products from the CM system
 - Maintaining the integrity of baselines
 - Providing accurate status and current configuration data to developers, end users, and customers.

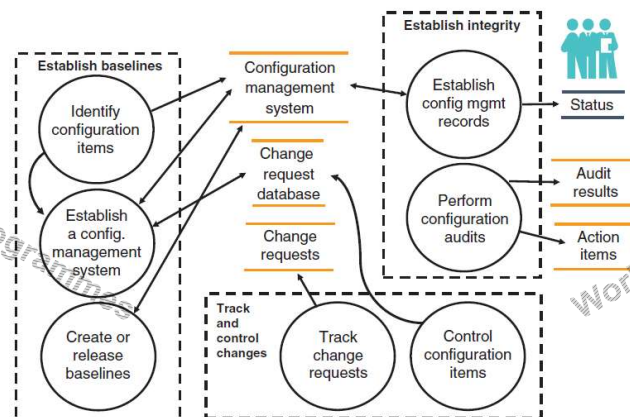
Level	Process	Key process area	Quality
5 Optimizing	Continuous process improvement	Operational performance management, causal analysis and resolution	Quality productivity
4 Quantitatively managed	Quantitative management	Operational process performance, organizational process management, product integration, verification	
3 Defined	Process management	Requirements development, technical solution, product integration, verification, organizational process framework, organizational process standards, organizational knowledge, integrated project management, team management, resource analysis and resolution	
2 Managed	Basic process management	Requirement management, project planning, project monitoring and control, supplier management, configuration management, organizational process framework, organizational process standards, organizational knowledge, integrated project management, team management, resource analysis and resolution	
1 Initial			Risk network

BITS Pilani, Pilani Campus

12

Specific Objectives (Goals) and Practices as per CMMI-Dev

- SG 1 Establish baselines
 - SP 1.1 Identify configuration items
 - SP 1.2 Establish a configuration management system
 - SP 1.3 Create or release baselines
- SG 2 Track and control changes
 - SP 2.1 Track change requests
 - SP 2.2 Control configuration items
- SG 3 Establish integrity
 - SP 3.1 Establish configuration management records
 - SP 3.2 Perform configuration audits



CM activities according to CMMI

SCM - Plan



IEEE 828 defines six SCM information categories to be placed in the SCM plan –

1. Introduction to the plan (the purpose, scope, and terminology)
2. SCM management (organization, responsibilities, authorities, applicable policies, guidelines, and procedures)
3. SCM activities (configuration identification, configuration control, and other activities)
4. SCM schedule (identification of the SCM activities in the project schedule)
5. SCM resources (tools, servers, and human resources)
6. SCM maintenance and updates.

BITS Pilani, Pilani Campus

15

SCM - Plan Identification of CI



- Each CI should be assigned a unique identifier.
- Identifier is composed of a fixed part and a variable part.
 - Fixed part is Name
 - Variable part (Revision/Version/BuildNo) is evolution
 - Revision – Major release is made. New feature, new functionality, etc
 - Version – Minor bug fixes, etc
 - Build No – Rolling Number every time a build is generated

BITS Pilani, Pilani Campus

16

SCM – Plan Identification of CI



Delivery - Release of a system or component to its customer or intended user.

Release - A delivered version of an application that may include all or part of an application.

Collection of new and/or changed configuration items that are tested and introduced into the live environment together.

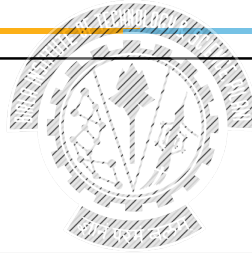
Version - An initial release or re-release of a computer software configuration item, associated with a complete compilation or recompilation of the computer software configuration item.

An initial release or complete re-release of a document, as opposed to a revision resulting from issuing change pages to a previous release.

ISO 24765 [ISO 17a]

BITS Pilani, Pilani Campus

17



Baselines



- State of a Configuration Item at the end of a stage or beginning of a new stage.
- This baseline is used for the next stage or for the entire project.
- It is reviewed, finalized, fixed and approved.
- Any changes to the baselines represent current configuration.
- Commonly referred baselines -
 - Specification
 - Design
 - Construction
 - Integration
 - Validation
 - Delivery

BITS Pilani, Pilani Campus

18

Baselines



Documents that can be baselined

Design documents

- Software development plan (SDP)
- Software requirements document (SRD)
- Interface design specifications
- Preliminary design document (PDD)
- Critical design document (CDD)
- Database description
- Software test plan (STP)
- Software test procedure (STPR)
- Software test report (STR)
- Software user manuals
- Software maintenance manuals
- Software installation plan (SIP)

Software code

- Source code
- Object code
- Prototype software

Data files

- Test cases and test scripts
- Parameters, codes, etc.

Software development tools

(the versions applied in the development and maintenance stages)

- Compilers and debuggers
- Application generators
- CASE tools

BITS Pilani, Pilani Campus

19

Branching

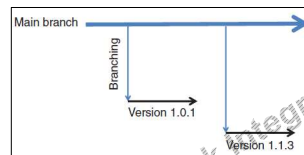


Branching is needed because

- Multiple people are working on project – Development team, Test Team
- Project is evolving continuously

A simple Branching Strategy

- The main branch;
- The versions branch.
- Development happens on the main Branch.
- Anytime a version is delivered to customer, a version branch is created



This strategy is appropriate when:

- A very small team is working on the project
- Corrections to production software require a fusion to the main branch.
- New version shipped from the main branch includes all previous changes from this branch.

BITS Pilani, Pilani Campus

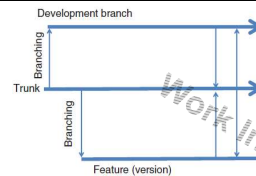
20

Branching



A Typical Branching Strategy

- The main branch;
 - The development branch;
 - The production branch
- No one works directly in Main Branch
 - Main branch is used only for integration from Development Branch.
 - Stable changes from Development branch goes to Main Branch
 - Stable version of Development branch is forked to Feature (Version) Branch
 - Bug fixes happen in Feature Branch.
 - These fixes are merged back into Development Branch and Main Branch.



BITS Pilani, Pilani Campus

21

Configuration/Change control



Configuration control or change control.

Definition -

An element of configuration management consisting of the evaluation, coordination, approval or disapproval, and implementation of changes to configuration items after formal establishment of their configuration identification. CMMI [SEI 10a]

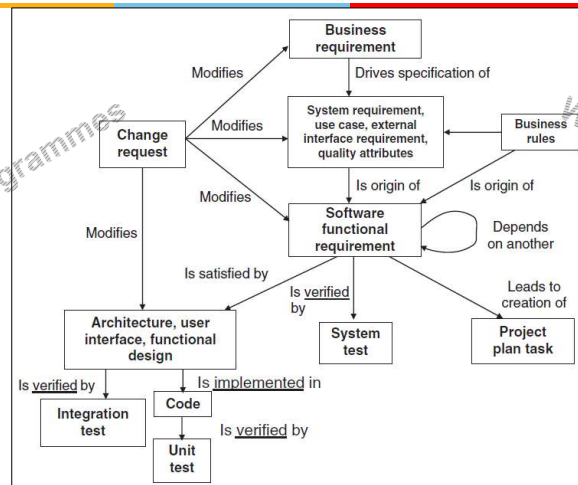
Changes need to be

- Evaluated
- Co-ordinated
- Approved
- Implemented On
 - Configuration Items with established Configuration Identification.

BITS Pilani, Pilani Campus

22

Configuration/Change control



Impact of a change request on a software

BITS Pilani, Pilani Campus

23

Configuration/Change control

Change source

Origin of Changes. Changes can come from -

- Problem reports (PR), or Bug report (TR), from
 - Development team
 - Maintenance team or
 - Customer
- Modification/evolution requests (MR) due to
 - An environment change or
 - The need for new features
- Requests from maintenance and infrastructure to
 - Improve the maintainability of the software.

BITS Pilani, Pilani Campus

24

Configuration/Change control Phases of change



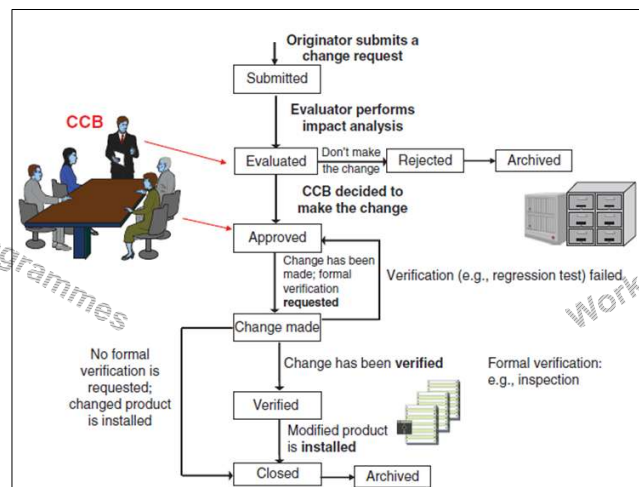
Sequence of phases of change can be as follows

- Customer (internal or external) or a team member submits a request;
- Maintainer evaluates the priority, impact and cost
- Change Control Committee (CCC) or Configuration Control Board (CCB) reviews and approves the change
- Change is made to the software and associated documentation
- Change is tested and approved by the end-user
- Change is moved to production or installed in a system and verified afterward
- Change request or ticket is closed and then later archived.

BITS Pilani, Pilani Campus

25

Configuration/Change control Phases of change



BITS Pilani, Pilani Campus

26

Configuration/Change control Change control Board



Configuration Control Board

A group of people responsible for evaluating and approving or disapproving proposed changes to configuration items, and for ensuring implementation of approved changes.

ISO 24765 [ISO 17a].

Change control board (CCB) or a change management office

- Authority that can accept or reject a proposed change to a software.
- Nature of Board depends on Size and nature of the project.
 - For small projects, even a project manager can act as CCB
- SCM representative, SQA representative can be part of CCB

BITS Pilani, Pilani Campus

27

Configuration/Change control Change control Board



The main tasks of the CCB are:

- Take a go/no-go decision
- Assign a priority
- Reject and assign the change to a future version
- Answer request issuers
- Issue access rights to libraries where changes are to be made.
- Make the decision (change order) public so everybody knows

The change request includes the following

- Details of changes to be made.
- Identification of the affected software
- List of items in a baseline to modify
- SQA tasks to ensure quality checks

BITS Pilani, Pilani Campus

28

Configuration/Change control Change Management Policy

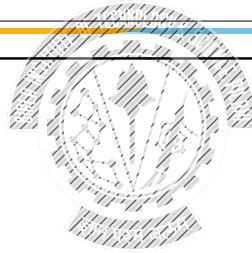


The elements of a change management policy are:

- The original text describing a change request will not change or will not be deleted from the repository
- Each changed requirement will be traceable to an approved change request
- All changes to requirements should follow the SCM process. If a change request is not documented, it will not be considered
- A CCB is created for each project
- The contents of the change request library must be available to all project team members;
- No design or development/maintenance work can be initiated on unapproved changes, with the exception of exploration or feasibility studies required by the CCB.

BITS Pilani, Pilani Campus

29



Work Integrated Learning Programmes




SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -12

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap



Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes	Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – - Unit, Integration, System Testing
--	--

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 06

- Test Design Techniques
- Combinatorial Testing
- Boundary Value Analysis
- Equivalence Class Partitioning

Lecture - 07

- Test Methodology
- Decision Table Testing

Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

Lectures 11

- Software configuration Management
 - Definitions, Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMI-Dev
- Baseline
- Branching
- Change Control

BITS Pilani, Pilani Campus

3

Agenda



- Introduction to Test Process Improvement
- Quality/Test Metrics

BITS Pilani, Pilani Campus

4

Test process Improvement - What



- As name suggests - Improvement of the test process from various standpoints.
- An iterative process.
- Analyze the existing process, find issues, find corrective actions, implement.
- Can be a continuous or non-continuous

BITS Pilani, Pilani Campus

5

Test process Improvement - Why



Why to Conduct Test Process Improvement?

- **Testing improvements** - When a test process is improved, naturally we see an improvement in the testing phases and the way things are executed.
- **Good quality software** - More Exhaustive Testing. This will help expose more bugs which will ultimately improve the quality of the application.
- **Align testing with other phases** - Helps align with changes in other phase like design, development, CI pipelines that get optimized with time.
- **Provide effectiveness in test activities** - Helps streamlining the testing activities and improving each of them individually.
- **Improved costs** - Improved processes are more effective in getting things done and hence require much fewer resources than before. E.g - Introduction to test automation.
- **Boosted business growth** - Help deliver a high-quality application, with minimum costs and in the shortest possible time.

BITS Pilani, Pilani Campus

6

Test process Improvement – Objectives



Objectives of Test Process Improvement

- To enhance testing practices, efficiency, and overall software quality.
- Adapt to changing requirements.
- Deliver high-quality software.
- Increases test coverage and test efficiency to identify more scenarios and to optimize resource utilization.
- Deep understanding of the current testing processes, methodologies, and practices. Expands test automation coverage to efficiently execute repetitive test cases.
- Highlights the areas in test cases that require changes. Enhance the test case design.
- Traceability between requirements, test cases, and defects. Enhanced visibility in the testing process and easier impact analysis.
- Mechanisms for continuous feedback, enabling timely identification and resolution of issues.
- Embrace Agile and DevOps methodologies to align testing activities with development cycles, fostering collaboration and rapid delivery.

BITS Pilani, Pilani Campus

7

Test process Improvement – When



When to Perform Test Process Improvement?

- **Significant rise in bugs** - A rise in bugs indicates the weak areas of our application have increased and our testing methods are not proving to be effective.
- **Increase in complexity (test management)** - People's skills needs. Testing has become more complex and so is the need for Test resources.
- **Increase in involved resources** – Increase in the size of test Team. Or when size of the Test team has not increased as per size of the project.
- **Increase in time of testing** – When the timelines of testing phase is large. It will cause delay to project timelines.
- **Increase in testing costs** – Includes cost of resources, timelines – delayed deliveries, tools, etc

BITS Pilani, Pilani Campus

8

Test process Improvement - When



When to Perform Test Process Improvement?

- **Newer methods have arrived** – When new technologies are available in the market that can help Testing. Initially cost can be high in purchasing, integrating, adapting, etc, but the return on investment in the long run can be high.
- **No retrospect for a long time** - When test process improvement has not been done for a long time. It is always good to take a break and spend time in Analyzing the existing process at frequent intervals and make changes accordingly.

BITS Pilani, Pilani Campus

9

Test process Improvement - How



How to Implement Test Process Improvement?

- **Diagnose the test improvement** - First diagnose the situation. Focus on why test improvement is being done. Find the issues and report the findings. Give details (including historic data) on problem areas.
- **Initiate the process/planning phase** - Plan the actions. Document objectives such as budget control, goals, risks in implementation, steps to be followed, and coverage of process improvement.
- **Acting on the plan** - Actual implementation of test process improvement as per plan. Constant monitoring and tracking is required so as to adhere to schedule.
- **Verify, Report, and Learn** – Verify results with planned expectations. Report results including metrics.
 - Lastly, retrospect on the test improvement process from phases 1 to 4 and document learnings throughout the process.

BITS Pilani, Pilani Campus

10

Test process Improvement - Benefits



Benefits of Test Process Improvement

- Identifies the weaknesses in the testing process, thus enhancing the effectiveness and efficiency of overall testing.
- Enhance the test coverage and improve test case design, facilitating early detection of defects.
- Promotes improved collaboration between development and testing teams
- Shared understanding of quality goals, leading to smoother workflows and accelerated feedback loops.
- Streamlines testing workflows and incorporates efficient practices.
- Reduced testing cycles and contributing to faster delivery of high-quality software.
- Higher test automation coverage and effectiveness of automated tests.
- Meaningful metrics and reporting mechanisms to gain actionable insights.
- Align testing practices with Agile and DevOps methodologies, ensuring that testing keeps pace with rapid development cycles and supports continuous integration and delivery.

BITS Pilani, Pilani Campus

11

Test process Improvement - Measure



How to Measure Testing Process Improvement Impact?

- **Define Clear Objectives**
- **Establish Baseline Metrics:** Before implementing improvements, establish baseline metrics to understand the current state of the testing process.
- **Implement Changes Gradually:** Helps in more controlled assessment of the impact of each change on the overall testing process.
- **Monitor Key Performance Indicators (KPIs):** Identify key performance indicators that directly align with the objectives of the testing process improvement.
- **Compare Metrics Before and After Implementation**
- **Calculate Return on Investment (ROI):** Evaluate the return on investment by comparing the benefits gained from testing process improvements against the costs incurred in implementing those changes.
- **Surveys and Feedback:** Administer surveys or conduct feedback sessions with the testing team to gauge their perceptions of the impact of process improvements.

BITS Pilani, Pilani Campus

12

Test process Improvement - Model - TMMi



Similar to software development models (such as Agile and Waterfall).

Testing Maturity Model integration (TMMi) –

- TMMi is a “maturity model” - Focuses on the maturity levels of testing lifecycle Processes.
- Introduced in 2005.

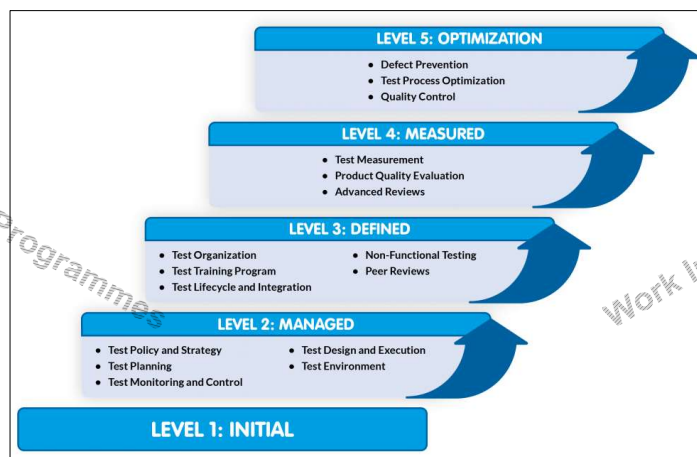
The benefits of using TMMi models are:

- Covers test-related activities extensively.
- Covers the best practices from the existing models.
- Covers the requirements for current trends and needs.
- Covers all aspects of test quality as described by industry experts.
- Provides a common standard for use.
- Can be applied to all phases of the software development lifecycle.

BITS Pilani, Pilani Campus

13

Test process Improvement - Models - TMMi



Src : <https://www.tmmi.org/tmmi-model/>

BITS Pilani, Pilani Campus

14

Test process Improvement - Model - TMMI - Levels



Testing Maturity Model integration (TMMi) – Levels

- Maturity Level 1 – Initial –
 - Not defined by the TMMi model.
 - Basic level.
 - This includes debugging and chaotic (unorganized) processes.
- Maturity Level 2 – Managed –
 - Basic test approach towards testing and manages the same approach.
 - Foundational structure in place that includes test planning, test policy, monitoring, and setting up of a test environment.
- Maturity Level 3 – Defined –
 - Standardizes the testing process across the organization
 - Each project to follow the same process.
 - Testing is integrated very early into the development making it part of the development cycle.
 - Teams are required to be trained in testing and each member has a specific job for testing
 - Test teams are more organized.
 - Non-functional testing be planned and executed

BITS Pilani, Pilani Campus

15

Test process Improvement - Model - TMMI - Levels



Testing Maturity Model integration (TMMi) – Levels

- Maturity Level 4 – Measured
 - Outcomes and parameter values are applied to all the projects to ensure a bug-free application
 - The review practices are more advanced and thorough in nature.
- Maturity Level 5 – Optimization
 - Series of methods set up for optimization of the processes followed in testing
 - Continuous optimization leads to a bug-free application.

BITS Pilani, Pilani Campus

16

Test process Improvement - Model - TPI-Next



Test Process Improvement (TPI) Next

- The Test Process Improvement Next model, also called TPI Next, is the next generation of its predecessor TPI (Test Process Improvement) model.
- The TPI Next model describes four maturity levels:
- Maturity Level 1 – Initial
 - There is no fixed process.
 - The organization does not follow any rules and the process is chaotic.
 - Organization is already at the maturity level 1 with whatever approach they follow.
- Maturity Level 2 – Controlled
 - Streamline the process with minimum structure.
 - The structure is organized and helps in reducing bugs.
 - Controlled state of the process and in no way is optimized.
 - Lot of bugs will still exist if the organization is at maturity level 2.

BITS Pilani, Pilani Campus

17

Test process Improvement - Model - TPI-Next



- Maturity Level 3 – Efficient
 - Efficiency of the currently working process in testing.
 - Not spending extra time on the redundant processes that arise when the testing process has certain overlapping stages.
 - Streamline processes and combine each process for a unified testing cycle.
- Maturity Level 4 – Optimizing
 - Assign coordinators who evaluate the process continuously from time to time.
 - Optimize the process and improve it for the next project.
 - Includes evaluating the bugs faced, any delays, or any other type of hindrances faced during the run.

Criteria	TMMi	TPI-Next
Representation	Staged model	Continuous model
Test levels	All test levels (unit, integration, system, user-acceptance testing)	Focus on higher test levels (System and user- acceptance testing)
Supported test methodology	Test-method independent	Linked to Tmap (tmap.net)
Terminology	ISTQB based	TMap based
Base SPI model	Related to CMMi	None
Certification	Possible through formal assessment	None

BITS Pilani, Pilani Campus

18

Test process Improvement – Tools and Techniques



Essential Tools and Techniques for Test Process Improvement

- Test Management Tools
 - Used for orchestrating and managing the entire testing process.
 - Facilitate test case design, execution, and reporting, streamlining collaboration among testing teams.
 - Popular examples include Jira, TestRail, etc.
- Automation Testing Tools
 - For Automating repetitive test cases
 - Selenium - An open-source framework, supports web application automation with compatibility across multiple programming languages.
 - Testsigma - Cloud-based test automation platform that offers scriptless test automation
- Continuous Integration/Continuous Delivery Tools (CI/CD)
 - Automate integration and deployment processes, ensuring a continuous and efficient development pipeline.
 - DevOps is a part of the automation testing process.
 - Enables automatic testing and delivery of the software, all the while logging the defects as well.
 - Tools such as Jenkins and GitLab

BITS Pilani, Pilani Campus

19

Test process Improvement – Tools and Techniques



Essential Tools and Techniques for Test Process Improvement

- RCA Techniques (Root cause Analysis)
 - Help identify and address the root causes of defects, preventing their recurrence.
 - RCA techniques like the 5 Whys and Fishbone Diagram (Ishikawa).
 - 5 Whys involves asking "why" multiple times to delve deeper into the root cause
 - The Fishbone Diagram provides a visual representation categorizing potential causes, aiding in systematic analysis and problem-solving.
- Risk-Based Testing
 - Risk-based testing focuses on areas with the highest risk of failure, efficiently allocating resources in those areas.
 - Techniques such as Failure Mode and Effect Analysis (FMEA) and Risk Matrix systematically evaluate and prioritize potential failure modes and risks.
 - FMEA identifies potential failure modes and their impact, allowing for proactive risk mitigation.
 - The Risk Matrix assesses and prioritizes risks based on likelihood and impact, guiding testing efforts toward critical areas.

BITS Pilani, Pilani Campus

20

Test process Improvement – Best Practices

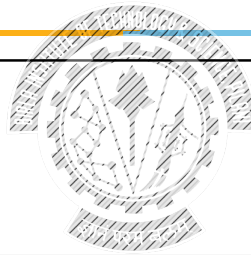


Best practices span the entire software development life cycle, from planning to execution, emphasizing collaboration, efficiency, and a commitment to ongoing learning.

- Establish **continuous feedback mechanisms** throughout the SDLC.
- Conduct **thorough root cause analyses for defects**. Utilize techniques like the 5 Whys or Fishbone Diagram to identify the underlying causes of issues.
- Embrace **iterative test planning** that aligns with Agile and DevOps methodologies.
- Aim for **comprehensive test coverage** by addressing various testing levels (unit, integration, system, etc.).
- Aim for comprehensive test coverage both **Functional and Non-Functional** (Performance, Security, Reliability, etc.).
- Prioritize **testing efforts based on risk assessment**. Identify high-risk areas and allocate resources accordingly.
- Implement **automated regression testing** to quickly validate existing functionalities after code changes.

BITS Pilani, Pilani Campus

21



Test process Improvement – Best Practices



Best Practices

- **Embrace CI/CD** practices to automate and streamline the integration and deployment processes.
- Incorporate **performance testing early** in the development life cycle.
- Establish **robust test data management** practices to ensure the availability of relevant and realistic test data.
- Foster a culture of **continuous learning** within the testing team. Encourage the adoption of new tools, technologies, and best practices.
- Encourage **collaboration** between development, testing, and other cross-functional teams.
- Implement **shift-left testing** practices by involving testing activities early in the development process.

BITS Pilani, Pilani Campus

22

Test process Improvement - Challenges



Challenges of Test Process Improvement

These are problems that can happen before and during improvement process of testing.

- Limited resources, including time, budget, and skilled personnel.
- Legacy systems may face challenges when integrating modern testing practices.
- Employees lack the necessary training, leading to skill gaps that hinder effective implementation of improved practices.
- Support from executive leadership
- Integrating test process improvements seamlessly into existing development workflows can be challenging.
- Organizations operating in regulated industries face challenges in aligning test processes with complex regulatory requirements.
- Transitioning to Agile and DevOps methodologies poses challenges for organizations accustomed to traditional development approaches.
- Applying improved processes across various projects and teams can be challenging.

BITS Pilani, Pilani Campus

23

S/W Quality Metrics - Definition



Quality metrics is defined as a category of SQA tools:

1. A quantitative measure of the degree to which an item possesses a given quality attribute.
2. A function whose inputs are software data and whose output is a single numerical value that can be interpreted as the degree to which the software possesses a given quality attribute.

BITS Pilani, Pilani Campus

24

S/W Quality Metrics- Objective



Main objectives of software quality metrics -

1. To facilitate management control as well as planning and execution of the appropriate managerial interventions. Based on -
 - Deviations of actual functional (quality) performance from planned performance
 - Deviations of actual timetable and budget performance from planned performance.
2. To identify situations that require or enable development or maintenance process improvement in the form of preventive or corrective actions introduced throughout the organization. Based on -
 - Accumulation of metrics information regarding the performance of teams, units, etc.

BITS Pilani, Pilani Campus

25

S/W Quality Metrics Baseline and comparison



Quality metrics can be compared to -

1. Defined software quality standards
2. Quality targets set for organizations or individuals
3. Previous year's quality achievements
4. Previous project's quality achievements
5. Average quality levels achieved by other teams applying the same development tools in similar development environments
6. Average quality achievements of the organization
7. Industry practices for meeting quality requirements

BITS Pilani, Pilani Campus

26

S/W Quality Metrics Classification



Quality metrics can be classified into -

Classification based on phase of life cycle of software system -

1. Process metrics, related to the software development process
2. Product metrics, related to software maintenance

Measured in terms of

1. KLOC – Thousand lines of code
2. Function Point – Functionality

BITS Pilani, Pilani Campus

27

S/W Quality Metrics Process Metrics



Process metrics can be categorized as -

1. Software process quality metrics
 - Error density metrics
 - Number of Errors per lines of code or per Function Point
 - Errors can be weighted based on severity of error.
 - Error severity metrics.
 - Average severity of Errors
2. Software process timetable metrics
 - Average delay in completion of Milestones
3. Error removal effectiveness metrics
 - Errors Removal Effectiveness
 - Weighted Errors Removal Effectiveness
4. Software process productivity metrics
 - Development Productivity Hours/KLOC
 - Function point Development Productivity Hours/Function Point
 - Code Reuse - Reused lines of code
 - Documentation Reuse – Reused documentation

BITS Pilani, Pilani Campus

28

S/W Quality Metrics Product Metrics



Product metrics - refer to the software system's operational phase

Services provided can be

1. HelpDesk Services - For usage and application of S/W. Depends on S/W intuitiveness, friendliness.
 - HD calls density metrics – Number of calls in a timeframe.
 - Metrics of the severity of the HD issues raised – Average severity of HD Calls.
 - HD success metrics – the level of success in responding to these calls – Meeting SLA in resolution.
2. Corrective Maintenance Services – When failure occurs at customer site.
 - Software system failures density metrics – Failures per KLOC or FP
 - Software system failures severity metrics – Average severity of Failure
 - Failures of maintenance services metrics
 - Unable to complete the failure correction on time
 - Correction fails to fix failure - Maintenance Repeated repair Failure
 - Software system availability metrics
 - Full availability – All software system functions perform properly
 - Vital availability – No vital functions fail (but non-vital functions may fail)
 - Total unavailability – All software system functions fail.

BITS Pilani, Pilani Campus

29

S/W Quality Metrics Implementation



Implementation requires

- Definition of software quality metrics – relevant and adequate for teams, departments, etc.
 - Definition of attributes to be measured
 - Definition of the metrics that measure the required attributes
 - Comparative target values based on standards, previous year's performance, etc.
 - Reporting method, including reporting process and frequency of reporting
 - Metrics data collection method.
- Regular application by unit, etc.
 - Assigning responsibility for reporting and metrics data collection.
 - Instruction to the team regarding the new metrics.
 - Follow-up :
 - Updates and changes of metrics definitions.

BITS Pilani, Pilani Campus

30

S/W Quality Metrics Implementation - 2



Implementation requires

- Statistical analysis of collected metrics data
 - Descriptive statistics
 - Analytical statistics.
- Subsequent actions:
 - Changes in the organization and methods (Process) used in S/W development
 - Change in metrics and metrics data collection
 - Application of data and data analysis to planning corrective actions for all the relevant units.

BITS Pilani, Pilani Campus

31

S/W Quality Metrics Limitations



Application of quality metrics is strewn with obstacles. These can be grouped as follows:

- Budget constraints in allocating the necessary resources (manpower, funds, etc.).
- Human factors, especially opposition of employees to evaluation of their activities.
- Uncertainty regarding the data's validity, rooted in partial and biased reporting.
- Parameters KLOC, Function point are subjective
 - Programming style
 - Volume of documentation comments included in the code
 - Software complexity
 - Percentage of reused code
 - Professionalism and thoroughness of design review and software testing teams.
 - Reporting style of the review and testing results

BITS Pilani, Pilani Campus

32




SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -13

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap



Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks	Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance, Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – - Unit, Integration, System Testing Lecture - 06 - Test Design Techniques - Combinatorial Testing - Boundary Value Analysis - Equivalence Class Partitioning Lecture - 07 - Test Methodology - Decision Table Testing
--	---

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

Lectures 11

- Software configuration Management
 - Definitions, Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMi-Dev
- Baseline
- Branching
- Change Control

Lectures 12

- Introduction to Test Process Improvement
- Quality/Test Metrics

BITS Pilani, Pilani Campus

3

Agenda



- Test Organization
- Roles in Test

BITS Pilani, Pilani Campus

4

Test Organisation



- Software Test Team can be different from Software Quality Assurance Team
- Must have domain knowledge to test the software under development
- Proper Structure - Hierarchy and roles and responsibilities should be clearly defined
 - Test Director
 - Test Manager
 - Test Lead
 - SDET (Software Development Engineer in Test)
 - STE (Software Test Engineer)

BITS Pilani, Pilani Campus

5

Test Organisation



- Interaction with Development team (Technical)
 - Closely working with development team.
 - Both White Box Testing and Black Testing
 - Knows the development process.
 - Provide input to Dev team to provide Test Hooks
 - Take help from Dev for Test Framework development
 - Working independently of development team.
 - Mostly Black Box Testing
 - Have independent view of the product.
 - Not influenced by Development Team
- Works independently and without influence from Dev, PM team
 - They are the guardians of Quality

BITS Pilani, Pilani Campus

6

Test Organisation

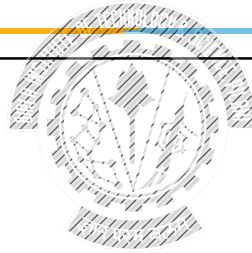


Interaction with other Teams

- With Business Teams.
 - Keep them updated of Project Status
 - Answer Queries of Customers
- With Release Management Teams
 - Helps them planning the releases for hardware, software.
 - Keep them updated of quality of product
 - Helps them in setting up the release process and environment
- With Project Management Team
 - Provide inputs for Hiring Test resources
 - Provide inputs on H/W and S/W procurement for Testing needs
 - Finalizing templates for testing documents, test reports and other procedures and get them reviewed by Project Management team.

BITS Pilani, Pilani Campus

7



Roles in Test Organisation



- **Software Test Manager**
 - Knows testing methodologies in detail
 - Have knowledge of Business domain
 - Have knowledge of client's requirement
 - Good at project planning, task and people coordination
 - Have good knowledge of testing tools. Selection of right test tools.
 - Responsible for all interdepartmental meetings.
 - Interaction with the customers whenever required.
 - Recruiting software testing staff.
 - Schedule testing activities
 - Create budget for testing and prepare test effort estimations.
 - Carry out continuous test process improvement with the help of metrics.
 - Check the quality of requirements, how well they are defined.
 - Trace test procedures with the help of test traceability matrix.
 - Status Reporting, Test Metrics to Management
 - Final call on Shipping the product

BITS Pilani, Pilani Campus

8

Roles in Test Organisation



- **Software Test Lead**

- Technical experience which includes
 - Programming,
 - Handling Database Technologies and
 - Various operating systems
- Hands on in writing Code, Scripts, Test cases
- Good experience in Debugging, finding root cause of the Defect
- Team Management and Coordination
- Identify the scope of the testing for his components/Area
- Review Test Plans, Test Cases, Test Scripts, etc
- Communication channel between Test Team and Test Management.

BITS Pilani, Pilani Campus

9

Roles in Test Organisation



- **Software Developer in Test**

- Have Technical expertise in Programming, Databases, etc
- Write Test Plans, Test Cases, Test Scripts for assigned components/Area
- Debugging, finding root cause of the Defect
- Developing inhouse proprietary Test Framework
- Have knowledge of Testing tools
- Use Testing tools to automate Test cases.

- **Software Test Engineer**

- Design and Execute Manual Test cases
- Design and Execute Automated Test cases
- Prepare Test Report of Test cases executed
- Log Bugs in Bug Management Tool
- Debug and find root cause of Bugs reported from other teams.

BITS Pilani, Pilani Campus

10

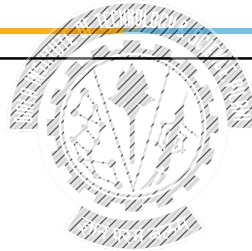
Career Roles in QA/Testing



- The choice between software testing vs quality assurance should be based on the unique requirements of each project or company.
- Quality assurance vs testing are two separate yet interdependent processes.
- Here are what you need to think about before committing:
 - Quality assurance encompasses a wider range of tasks, quality planning, process enhancement, & audits.
 - Software testing generally focuses on discovering defects & issues in the software through conducting tests.
- Roles in Testing
 - Software Tester/Test Engineer
 - SDET (Software Developer in Test)
 - Test Lead
 - Test Manager
 - Test Architect
 - Test Director
 - Delivery Head
- Roles in Testing
 - QA Engineer
 - QA Analyst
 - Auditor
 - Compliance Officer

BITS Pilani, Pilani Campus

11



Work Integrated Learning Programmes



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -14

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap

Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks	Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance, Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – - Unit, Integration, System Testing Lecture - 06 - Test Design Techniques - Combinatorial Testing - Boundary Value Analysis - Equivalence Class Partitioning Lecture - 07 - Test Methodology - Decision Table Testing
--	---

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

Lectures 11

- Software configuration Management
 - Definitions, Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMi-Dev
- Baseline
- Branching
- Change Control

Lectures 12

- Introduction to Test Process Improvement
- Quality/Test Metrics

Lectures 13

- Test Organization
- Roles in Test

BITS Pilani, Pilani Campus

3

Agenda



- Agile Test Planning and Execution
- Six Sigma in S/W Testing
- Continuous Testing in DevOps

BITS Pilani, Pilani Campus

4

Agile Development - Overview



Agile development is a project management methodology that values individuals and interactions over processes and tools. The Agile Manifesto, outlines the four main values and twelve principles of Agile development.

The four Agile values :

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile development focuses on

- Creating working software quickly
- Collaborating with customers frequently
- Adapt to changes easily.

Beneficial for projects that are complex or have uncertain requirements.

BITS Pilani, Pilani Campus

5

Agile Variations



There are many variations of Agile Development. 3 major ones are -

1. Extreme Programming

- Customer-driven, user stories driving both a release plan and system testing.
- Pair of developers work closely together, hence continuous code walk-through
- No preliminary design phase, this is a bottom-up process.

2. Test Driven Development

- Code is based on how it will be tested.
- Tests are designed and executed and allowed to fail.
- Then code is written, and tests are made to pass.
- Bottom-up approach. Prohibits a single, high-level design step.

3. Scrum

- Sprints of 2 to 4 weeks.
- Daily StandUp meeting. (Status Meeting)
- Deliverable code at the end of sprint.
- Unit Testing and integration/System Testing.

BITS Pilani, Pilani Campus

6

Agile Development - Stages



Agile process can be broken down into three main stages:

1. Preparation

- Product owner creates a backlog of features, also known as the product backlog.
- Development team estimates how long each feature will take to build.

2. Sprint planning

- A sprint is a set period (usually two weeks).
- Meeting to decide features from the product backlog to be worked on during the sprint.
- Sprint Backlog - Task to be included in sprint – (includes coding tasks, testing tasks, documentation task). This information is then added to the sprint Backlog.

BITS Pilani, Pilani Campus

7

Agile Development - Stages



3. Sprint

- Team works on completing the tasks in the sprint backlog.
- New issues to address, added to the product backlog and prioritized
- Sprint Backlog to be completed in the Sprint
- If tasks not completed, carry them over to the next sprint.
- Sprint Review Meeting - Demo completed features to the product owner and stakeholders.
- Retrospective meeting - Discuss what went well during the sprint and what can be improved.
 - Create action plan for future sprints.
 - Feedback loop helps each sprint is more successful than the previous one.

BITS Pilani, Pilani Campus

8

Agile Development - Why - Importance



Why is Agile Development important?

- Ensure that development teams complete projects on time and within budget.
- Improve communication between the development team and the product owner.
- Reduce the risks associated with complex projects.
- Allows for teams to make changes quickly and easily without affecting the overall project timeline

BITS Pilani, Pilani Campus

9

Agile Development - Benefits



What are the benefits of Agile Development methodology?

- **Increased flexibility:** More flexible than other project management methodologies.
 - Development teams can make changes on the fly more easily.
- **Improved communication:** Improve communication between the development team and the product owner.
 - Greater focus on collaboration and feedback.
- **Reduced risks:** Reduced the risks associated with complex projects.
 - By breaking down complex projects into smaller sprints, project managers can dissect them and achieve shareholder demands.
- **Increased customer satisfaction:** Customer is involved in the development process and provides feedback at each stage of the project.

BITS Pilani, Pilani Campus

10

Agile Development - Why - Importance



What are the disadvantages of Agile Development methodology?

- **Limited control:** Since agile development is more flexible, it can be difficult for a project owner to exert control over the project.
- **Lack of documentation:** Less documentation than other project management methodologies.
- **High level of collaboration:** Can be a problem for remote teams that are not used to working together.
- **Complex projects can be lengthy:** More time than other project management methodologies due to frequent meetings.

BITS Pilani, Pilani Campus

11

Agile Testing - Overview



Agile testing is a software testing practice that follows the Agile software development methodology.

- Agile testing focuses on ensuring quality throughout the Agile software development process.
- Continuous integration and continuous delivery are two important aspects of agile testing.
 - Continuous integration - Developers integrate their code changes into a shared mainline several times a day.
 - Continuous delivery - Every change that passes all tests is automatically released into production.

BITS Pilani, Pilani Campus

12

Agile Testing - Principles



The main principles of agile testing are:

- **Early and continuous testing:** Testers should start early and test the software continuously throughout the development cycle.
- **Whole team approach:** All team members are responsible for ensuring the quality of the product. This includes developers, testers, business analysts, and product owners.
- **Frequent deliveries:** Deliver working software frequently, typically every two weeks.
- **Close collaboration:** Close collaboration between all team members. Everyone is on the same page and that there are no surprises.

BITS Pilani, Pilani Campus

13

Agile Testing - Principles - Cont



The main principles of agile testing are:

- **Customer involvement:** Customers are involved throughout the development process and provide feedback at each iteration.
- **Working software:** Agile teams focus on quality software management during each iteration. Code produced in sprint is tested within the sprint.
- **Flexible approach:** Teams can change the requirements at any time during the development process.

BITS Pilani, Pilani Campus

14

Agile Testing - Skills



The most important skills for agile testers include:

- **Strong communication skills:** Testers must be able to communicate effectively with all team members.
 - They need to be able to understand the customer's needs and requirements.
- **Strong technical skills:** Testers need to have strong technical skills.
 - They should be able to understand the code and the technology stack.
- **Strong problem-solving skills:** Able to identify and solve problems quickly.
 - They should also be able to think creatively and come up with new solutions as needed.
- **Strong team player:** Testers need to be able to work well in a team environment.
 - They should be flexible and adaptable, and able to work well under pressure.

BITS Pilani, Pilani Campus

15

Agile Testing - Challenges



Challenges during agile testing –

- Constant changes require testers need to be adaptable, and they need to be able to change their approach on the fly.
- Testers need to be able to communicate effectively with all team members, as well as with the customer.
- Agile development environments often have tight deadlines. Testers need to be able to work well under pressure and they need to be able to deliver quality results quickly.
- In Waterfall testing, requirements are finalized at the beginning of the project, and do not change. Testers can design their test cases upfront, and they can execute them when the developers finish the code. In Agile projects, however, the requirements are constantly changing. Testers need to be much more adaptable. They need to be able to design their test cases on the fly and they need to be able to execute them quickly.

BITS Pilani, Pilani Campus

16

Agile Testing - Activites



The main testing activities in agile are:

- **Requirement analysis:** Testers work with business analysts and product owners to understand the requirements of the new feature.
- **Test design:** Testers design the test cases for the new feature.
- **Test execution:** Testers execute the test cases and report any bugs.
- **Defect management:** Testers work with developers to fix the defects.
- **Release management:** Testers help to plan and execute the release of the new feature.

BITS Pilani, Pilani Campus

17

Agile Testing - Best Practices



Some best practices for agile testing include:

- **Automate as much as possible** - Helps speed up the testing process and enables developers to find more defects.
- **Use an automated tool to track defects** - Helps development teams source and mitigate bugs quickly and efficiently.
- **Use continuous integration and continuous delivery** - Helps to ensure that changes happen frequently and that they are always tested before finalization.
- **Communicate early and often** - Testers need to be able to effectively communicate with all team members, as well as with the customer.

BITS Pilani, Pilani Campus

18

Agile Testing - Testing Methodologies



Agile Testing Methodology

Tests come first in Agile development.

Acceptance criteria is created at the time of creation of a user story.

- Test-Driven Development (TDD) - This type of development begins by discussing what needs to be Tested and then creating a user story. So, unit and component tests are written. Then the user story. Finally, the code.
- Acceptance Test-Driven Development (ATDD) - Similar to TDD. Starts with customer input on functionality. User acceptance test (UAT) are written and then the code.
- Behavior-Driven Development (BDD) - Stems from TDD and ATDD. The purpose of development needs to be tied to a business outcome. User story needs to answer why, in business terms, this feature is being developed. Tests are included in user stories as scenarios or specifications.

BITS Pilani, Pilani Campus

19

Agile Testing - Testing Methodologies



- Exploratory Testing - Testers follow their intuition, rather than a predefined path.
 - It's typically manual. Testers record what they're doing and save it as a test. They then figure out what exactly it is that they're testing as they go. Exploratory testing is typically used to find hidden risks within a product. These would be bugs that are missed in functional tests done in test-driven development.
- Session-Based Testing - Session-based testing has some similarities to exploratory testing. But there's a little more structure in session-based testing. Instead of testers figuring out what they're testing as you go, they start with a mission in mind. Session-based testing is also used to find hidden bugs within a project.

BITS Pilani, Pilani Campus

20

Six Sigma in Software Testing - Definition, Objectives



- Six Sigma is the process of producing high and improved quality output.
- Phases – identification and elimination.
- The cause of defects is identified and appropriate elimination is done which reduces variation in whole processes.
- A six sigma method is one in which 99.99966% of all the products to be produced have the same features and are free from defects
- Six Sigma means a measure of quality that strives for near perfection.
- Six Sigma is a disciplined, data-driven approach and methodology for eliminating defects.

- Assure High Software quality
- Emphasize results of “successful” inspections and tests, namely defects and failures
- Assess cost-effectiveness quantitatively
- Identify best practices
- Continuously improve.

BITS Pilani, Pilani Campus

21

Six Sigma in Software Testing - Best Practices



Best Principles and Practices

- Key engineering principles
 - Pragmatic Iterative and Incremental Process,
 - Client Orientation,
 - High-Yield and Coverage-Driven Design Inspection
 - High-Yield Testing Process
 - Well-Integrated Team of highly qualified experts
 - Monitoring+Integrated Management etc.
- Setting Management objectives (in Quality, Productivity and Resources)
- Integrate Measurement Process (define and measure Process and Product Metrics)
- Predict, Assess and Monitor Cost of Quality
- Apply Constant Process Improvement (CPI) based on Optimization techniques of: Product quality, Development cost, Time to market, and Development Resources (personnel and facility).

BITS Pilani, Pilani Campus

22

Six Sigma in Software Testing - Testing Methodologies

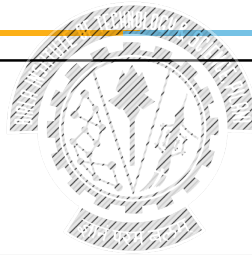


Six Sigma (6σ) Phases - To assure stable i.e. controllable and predictable software testing process

- **Define** the Customer, their Critical to Quality (CTQ) issues, and the Core Business Process involved.
 - Define who customers are, what their requirements are for products and services, and what their expectations are.
 - Define project boundaries - stop and start of the process.
 - Define the process to be improved by mapping the process flow.
- **Measure** the performance of the Core Business Process involved.
 - Develop a data collection plan for the process.
 - Collect data from many sources to determine types of defects and metrics.
 - Compare to customer survey results to determine shortfall.

BITS Pilani, Pilani Campus

23



Six Sigma in Software Testing - Testing Methodologies



- **Analyze** the data collected and process map to determine root causes of defects and opportunities for improvement.
 - Identify gaps between current performance and goal performance.
 - Prioritize opportunities to improve.
 - Identify sources of variation.
- **Improve** the target process by designing creative solutions to fix and prevent problems.
 - Create innovative solutions using technology and discipline.
 - Develop and deploy implementation plan.
- **Control** the improvements to keep the process on the new course.
 - Prevent reverting back to the "old way".
 - Require the development, documentation and implementation of an ongoing monitoring plan.
 - Institutionalize the improvements through the modification of systems and structures (staffing, training, incentives)

BITS Pilani, Pilani Campus

24

Continuous Testing in DevOps



Difference between traditional Testing and DevOps Testing

- Traditional QA testing in a waterfall Model
 - Testing is a distinct phase following software development
 - Conducted by an SQA test team that is “independent” of the development team.
- DevOps – Development and Operations
 - Automated integration, build and delivery processes
 - Integrated test orchestration and automation tools and frameworks.

BITS Pilani, Pilani Campus

25

Continuous Testing in DevOps Best Practices



Basic Tenets of Best Practices in DevOps Testing

- **Shift Left** : Conduct each test as early in the pipeline as possible
- **Fail Early** : Arrange the tests so that the most likely problems are found in the earliest possible stage in the DevOps pipeline.
- **Fail Often**: Run tests frequently and with many different conditions
- **Test Fast**: Arrange tests to run in quick cycles.
- **Relevance**: Focus on the most important tests and results.

BITS Pilani, Pilani Campus

26

Continuous Testing in DevOps Best Practices



Best Practices in DevOps Testing

1. DevOps test culture – Quality is everyone's concern

2. Continuous test strategies - Small changes are tested incrementally by a cross-functional team over all stages of the continuous delivery pipeline. Agile is compatible with Dev-Ops

- **A/B testing** - Also known as Split Testing or Bucket Testing, compares two versions of an app to identify the better performer.
- **Green/Blue** deployments – Moving traffic to newer version (Green) while old one (Blue) is still in use.
- **Canary testing** - Testing a new software version or a new feature with real users in a live (production) environment. It is done by pushing some code changes live to a small group of end users who are usually unaware that they are receiving new code.
- **Chaos Monkey** - To test the resiliency and recoverability by randomly terminating instances in production.

BITS Pilani, Pilani Campus

27

Continuous Testing in DevOps Best Practices



3. End-to-end test integration

- Pre-Flight tests (include unit testing, static code analysis scanning, functional, regression and performance tests) prior to committing software changes
- Code commit stage – Validate pre-flight test on committed code
- Build stage – To ensure build integration is successful
- Code Test stage - Functional and performance tests
- Regression stage - A set of regression tests
- Integrate packages stage - To ensure the image packages are good
- System test and regression stage - To ensure system test and regression package assessment criteria are met.
- Delivery stage - All deployed software tested to ensure delivery test criterion.
- Infrastructure tests to verify the infrastructure variations for deployment.
- Automation tests.
- Monitoring and management tests

BITS Pilani, Pilani Campus

28

Continuous Testing in DevOps Best Practices



4. DevOps test infrastructures

- Tests are to be conducted in a production-equivalent test environment.
- Test infrastructure like
 - Bare metal servers
 - Virtual machines
 - Containers running in a private data center
 - Cloud or
 - Multi-Cloud environments.
- Infrastructure-as-a-Service (IaaS), Platform-as-a-Service (PaaS) or Software-as-a-Service (SaaS) environments.
- Variety of network, cluster, load-balancing, storage and database configurations.
- Disaster recovery and restoring service infrastructures.
- Elastic infrastructure for testing.
- Dynamic infrastructure configuration tools, cloud services, etc

BITS Pilani, Pilani Campus

29

Continuous Testing in DevOps Best Practices



5. DevOps test frameworks

- Test framework - Automated test setup, test execution, and test results.
- Includes tests, test tools, applications, test infrastructures, and test data.
- Select tests, control the execution of tests, monitor tests in progress and report test results, logs and verdict data

6. Test Analytics

- Test results analysis to keep up with accelerated tests.
- Analysis tools built into test frameworks, test results dashboards, test results analyzers.

7. Database DevOps Testing

- Replicate production level data volumes

8. Continuous Test Management

- Includes Test planning, test guidelines, test code version management, and version management of test results.

BITS Pilani, Pilani Campus

30



SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -15

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap

Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks	Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance, Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – - Unit, Integration, System Testing Lecture - 06 - Test Design Techniques - Combinatorial Testing - Boundary Value Analysis - Equivalence Class Partitioning Lecture - 07 - Test Methodology - Decision Table Testing
--	---

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 08

- Revision

Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

Lectures 11

- Software configuration Management
 - Definitions, Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMi-Dev
- Baseline
- Branching
- Change Control

Lectures 12

- Introduction to Test Process Improvement
- Quality/Test Metrics

Lectures 13

- Test Organization
- Roles in Test

Lectures 14

- Agile Test Planning and Execution
- Six Sigma in S/W Testing
- Continuous Testing in DevOps

BITS Pilani, Pilani Campus

3

Agenda



- Future of SQA
- Testing in Block Chain
- AI/ML in Software Testing

BITS Pilani, Pilani Campus

4

Future of SQA Challenges



The challenges SQA will face in the future –

- Growing complexity and size of software packages
 - Increasingly complex algorithms
 - Increasing number of source of inputs and outputs.
 - Demands for greater accuracy, more complete information and shorter reaction times.
- Growing integration and interface requirements
 - ERP (Enterprises Resource Planning) System for production planning and control, sales, inventory management and financial systems in one program.
 - CRM (Customer Relations Management) systems that deal with all customer related systems: purchase records, payments, complaints, services provided, consumer socio-economic characteristics, etc.
 - External interfaces, namely software–software, software–firmware and firmware–firmware interfaces.

BITS Pilani, Pilani Campus

5

Future of SQA – Challenges



- Shorter project schedules
 - Employment of larger development teams. Requires greater coordination and cooperation.
 - Much less time available for review and testing activities leading to quality issues.
- Growing intolerance of defective software products.
 - Increasingly dependency on software products, hence the intolerance.
 - Software for critical areas – Health, Financial, Airlines, etc
- Software products developed and used across regions.
 - Development, Product, Test Teams working at different locations.
 - Product developed for different locations – Language, TimeZone, Currency, etc

BITS Pilani, Pilani Campus

6

Future of SQA – Solution to Challenges

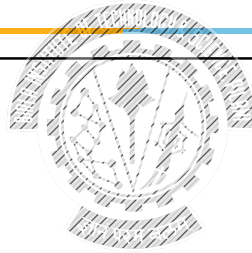


Ways to overcome challenges –

- Extended use of CASE (Computer Aided Software Engineering) tools. These tools
 - Provide updated accurate documentation to support development of S/W.
 - Support coordination of large teams by providing documentation of design and code.
 - Enable automation of segments of the development process.
 - Support maintenance by updating documentation and automating activities.
- Expanded use of professional standards
 - Standard terminology helps in communication and coordination between teams.
 - Facilitating mobility and absorption of software professionals between teams.
 - Simplify reuse of code.

BITS Pilani, Pilani Campus

7



Future of SQA – Solution to Challenges



Ways to overcome challenges –

- Extended use of automated testing
 - Effective and efficient tools for regression testing, test records and summaries.
 - Helps in comply with shorter schedules
- Expanded software reuse.
 - Reduced Testing effort. Hence shorter project schedules.
 - Software reuse increases standardization, resulting in smoother interface development and system integration.

BITS Pilani, Pilani Campus

8

Block Chain - Overview



What is Blockchain - Blockchain is a decentralized, distributed ledger technology that enables secure and transparent recording of transactions across a network of computers.

Here's how blockchain works:

- **Decentralization:** Each node (Machine/Server) in the network maintains a copy of the entire blockchain, creating redundancy and enhancing security.
- **Distributed Ledger:** Transactions are encrypted and stored in Blocks. Each block is linked to previous block, using cryptographic hash, thus creating a blockchain.
- **Consensus Mechanisms:** Blockchain networks use consensus mechanisms to validate and agree on the validity of transactions before they are added to the blockchain.
- **Cryptographic Security:** Blockchain uses cryptographic techniques such as digital signatures and hash functions to ensure the security and authenticity of transactions
- **Smart Contracts:** Smart contracts automate and enforce the execution of transactions. Automatically execute predefined actions when certain conditions are met. All participants can be immediately certain of the outcome, without any intermediary's involvement or time loss.

BITS Pilani, Pilani Campus

9

Block Chain Testing Applications



Blockchain has a wide range of applications across industries, including:

- **Cryptocurrencies:** Cryptocurrencies like Bitcoin, Ethereum, and others. These digital currencies utilize blockchain technology to facilitate secure and decentralized peer-to-peer transactions.
- **Supply Chain Management:** Blockchain can be used to track and trace the movement of goods and raw materials across the supply chain, improving transparency, accountability, and efficiency.
- **Financial Services:** Can be applied to financial services by enabling faster, cheaper, and more secure transactions, as well as facilitating services such as cross-border payments, remittances, and peer-to-peer lending.
- **Identity Management:** Blockchain-based identity solutions offer a secure and tamper-proof way to manage digital identities, reducing the risk of identity theft and fraud.

BITS Pilani, Pilani Campus

10

Block Chain Testing Applications



- **Healthcare:** Blockchain can be used to securely store and share medical records, enable interoperability between healthcare systems, and track the provenance (Origin) of pharmaceuticals and medical devices.
- **Real Estate:** Blockchain can streamline real estate transactions by providing a transparent and secure record of property ownership, title transfers, and land registry information.

Overall, blockchain technology has the potential to revolutionize various aspects of our economy and society by enhancing security, transparency, and efficiency in transactions and data management.

BITS Pilani, Pilani Campus

11

Block Chain Testing Testing



Testing blockchain systems should cover various aspects of functionality, security, performance, and scalability.

Types of testing on Block chain System:

- **Unit Testing:** To verify the correctness of individual components, such as smart contracts, cryptographic functions, and consensus algorithms.
- **Integration Testing:** To verify the interaction between different components of the blockchain system, such as the blockchain protocol, smart contracts, and user interfaces.
- **Functional Testing:** To validate the functional requirements of the blockchain system, such as transaction processing, asset transfers, and data storage.
- **Security Testing:** Security testing is crucial for identifying vulnerabilities and weaknesses in the blockchain system. To test system's resistance to common security threats, such as double spending and smart contract vulnerabilities.

BITS Pilani, Pilani Campus

12

Block Chain Testing Testing



- **Performance Testing:** Test under normal and peak load conditions. Measure key performance metrics, such as transaction throughput, latency, and scalability, to ensure that the system can handle the expected workload efficiently.
- **Scalability Testing:** To test how system handle increasing transaction volumes and user activity over time. Measure how the system scales in terms of capacity, throughput, and resource utilization.
- **Consensus Mechanism Testing:** Test consensus mechanisms that is used to agree on the validity of transactions and maintain the integrity of the ledger.
- **Fault Tolerance Testing:** Fault tolerance testing assesses the system's resilience to failures and disruptions, such as node failures, network partitions, and malicious attacks. Test that the system can recover gracefully from failures and maintain data consistency and availability.
- **Regression Testing:** Regression testing to ensures that new changes or updates to the blockchain system do not introduce regressions or unintended side effects.

BITS Pilani, Pilani Campus

13

Block Chain Testing Testing



- **User Acceptance Testing (UAT):** UAT involves testing the blockchain system with end users to validate that it meets their requirements and expectations.
- **Compliance Testing:** To ensure that the blockchain system complies with relevant laws, regulations, and standards, such as GDPR ((General Data Protection Regulation), HIPAA (Health Insurance Portability and Accountability Act), or financial regulations.

Overall, testing blockchain systems requires a combination of techniques and methodologies to address the unique challenges and requirements of decentralized, distributed ledger technology.

Testing approaches should ensure the reliability, security, and performance of the blockchain system.

BITS Pilani, Pilani Campus

14

AI/ML in S/W Testing

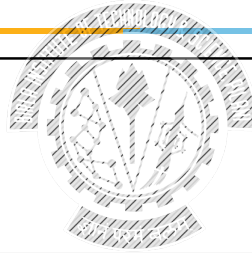


AI/ML can be used in S/W Testing in following ways

- Automated Smart Test Case Generation.
 - Test scripts, code can be generated using ChatGPT or other AI/ML Tools.
 - Basic code can be generated and then pruned to meet our needs
- Test Case Recommendation
 - AI/ML can learn data patterns and help in Test case generation with different inputs
- Test Data Generation
 - Large set of Test Data can be generated using AI/ML
 - Can take care of time zones, measurement units, Currency units, etc.
- Test Maintenance For Regression Testing
 - AI can automatically adjust the test scripts whenever there's a code change

BITS Pilani, Pilani Campus

15



AI/ML in S/W Testing



AI/ML can be used in S/W Testing in following ways

- Visual Testing
 - The computer can recognize extremely minor, pixel-level visual inconsistencies between screenshots as visual bugs.
 - Different computers render images differently, so the images in comparison may technically be different but not visually noticeable.
 - Certain UI elements change all the time, and test scripts don't understand that those changes happen for a good reason. They only understand that visual differences mean visual bugs.

BITS Pilani, Pilani Campus

16

AI/ML in S/W Testing Benefits



Benefits of AI/ML in S/W Testing

- With AI evolving, productivity of developers increases, leading to more code at more speed, and of course, more quality issues. Testers need AI too so that they can boost productivity and move at a corresponding speed.
- AI-based testing can totally help them resolve new challenges.
- Accelerates test creation and brings better test maintenance.
- More informed decisions thanks to recommendations from AI.
- Streamlined testing process helps everything run more smoothly and efficiently.

BITS Pilani, Pilani Campus

17

AI/ML in S/W Testing Challenges



Challenges of AI/ML in Software Testing

- **Training Data Quality:** AI/ML models require a large and diverse datasets. Not viable in early days.
- **Unforeseen Test Cases:** AI/ML models might miss certain scenarios if they have not been adequately trained.
- **Overfitting and Underfitting:** Overfitting occurs when a model is too specific to the training data and doesn't generalize well, while underfitting occurs when the model is too simple to capture patterns. Ensure to balance these two extremes when working with AI.
- **Model Drift:** Model becomes less effective due to changes in the software usage, known as model drift. This requires continuous data monitoring to combat.

BITS Pilani, Pilani Campus

18

AI/ML in S/W Testing Best Practices



Best Practices When Using AI/ML in Software Testing

- **Gain a foundational understanding of AI/ML systems as well as your workflows:** Understand your own workflows, then find out ways that you can integrate two of them with each other.
- **Be patient:** AIs take time to develop and learn.
- **Learn prompt engineering:** Provide well-structured and precise input prompts to generate accurate and relevant outputs from the models. Prompt engineering is all about bringing context, specifications, and boundaries to the table.
- **Remember that it is just a tool:** No matter what, AI is only a tool. Testers will not be replaced by AI, but rather, they will be empowered by the technology. Skills, experience, and creativity of the tester can extract best out of these tools..

BITS Pilani, Pilani Campus

19



Work Integrated Learning Programmes




SOFTWARE QUALITY ASSURANCE AND TESTING

Lecture -16

Harish Aggarwal

BITS Pilani
Pilani Campus

1

Re-Cap



Lecture - 01 - Definitions – S/W Quality, S/W Quality Assurance - Importance of QA - Causes of Defects - Business Models in S/W - S/W Cost, Quality Cost - Quality culture Lecture - 02 - Software Quality Models - McCall, IEEE 1061, ISO 25000 Series - Quality Requirements - Quality of Requirements - Frameworks - ISO/IEC/IEEE 12207 - Software Life Cycle Processes Lecture - 03 - Frameworks - CMMI-Development - ITIL Framework - Other Frameworks	Lecture - 04 - ISO 25010 – Quality Attributes - Functional Suitability, Performance, Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability Lecture - 05 - Software Testing Fundamentals - Software Verification and Validation - Types of Testing - Black Box, White Box Testing - Test Levels and Types – Unit, Integration, System Testing Lecture - 06 - Test Design Techniques - Combinatorial Testing - Boundary Value Analysis - Equivalence Class Partitioning Lecture - 07 - Test Methodology - Decision Table Testing Lecture - 08 - Revision
--	---

BITS Pilani, Pilani Campus

2

Re-Cap



Lecture - 09

- Test Execution Process
- Test Case Design
- Automation Testing
- Alpha and Beta Site Testing Programs
- Regression Testing Strategies

Lectures 10

- Reviews
 - Informal Reviews – Personal, Peer Formal
 - Reviews – Walkthrough, Inspection, Audit
- Audits - Internal, External, Third Party
- Project Assessment and Control Process
- Corrective Actions

Lectures 11

- Software configuration Management
 - Definitions, Benefits
- SCM vis-à-vis IEEE 828, ISO 12207, CMMi-Dev
- Baseline
- Branching
- Change Control

Lectures 12

- Introduction to Test Process Improvement
- Quality/Test Metrics

Lectures 13

- Test Organization
- Roles in Test

Lectures 14

- Agile Test Planning and Execution
- Six Sigma in S/W Testing
- Continuous Testing in DevOps

Lectures 15

- Future of SQA
- Testing in Block Chain
- AI/ML in Software Testing
- Career Roles in QA/Testing

BITS Pilani, Pilani Campus

3

Agenda



• Measurements

- As per ISO 12207
- As per PSM
- As per ISO/IEC/IEEE 15939
- As per CMMI Dev
- As per IEEE 730

BITS Pilani, Pilani Campus

4

Measurement



What is not measured, cannot be improved

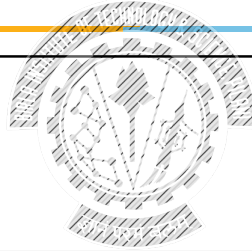
Key Roles related to Software Measurement

- **Understand and characterize:**

- Measures allow us to learn about software processes, products, and services.
- Establish baselines, standards, and business and technical objectives
- Document the software process models used
- Set improvement objectives for software processes, products, and services
- Better estimate effort and schedule costs for a specific project

BITS Pilani, Pilani Campus

5



Measurement



- **Evaluate:** Conduct cost/benefit analysis and determine if the objectives have been met
- **Control:** Control of resources, processes, products, and services by sounding alarms when control limits are surpassed, performance criteria are not met and standards are not followed
- **Predict:** when software processes are stable and under control, measures can be used to predict budgets, schedules, resources needed, risks, and even quality issues
- **Improve:** measures allow us to identify the root causes of defects and other inefficiencies where improvements can be proposed.

BITS Pilani, Pilani Campus

6

Measurement Process



- Measurement process as described in ISO 12207
- Practical software and systems measurement (PSM)
 - Initially developed to guide American Defense software projects
- ISO/IEC/IEEE 15939 standard on software measurement
 - Systems and Software Engineering - Measurement Process.
- Measurement as per CMMI point of view.
- Measurement as per IEEE 730.

BITS Pilani, Pilani Campus

7

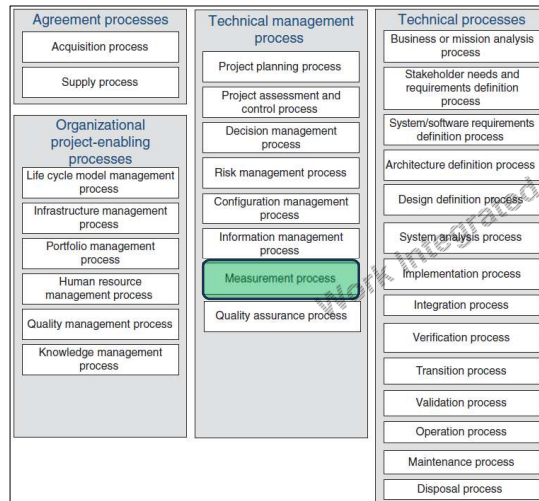
ISO/IEC/IEEE 12207 Software Life Cycle Processes



- Establishes a common framework for software life cycle processes.

Each process has –

- Title
- Purpose
- Outcome
- Activities
- Task



BITS Pilani, Pilani Campus

8

Measurement Process ISO 12207



Measurement Process

Purpose

- The purpose of the Measurement Process is to collect, analyze, and report data relating to the products developed and processes implemented within the organizational unit, to support effective management of the processes, and to objectively demonstrate the quality of the products.

Outcomes

As a result of successful implementation of the Measurement Process:

- a) The information needs of technical and management processes are identified;
- b) An appropriate set of measures, driven by the information needs are identified and/or developed;
- c) Measurement activities are identified and planned;
- d) The required data are collected, stored, analyzed, and the results interpreted;
- e) Information products are used to support decisions and provide an objective basis for communication;
- f) The Measurement Process and measures are evaluated; and
- g) Improvements are communicated to the Measurement Process owner.

BITS Pilani, Pilani Campus

9

Measurement Process ISO 12207



Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the Measurement Process.

Measurement planning. This activity consists of the following tasks:

1. The project shall describe the characteristics of the organization that are relevant to measurement.
2. The project shall identify and prioritize the information needs.
3. The project shall select and document measures that satisfy the information needs.
4. The project shall define data collection, analysis, and reporting procedures.
5. The project shall define criteria for evaluating the information products and the measurement process.
6. The project shall review, approve, and provide resources for measurement tasks.
7. The project shall acquire and deploy supporting technologies.

Measurement performance. This activity consists of the following tasks:

1. The project shall integrate procedures for data generation, collection, analysis and reporting into the relevant processes.
2. The project shall collect, store, and verify data.
3. The project shall analyze data and develop information products.
4. The project shall document and communicate results to the measurement users.

Measurement evaluation. This activity consists of the following tasks:

1. The project shall evaluate information products and the measurement process.
2. The project shall identify and communicate potential improvements.

BITS Pilani, Pilani Campus

10

THE PRACTICAL SOFTWARE AND SYSTEMS MEASUREMENT METHOD (PSM)



PSM Covers three perspectives

- (1) The project manager so as to provide a good understanding of the measures and how to use them to manage their project.
- (2) The technical staff that conducts measurement during planning and execution phases; and
- (3) The management team so that they can understand the measurement requirements associated with software.

BITS Pilani, Pilani Campus

11

THE PRACTICAL SOFTWARE AND SYSTEMS MEASUREMENT METHOD (PSM)



PSM – Nine principles

- (1) Use issues and objectives to drive the measurement requirements.
- (2) Define and collect measures based on the technical and management processes.
- (3) Collect and analyze data at a level of detail sufficient to identify and isolate problems.
- (4) Implement an independent analysis capability.
- (5) Use a systematic analysis process to trace the measures to the decisions.
- (6) Interpret the measurement results in the context of other project information.
- (7) Integrate measurement into the project management process throughout the life cycle.
- (8) Use the measurement process as a basis for objective communications.
- (9) Focus initially on project-level analysis.

BITS Pilani, Pilani Campus

12

THE PRACTICAL SOFTWARE AND SYSTEMS MEASUREMENT METHOD (PSM)



PSM – Ten Parts

- (1) Part 1, The Measurement Process
- (2) Part 2, Tailor Measures
- (3) Part 3, Measurement Selection and Specification Tables
- (4) Part 4, Apply Measures
- (5) Part 5, Measurement Analysis and Indicator
- (6) Part 6, Implement Process
- (7) Part 7, Evaluate Measurement
- (8) Part 8, Measurement Case Studies
- (9) Part 9, Supplemental Information, - glossary, acronyms, etc
- (10) Part 10, Department of Defense Implementation Guide

BITS Pilani, Pilani Campus

13

THE PRACTICAL SOFTWARE AND SYSTEMS MEASUREMENT METHOD (PSM)



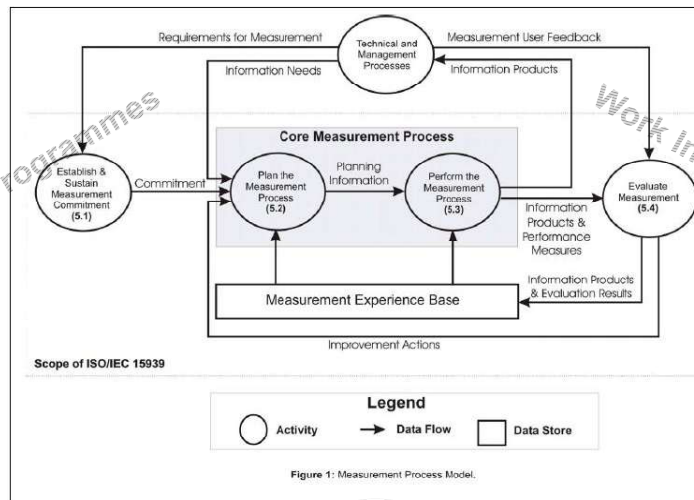
PSM – Category of information produced

1. Schedule
2. Resources and Costs
3. Software size and stability
4. Product Quality
5. Process Performance
6. Effectiveness of technology used
7. User Satisfaction

BITS Pilani, Pilani Campus

14

ISO/IEC/IEEE 15939 STANDARD



BITS Pilani, Pilani Campus

15

ISO/IEC/IEEE 15939 STANDARD



Systems and Software Engineering - Measurement Process

ISO 15939 defines a software measurement process that applies to both the systems and software engineering disciplines for software suppliers and acquirers

Outcome of Successful Measurement process –

1. Information needs are identified
2. An appropriate set of measures, based on the information needs, is identified or developed
3. Required data is collected, verified, and stored
4. The data is analyzed and the results interpreted
5. Information items provide objective information that supports decisions
6. Organizational commitment for measurement is sustained
7. Identified measurement activities are planned
8. The measurement process and measures are evaluated
9. Improvements are communicated to the measurement process owner.

BITS Pilani, Pilani Campus

16

ISO/IEC/IEEE 15939 STANDARD



Activity of Measurement process –

1. Activity 1: Establish and sustain measurement commitment

- Accept the requirements for measurement
- Assign resources

2. Activity 2: Prepare for measurement

- Define the measurement strategy
- Describe the characteristics of the organization that are relevant to measurement.
- Identify and prioritize the information needs
- Select and specify measures that satisfy the information needs
- Define data collection, analysis, access, and reporting procedures
- Define criteria for evaluating the information items and the measurement process
- Identify and plan for the enabling systems or services to be used
- Review, approve, and provide resources for measurement tasks
- Acquire and deploy supporting technologies

BITS Pilani, Pilani Campus

17

ISO/IEC/IEEE 15939 STANDARD



Outcome of Successful Measurement process –

3. Activity 3: Perform measurement

- Integrate procedures for data generation, collection, analysis, and reporting into the relevant processes
- Collect, store, and verify data
- Analyze data and develop information items
- Record results and inform the measurement users

4. Activity 4: Evaluate measurement

- Evaluate information products and the measurement process
- Identify potential improvements.

BITS Pilani, Pilani Campus

18

CMMI for Development



Level	Focus	Key process area	
5 Optimizing	Continuous process improvement	Organizational performance management causal analysis and resolution	
4 Quantitatively managed	Quantitative management	Organizational process performance quantitative project management	
3 Defined	Process standardization	Requirements development Technical solution Product integration Verification Validation Organizational process focus Organizational process definition Organizational training Integrated project management Risk management Decision analysis and resolution	
2 Managed	Basic project management	Requirement management Project planning Project monitoring and control Supplier agreement management Measurement and analysis Process and product quality assurance Configuration management	
1 Initial			

BITS Pilani, Pilani Campus

19

Measurement CMMI - Dev



Measurement and Analysis - The purpose of measurement and analysis (MA) is to develop and sustain a measurement capability used to support management information needs.

- SG 1 Align Measurement and Analysis Activities - Measurement objectives and activities are aligned with the identified information needs and objectives.
 - SP 1.1 Establish measurement objectives
 - SP 1.2 Specify measures
 - SP 1.3 Specify data collection and storage procedures
 - SP 1.4 Specify analysis procedures
- SG 2 Provide Measurement Results - Measurement results, which address identified information needs and objectives, are provided.
 - SP 2.1 Obtain measurement data
 - SP 2.2 Analyze measurement data
 - SP 2.3 Store data and results
 - SP 2.4 Communicate results

BITS Pilani, Pilani Campus

20

Measurement As per IEEE 730 SQAP



5.4.6 Measure products

5.4.6.1 Purpose

Determine whether product measurements demonstrate the quality of the products and conform to standards and procedures established by the project.

5.4.6.2 Outcomes

This activity shall produce the following outcomes:

- Software product measurements conform to project's processes and plans, and conform to standards and procedures established by the project or organization.
- Software product measurements accurately represent software product quality.
- Software product measurements are shared with project stakeholders.
- Software product measurements are performed on software products developed by the supplier as well as all of the supplier's subcontractors.
- Software product measurements are presented to management for review and potential corrective and preventive action.
- Non-conformances are raised when required measurement activities are not performed as defined in project plans.

5.4.6.3 Tasks

To accomplish this activity, the SQA function shall perform the following tasks:

- 1) Identify the standards and procedures established by the project or organization.
- 2) Determine whether proposed product measurements are consistent with standards and procedures established by the project.
- 3) Determine whether the proposed product measurements are representative of product quality attributes.
- 4) Analyze product measurement results to identify gaps and recommend improvements to close gaps between measurements and expectations.
- 5) Evaluate product measurement results to determine whether improvements implemented as a result of product quality measurements are effective.
- 6) Analyze product measurement procedures to confirm they are sufficient to satisfy the measurement requirements defined in project's processes and plans.
- 7) Perform Task 1 through Task 6, above, for software products developed by all subcontractors.

BITS Pilani, Pilani Campus

21

Measurement As per IEEE 730 SQAP



5.5.5 Measure processes

5.5.5.1 Purpose

Determine whether the process measurements support effective process management and conform to the project's processes and plans and conform to established standards and procedures.

5.5.5.2 Outcomes

This activity shall produce the following outcomes:

- Software process measurement activities conform to project's processes and plans.
- Software process measurement activities conform to standards and procedures established for the project or organization.
- Reports documenting software measurement results are prepared and reviewed with project stakeholders and management.
- Non-conformances are raised when software measurement activities do not conform to project's processes and plans.
- Software process measurements accurately represent software process quality.
- Software process measurements are shared with project stakeholders.
- Software process measurements are performed on all of the project's and subcontractors' processes.

5.5.5.3 Tasks

To accomplish this activity, the SQA function shall perform the following tasks:

- 1) Identify the standards and procedures established for the project or organization.
- 2) Evaluate whether the process measurement activities are being executed in conformance to the project's processes and plans.
- 3) Evaluate whether the process measurement activities conform to standards and procedures established by the project and the organization.
- 4) Analyze process measurement procedures to confirm they are sufficient to satisfy the measurement requirements defined in project's processes and plans.
- 5) Review the process measurement plan to determine whether the SQA function's measurement needs are addressed.
- 6) Analyze process measurement procedures to confirm they are sufficient to satisfy the measurement requirements defined in project plans and the contract.
- 7) For each subcontractor, perform Task 1 through Task 6, above, for each of their software processes.

BITS Pilani, Pilani Campus

22

Sample Compre Marks BreakUp

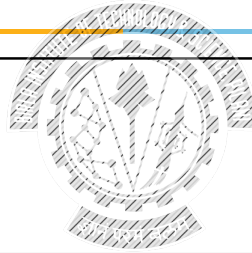


Regular			
Question	Marks	% Marks	Time
1	6	15.0%	22.5
2	4	10.0%	15
3	5	12.5%	18.75
4	6	15.0%	22.5
5	8	20.0%	30
6	5	12.5%	18.75
7	6	15.0%	22.5
Total	40		150

MakeUp			
Question	Marks	% Marks	Time
1	5	12.5%	18.75
2	5	12.5%	18.75
3	5	12.5%	18.75
4	6	15.0%	22.5
5	6	15.0%	22.5
6	6	15.0%	22.5
7	7	17.5%	26.25
Total	40		150

BITS Pilani, Pilani Campus

23



Feedback



- Feedback on mail
harish.aggarwal@pilani.bits-pilani.ac.in
- Feedback can be on -
 - Evaluation – Quiz, Paper Setting, etc
 - Course Content – Syllabus, Depth, Breadth, etc
 - Teaching Style.

BITS Pilani, Pilani Campus

24